



Εθνικό Μετσόβιο Πολυτεχνείο
Σχολή Ηλεκτρολόγων Μηχανικών
και Μηχανικών Υπολογιστών
Τομέας Τεχνολογίας Πληροφορικής
και Υπολογιστών

Χαρακτηρισμός των μετροπρογραμμάτων Koala

ΔΙΠΛΩΜΑΤΙΚΗ ΕΡΓΑΣΙΑ

ΓΕΩΡΓΙΟΣ ΚΑΟΥΚΗΣ

Επιβλέπων : Γεώργιος Γκούμας
Καθηγητής, Ε.Μ.Π.

Συν-Επιβλέπων : Νίκος Βασιλάκης
Επικ. Καθηγητής, Πανεπιστήμιο Μπράουν

Αθήνα, Φεβρουάριος 2026



Εθνικό Μετσόβιο Πολυτεχνείο
Σχολή Ηλεκτρολόγων Μηχανικών
και Μηχανικών Υπολογιστών
Τομέας Τεχνολογίας Πληροφορικής
και Υπολογιστών

Χαρακτηρισμός των μετροπρογραμμάτων Koala

ΔΙΠΛΩΜΑΤΙΚΗ ΕΡΓΑΣΙΑ

ΓΕΩΡΓΙΟΣ ΚΑΟΥΚΗΣ

Επιβλέπων : Γεώργιος Γκούμας
Καθηγητής, Ε.Μ.Π.

Συν-Επιβλέπων : Νίκος Βασιλάκης
Επικ. Καθηγητής, Πανεπιστήμιο Μπράουν

Εγκρίθηκε από την τριμελή εξεταστική επιτροπή την 27η Φεβρουαρίου 2026.

.....
Γεώργιος Γκούμας
Καθηγητής, Ε.Μ.Π.

.....
Νεκτάριος Κοζύρης
Καθηγητής, Ε.Μ.Π.

.....
Νίκος Βασιλάκης
Επικ. Καθηγητής, Πανεπιστήμιο Μπράουν

Αθήνα, Φεβρουάριος 2026

.....
Γεώργιος Καούκης

Διπλωματούχος Ηλεκτρολόγος Μηχανικός και Μηχανικός Υπολογιστών Ε.Μ.Π.

Copyright © Γεώργιος Καούκης, 2026.

Με επιφύλαξη παντός δικαιώματος. All rights reserved.

Απαγορεύεται η αντιγραφή, αποθήκευση και διανομή της παρούσας εργασίας, εξ ολοκλήρου ή τμήματος αυτής, για εμπορικό σκοπό. Επιτρέπεται η ανατύπωση, αποθήκευση και διανομή για σκοπό μη κερδοσκοπικό, εκπαιδευτικής ή ερευνητικής φύσης, υπό την προϋπόθεση να αναφέρεται η πηγή προέλευσης και να διατηρείται το παρόν μήνυμα. Ερωτήματα που αφορούν τη χρήση της εργασίας για κερδοσκοπικό σκοπό πρέπει να απευθύνονται προς τον συγγραφέα.

Οι απόψεις και τα συμπεράσματα που περιέχονται σε αυτό το έγγραφο εκφράζουν τον συγγραφέα και δεν πρέπει να ερμηνευθεί ότι αντιπροσωπεύουν τις επίσημες θέσεις του Εθνικού Μετσόβιου Πολυτεχνείου.

Περίληψη

Το ΚΟΑΛΑ αποτελεί μια σουίτα μετροπρογραμμάτων επικεντρωμένη στην έρευνα αξιολόγησης επιδόσεων για το κέλυφος UNIX και Linux. Συνδυάζει μια συστηματική συλλογή από ποικιλόμορφα προγράμματα κελύφους προερχόμενα από πραγματικά σενάρια χρήσης, ρεαλιστικά σύνολα εισόδων διαφόρων ειδών που υποστηρίζουν μικρής και μεγάλης κλίμακας εκτελέσεις για τα προγράμματα αυτά, εκτενή ανάλυση και χαρακτηρισμό που βοηθούν στην κατανόησή τους, καθώς και επιπλέον υποδομή και εργαλεία που στοχεύουν στη χρηστικότητα και την αναπαραγωγικότητα στην έρευνα συστημάτων. Τα μετροπρογράμματα ΚΟΑΛΑ εκτελούν ένα ευρύ φάσμα κοινών εργασιών κελύφους· αξιοποιούν όλα τα κύρια γλωσσικά χαρακτηριστικά του κελύφους POSIX· χρησιμοποιούν μια ποικιλία από εργαλεία POSIX, GNU Coreutils και λογισμικό τρίτων· λειτουργούν σε εισόδους μεταβαλλόμενου μεγέθους και σύνθεσης· και παρουσιάζουν ένα ευρύ φάσμα προφίλ εκτέλεσης, όπως προκύπτει από δυναμική ανάλυση, καθιστώντας τα ιδιαίτερα ετερογενή και κατάλληλα για την αξιολόγηση ποικίλων στρατηγικών βελτιστοποίησης. Η εφαρμογή του ΚΟΑΛΑ σε τέσσερα συστήματα που στοχεύουν στην επιτάχυνση της εκτέλεσης των προγραμμάτων κελύφους παρέχει μια πιο ολοκληρωμένη εικόνα των μεταξύ τους συμβιβασμών, γενικεύει τα κύρια ευρήματά τους και συμβάλλει σε μια βαθύτερη κατανόηση των συστημάτων αυτών.

Λέξεις κλειδιά

Κέλυφος, UNIX, Linux, Μετροπρογράμματα, POSIX, Αξιολόγηση Επιδόσεων

Abstract

KOALA is a benchmark suite designed for performance research for the UNIX and Linux shell. It combines a systematic collection of diverse shell scripts derived from real-world use cases, realistic input sets of various kinds that support small- and large-scale executions, extensive analysis and characterization that aid in their understanding, as well as additional infrastructure and tools aimed at usability and reproducibility in systems research. The KOALA benchmarks execute a broad spectrum of common shell tasks; they leverage all main language features of the POSIX shell; they use a variety of POSIX tools, GNU Coreutils, and third-party software; they operate on inputs of varying size and composition; and they exhibit a wide range of performance characteristics, captured through dynamic analysis, making them heterogeneous and suitable for evaluating a variety of optimization strategies. The application of KOALA on four systems aimed at accelerating the execution of shell programs provides a more comprehensive picture of the trade-offs among them, generalizes their main findings, and contributes to a deeper understanding of these systems.

Keywords

Shell, UNIX, Linux, Benchmarks, POSIX, Performance Evaluation

Ευχαριστίες

Θα ήθελα να εκφράσω την ειλικρινή μου ευγνωμοσύνη σε όλους όσους με στήριξαν κατά την ολοκλήρωση αυτής της διπλωματικής εργασίας και συνέβαλαν στο να κάνουν τα χρόνια μου ως φοιτητής τόσο ουσιαστικά και δημιουργικά.

Πρωτίστως, θέλω να ευχαριστήσω τον επιβλέποντά μου, Καθ. Γεώργιο Γκούμα, για την εμπιστοσύνη του, την καθοδήγησή του και για την ευκαιρία που μου έδωσε να πραγματοποιήσω τη διπλωματική μου εργασία στο Εργαστήριο Υπολογιστικών Συστημάτων. Η στήριξή του έχει επηρεάσει σημαντικά την ακαδημαϊκή μου εξέλιξη, καθώς τα μαθήματά του ήταν καθοριστικά για την αναζωπύρωση του ενθουσιασμού μου για τον τομέα κατά το τρίτο έτος των σπουδών μου.

Είμαι επίσης βαθύτατα ευγνώμων στον Καθ. Νίκο Βασιλάκη για την ευκαιρία που μου έδωσε να πραγματοποιήσω αυτή τη διπλωματική εργασία σε συνεργασία με την ομάδα Atlas του Πανεπιστημίου Brown. Η συνεχής ανατροφοδότηση και η εύστοχη καθοδήγησή του υπήρξαν καθοριστικές για την ολοκλήρωση αυτής της εργασίας και διαμόρφωσαν τα πρώτα στάδια της ερευνητικής μου πορείας. Ιδιαίτερες ευχαριστίες οφείλω στον Ευάγγελο Λάμπρου, για τη διαρκή υποστήριξή του, τις γόνιμες συζητήσεις και την πρακτική του βοήθεια, καθώς και στον Δρ. Lukas Lazarek, του οποίου οι συνεισφορές και τα εύστοχα σχόλια διαμόρφωσαν σημαντικά την κατεύθυνση και τη σαφήνεια αυτού του έργου.

Καθώς η παρούσα διπλωματική εργασία εκπονήθηκε στο πλαίσιο μιας ευρύτερης ερευνητικής προσπάθειας, θα ήθελα επίσης να ευχαριστήσω τον Ethan Williams για τη σημαντική και εκτενή συνεισφορά του, καθώς και τους υπόλοιπους συν-συγγραφείς του άρθρου KOALA—Zhuoxuan Zhang, Καθ. Michael Greenberg και Καθ. Κωνσταντίνο Καλλά—για τη συνεργασία τους καθ' όλη τη διάρκεια του ευρύτερου έργου.

Επιπλέον, ευχαριστώ θερμά τους συναδέλφους μου από την ερευνητική μονάδα Archimedes, την Ελένη, τον Γιάννη και τη Δανάη, για την υπομονή και τη στήριξή τους κατά τη διάρκεια της απαιτητικής διαδικασίας συγγραφής και ολοκλήρωσης της παρούσας διπλωματικής.

Θα ήθελα επίσης να εκφράσω τις ευχαριστίες μου στον Παναγιώτη, τον «ξάδελφό» μου και συνάδελφο απόφοιτο της Σχολής Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών, ο οποίος με βοήθησε σταθερά να διατηρώ την προσήλωσή μου και με στήριξε καθ' όλη τη διάρκεια των σπουδών μας στο ΕΜΠ, τον Φίλιππο, που πέρα από πολλές εξόδους μοιραστήκαμε και τις τελευταίες εξεταστικές, καθώς και στους συναδέλφους «compañeros», Αναστασία, Γιώργο, Νικόλα, Αθηνά και Ναταλία. Ευχαριστώ ακόμα τον Γιάννη, τον Νικόλα, τον Θέμη, τη Ναταλία, τον Ανδρόνικο, τη Λυδία, καθώς και όλους τους υπόλοιπους φίλους, συγγενείς και συνεργάτες, που—ο καθένας με τον δικό του τρόπο—φρόντισαν ώστε τα φοιτητικά μου χρόνια να είναι γεμάτα με τόσες χαρούμενες αναμνήσεις.

Πάνω απ' όλα, θα ήθελα να ευχαριστήσω τους γονείς μου, Ζαφείρη και Μαρία, των οποίων η ατελείωτη υπομονή, η αδιάκοπη στήριξη και η ακλόνητη πίστη σε εμένα αποτέλεσαν το θεμέλιο όλων των προσπαθειών μου.

Γεώργιος Καούκης,

Αθήνα, 27η Φεβρουαρίου 2026

Περιεχόμενα

Περίληψη	5
Abstract	7
Ευχαριστίες	9
Περιεχόμενα	11
Κατάλογος πινάκων	13
Κατάλογος σχημάτων	15
Κατάλογος κωδίκων	17
1. Εισαγωγή	19
2. Υπόβαθρο	21
2.1 Το Κέλυφος UNIX	21
2.2 Προσπάθειες Επιτάχυνσης και Βελτιστοποίησης	24
2.3 Αξιολόγηση Υπολογιστικών Συστημάτων	27
3. Σχεδιασμός και Υλοποίηση της Σουίτας ΚΟΑΛΑ	29
3.1 Σχεδιαστικές Αρχές	29
3.2 Παράδειγμα Χρήσης	30
3.3 Επισκόπηση της Σουίτας	31
3.4 Υποδομή και Διαμόρφωση Εκτέλεσης	35
4. Συντακτική Ανάλυση της Σουίτας ΚΟΑΛΑ	39
4.1 Μεθοδολογία	39
4.2 Συντακτικά Χαρακτηριστικά και Μοτίβα	39
4.3 Ανάλυση Εντολών και Συνιστωσών	41
4.4 Σύνοψη	42
5. Δυναμική Ανάλυση και Πειραματική Διαμόρφωση	43
5.1 Πειραματική Διαμόρφωση και Μεθοδολογία	43
5.2 Αποτελέσματα Δυναμικού Χαρακτηρισμού	45
5.3 Σύνοψη Δυναμικών Ευρημάτων	50
6. Αξιολόγηση Συστημάτων Βελτιστοποίησης	53
6.1 Κινητήριο Παράδειγμα: Το Σενάριο weather	53
6.2 Shark: Στατική Βελτιστοποίηση Εισόδου/Εξόδου	53
6.3 GNU parallel: Χειροκίνητος Παραλληλισμός	55
6.4 PASH: Παραλληλισμός Ροών Δεδομένων	57
6.5 hS: Εκτέλεση Εκτός Σειράς	58

6.6	Συγκριτική Σύνοψη Αξιολόγησης	60
7.	Συζήτηση	63
7.1	Διαφορετικές Στρατηγικές Επιτάχυνσης και τα Όριά τους	63
7.2	Το Κέλυφος ως Γλώσσα Ενορχήστρωσης	63
7.3	Πολυμορφία Δυναμικής Συμπεριφοράς	64
7.4	Η Αξία της Συστηματικής Αξιολόγησης	64
7.5	Συμπέρασμα	64
8.	Μελλοντικό Έργο και Συμπεράσματα	65
8.1	Μελλοντικό Έργο	65
8.2	Τελικά Συμπεράσματα	66
	Βιβλιογραφία	67
	Παράρτημα	73
A.	Παραδείγματα Σεναρίων Υποδομής για το Σύνολο Μετροπρογραμμάτων weather	73
B.	Πλήρες Πρόγραμμα Κελύφους που Παράγεται από το ΡΑΣΗ για το παράδειγμα weather.	79
C.	Κώδικας Συγκριτικής Αξιολόγησης	81
C.1	Μετροπρόγραμμα count_trigrams	81
C.2	Μετροπρόγραμμα covid	82
C.3	Μετροπρόγραμμα spell	83
D.	Συγκριτικά Διαγράμματα Επιτάχυνσης	85

Κατάλογος πινάκων

3.1	Σύνοψη των μετροπρογραμμάτων της σουίτας KOALA.	32
3.2	Εκτιμώμενος χρόνος ενσωμάτωσης ανά σύνολο.	36
5.1	Συνολικός αριθμός κλήσεων συστήματος και μέγιστος αριθμός ανοικτών περιγραφών αρχείων ανά σύνολο μετροπρογραμμάτων.	50
6.1	Εσωτερική διαχείριση του κώδικα του σεναρίου weather από τον χρονοδρομολογητή του hS.	60

Κατάλογος σχημάτων

2.1	Επισκόπηση του PASH.	25
2.2	Επισκόπηση του hS.	26
3.1	Ανάλυση Κύριων Συνιστωσών (PCA).	34
3.2	Αρχιτεκτονική Διεπαφής Μετροπρογραμμάτων.	35
4.1	Θερμικός χάρτης συντακτικών χαρακτηριστικών.	40
4.2	Κατανομή εντολών.	41
5.1	Ανάλυση χρόνου εκτέλεσης.	46
5.2	Ανάλυση κατανάλωσης μνήμης.	47
5.3	Ανάλυση Εισόδου/Εξόδου (I/O).	49
6.1	Σχετικές επιταχύνσεις του Shark στα μετροπρογράμματα της σουίτας KOALA.	55
6.2	Σχετικές επιταχύνσεις του GNU parallel στα μετροπρογράμματα της σουίτας KOALA.	57
6.3	Σχετικές επιταχύνσεις του PASH στα μετροπρογράμματα της σουίτας KOALA.	59
6.4	Σχετικές επιταχύνσεις του hS στα μετροπρογράμματα της σουίτας KOALA.	60
D.1	Συγκριτική επισκόπηση επιταχύνσεων.	85

Κατάλογος κωδίκων

2.1	Οκνηρία σε ρεαλιστικό σενάριο επεξεργασίας καταλόγων.	22
3.1	Παράδειγμα ανάλυσης μετεωρολογικών δεδομένων στο KOALA.	30
6.1	Το σενάριο <code>weather</code> μετασχηματισμένο βάσει του <code>Shark</code>	54
6.2	Το σενάριο <code>weather</code> μετασχηματισμένο βάσει του <code>GNU parallel</code>	56
6.3	Απλοποιημένη σημασιολογική αναπαράσταση του σεναρίου <code>weather</code> με χρήση του <code>PASh</code>	58
6.4	Απλοποιημένη σημασιολογική αναπαράσταση του σεναρίου <code>weather</code> με χρήση του <code>hS</code>	59
A.1	Παράδειγμα σεναρίου <code>install.sh</code> για το σύνολο μετροπρογραμμάτων <code>weather</code>	73
A.2	Παράδειγμα σεναρίου <code>fetch.sh</code> για το σύνολο μετροπρογραμμάτων <code>weather</code>	74
A.3	Παράδειγμα σεναρίου <code>execute.sh</code> για το σύνολο μετροπρογραμμάτων <code>weather</code>	75
A.4	Παράδειγμα σεναρίου <code>validate.sh</code> για το σύνολο μετροπρογραμμάτων <code>weather</code>	77
A.5	Παράδειγμα σεναρίου <code>clean.sh</code> για το σύνολο μετροπρογραμμάτων <code>weather</code>	77
B.1	Πλήρες πρόγραμμα κελύφους που παράγεται από το <code>PASh</code> για το παράδειγμα <code>weather</code> (<code>-width=2</code>).	80
C.1	Το αρχικό σενάριο <code>count_trigrams</code>	81
C.2	Το σενάριο <code>count_trigrams</code> βελτιστοποιημένο κατά <code>Shark</code>	82
C.3	Το πρώτο σενάριο του συνόλου <code>covid</code>	82
C.4	Το πρώτο σενάριο του συνόλου <code>covid</code> μετασχηματισμένο μέσω <code>GNU parallel</code>	83
C.5	Το αρχικό σενάριο <code>spell</code>	83
C.6	Το σενάριο <code>spell</code> μετασχηματισμένο μέσω <code>GNU parallel</code>	84

Κεφάλαιο 1

Εισαγωγή

Το κέλυφος UNIX είναι δημοφιλές, ευέλικτο και ισχυρό, με ποικιλία εφαρμογών που εκτείνονται από την επεξεργασία δεδομένων και τη διαχείριση συστημάτων έως τη βιοπληροφορική και τη συνεχή ολοκλήρωση και ανάπτυξη. Παράλληλα, το κέλυφος έχει συγκεντρώσει σημαντικό ερευνητικό ενδιαφέρον, με πρόσφατη ακαδημαϊκή δραστηριότητα [13, 23, 24, 26, 32, 42, 58, 63, 65, 73] να έχει οδηγήσει στην ανάπτυξη αρκετών βελτιστοποιητών που στοχεύουν στην επιτάχυνση προγραμμάτων κελύφους μέσω παράλληλων συστημάτων [32, 63, 73], κατανεμημένων συστημάτων [27, 47, 58], και άλλων μορφών κλιμάκωσης [24, 40, 43, 65].

Παρ' όλα αυτά, υπάρχει ένα σημαντικό πρόσκομμα για την αποτελεσματική διεξαγωγή έρευνας γύρω από το κέλυφος: δεν υπάρχει κάποια καθιερωμένη σουίτα μετροπρογραμμάτων για την αξιολόγηση των συστημάτων αυτών, με αποτέλεσμα να είναι δύσκολο να αξιολογηθούν νέα συστήματα με ρεαλιστικό και συγκρίσιμο τρόπο [6]. Έτσι, οι ερευνητές συχνά αναγκάζονται να δημιουργούν αυτοσχέδιες μικρο-δοκιμές αξιολόγησης (microbenchmarks), να συλλέγουν αποσπασματικά παραδείγματα από αποθετήρια ή να βασίζονται σε τυποποιημένες δοκιμές που εστιάζουν κυρίως στη συμπεριφορική ισοδυναμία και όχι στην απόδοση. Κατά συνέπεια, η έρευνα στο πεδίο στερείται βασικών πλεονεκτημάτων της συστηματικής συγκριτικής αξιολόγησης, όπως η επαναληψιμότητα, η αναπαραγωγιμότητα και η δίκαιη σύγκριση μεταξύ διαφορετικών συστημάτων.

Το KOALA είναι μια σουίτα μετροπρογραμμάτων προσανατολισμένη στην έρευνα για την αξιολόγηση επιδόσεων [37], η οποία στοχεύει στο κέλυφος UNIX και στο λειτουργικό σύστημα Linux, συνδυάζοντας συστηματική συλλογή προγραμμάτων, εκτενή χαρακτηρισμό και επαναχρησιμοποίηση υποδομή. Για την επίτευξη αυτού του στόχου, το KOALA παρέχει μια ποικίλη συλλογή από 126 πραγματικά προγράμματα κελύφους που εκτελούν εργασίες οι οποίες απαντώνται συχνά στην πράξη. Προσφέρει ρεαλιστικά δεδομένα εισόδου σε τρεις κλίμακες (ελάχιστη, μικρή και πλήρη), καθώς και αυτοματοποιημένα εργαλεία για την εγκατάσταση εξαρτήσεων, την εκτέλεση των μετροπρογραμμάτων, την επικύρωση των αποτελεσμάτων και την αναφορά της απόδοσης.

Η παρούσα διπλωματική εργασία αποτελεί υποσύνολο της ευρύτερης ερευνητικής προσπάθειας που παρουσιάστηκε στη δημοσίευση [37] στο USENIX ATC '25, εστιάζοντας ειδικά στον χαρακτηρισμό των μετροπρογραμμάτων και στην αξιολόγηση συστημάτων βελτιστοποίησης. Συγκεκριμένα, αναλύονται τα συντακτικά τους χαρακτηριστικά, η κατανομή στοιχείων POSIX και εντολών, καθώς και τα δυναμικά τους χαρακτηριστικά, όπως η συμπεριφορά κατά τον χρόνο εκτέλεσης, τα μοτίβα εισόδου/εξόδου (I/O) και η δυναμική των διεργασιών. Με αυτόν τον τρόπο παρέχεται ένας εκτενής χαρακτηρισμός που αναδεικνύει τα διαφορετικά προφίλ των σεναρίων και αποτυπώνει την ποικιλομορφία της σουίτας. Επιπλέον, η εργασία διερευνά τις πρακτικές προεκτάσεις της χρήσης του KOALA μέσω της αξιολόγησης τεσσάρων συστημάτων βελτιστοποίησης κελύφους: των Shark [6], GNU parallel [51], PASH [32, 74] και hS [40]. Πιο συγκεκριμένα, αναλύονται η προσπάθεια υιοθέτησης και οι συντακτικοί μετασχηματισμοί που απαιτούνται για την εφαρμογή αυτών των συστημάτων σε πραγματικά προγράμματα κελύφους, ενώ παρουσιάζονται οι επιταχύνσεις που επιτυγχάνονται, τα πλεονεκτήματα και τα μειονεκτήματα κάθε προσέγγισης. Τα αποτελέσματα δείχνουν ότι, παρότι τα συστήματα μπορούν να επιτύχουν σημαντικές επιταχύνσεις, η αποτελεσματικότητά τους καθορίζεται σε μεγάλο βαθμό από τη δομή και τα ιδιαίτερα χαρακτηριστικά κάθε προγράμματος.

Η δομή της παρούσας διπλωματικής εργασίας οργανώνεται ως εξής: αρχικά παρουσιάζεται το απαραίτητο θεωρητικό υπόβαθρο σχετικά με το κέλυφος UNIX, τις υπάρχουσες προσεγγίσεις βελτι-

στοποίησης και το τοπίο της συγκριτικής αξιολόγησης υπολογιστικών συστημάτων (Κεφάλαιο 2), ενώ στη συνέχεια περιγράφεται ο σχεδιασμός και η υλοποίηση της σουίτας ΚΟΑΛΑ, με ανάλυση των σχεδιαστικών αρχών, των υπολογιστικών πεδίων των μετροπρογραμμάτων και της υποδομής εκτέλεσής τους (Κεφάλαιο 3). Έπειτα ακολουθεί η στατική ανάλυση της σουίτας, η οποία εξετάζει τη συντακτική δομή, την κατανομή των εντολών και την ποικιλομορφία των προγραμματιστικών μοτίβων που συναντώνται στην πράξη (Κεφάλαιο 4), ενώ κατόπιν παρουσιάζονται τα αποτελέσματα του δυναμικού χαρακτηρισμού, τα οποία εμβαθύνουν στο προφίλ εκτέλεσης της σουίτας αναλύοντας τη χρήση του επεξεργαστή, την κατανάλωση μνήμης, τα μοτίβα εισόδου/εξόδου και την αλληλεπίδραση με το λειτουργικό σύστημα (Κεφάλαιο 5). Στη συνέχεια παρατίθεται η συγκριτική αξιολόγηση των τεσσάρων επιταχυντών (Κεφάλαιο 6), και η εργασία ολοκληρώνεται με τη σύνθεση των ευρημάτων, τη συζήτηση των σχεδιαστικών συμβιβασμών που αναδείχθηκαν (Κεφάλαιο 7), καθώς και την παρουσίαση του προτεινόμενου μελλοντικού έργου και των τελικών συμπερασμάτων (Κεφάλαιο 8). Τέλος, η υποδομή και τα μετροπρογράμματα της σουίτας ΚΟΑΛΑ διατίθενται ελεύθερα με άδεια MIT στο <https://github.com/kbensh/koala>.

Κεφάλαιο 2

Υπόβαθρο

Σε αυτό το κεφάλαιο παρουσιάζεται το απαραίτητο υπόβαθρο για την κατανόηση της συμπεριφοράς και της βελτιστοποίησης προγραμμάτων κελύφους UNIX. Αρχικά, αναλύεται το περιβάλλον του κελύφους και οι βασικές αφαιρέσεις που διέπουν την εκτέλεση και τη σύνθεση εντολών. Στη συνέχεια, εξετάζονται σε βάθος τα σύγχρονα συστήματα βελτιστοποίησης, αναδεικνύοντας τις τεχνικές που εφαρμόζουν, καθώς και τους περιορισμούς τους σε πραγματικά προγράμματα. Έπειτα, παρουσιάζεται το ευρύτερο τοπίο της αξιολόγησης υπολογιστικών συστημάτων, με έμφαση στις καθιερωμένες σουίτες μετροπρογραμμάτων και στις πρακτικές συγκριτικής αξιολόγησης. Τέλος, το κεφάλαιο αναδεικνύει το υφιστάμενο ερευνητικό κενό στην αξιολόγηση επιδόσεων για το κέλυφος και τεκμηριώνει την ανάγκη για μια ρεαλιστική, αναπαραγώγιμη και συστηματική προσέγγιση, η οποία καλύπτεται από τη σουίτα KOALA.

2.1 Το Κέλυφος UNIX

Το κέλυφος UNIX αποτελεί τον ακρογωνιαίο λίθο της αλληλεπίδρασης χρήστη-συστήματος εδώ και δεκαετίες, λειτουργώντας ταυτόχρονα ως ένας ισχυρός διαδραστικός διερμηνέας εντολών και ως μια εκφραστική γλώσσα προγραμματισμού υψηλού επιπέδου. Σύμφωνα με τη φιλοσοφία του UNIX [59], παρέχει απλούς και γενικούς μηχανισμούς σύνθεσης μικρών, ανεξάρτητων εργαλείων σε μεγαλύτερα προγράμματα. Μέσω αυτού του περιβάλλοντος, οι χρήστες έχουν τη δυνατότητα να συνθέτουν και να εκτελούν πολύπλοκα προγράμματα, συνδυάζοντας μικρότερες, εξειδικευμένες και ανεξάρτητες εντολές σε σύνθετες ροές επεξεργασίας.

Η λειτουργία και η συμπεριφορά του κελύφους καθορίζονται από το πρότυπο POSIX, το οποίο θεσπίζει τους κανόνες σύνταξης και τις σημασιολογικές ιδιότητες, εξασφαλίζοντας τη διαλειτουργικότητα και τη φορητότητα προγραμμάτων μεταξύ διαφορετικών λειτουργικών συστημάτων και αρχιτεκτονικών [60]. Σε αντίθεση με τις παραδοσιακές γλώσσες προγραμματισμού γενικού σκοπού, όπως η C ή η Java, το κέλυφος δεν σχεδιάστηκε με πρωταρχικό στόχο την υλοποίηση πολύπλοκων αλγορίθμων ή δομών δεδομένων, αλλά για τον αποδοτικό συντονισμό και τη διασύνδεση υπάρχοντων προγραμμάτων, εργαλείων και υπηρεσιών συστήματος. Με αυτόν τον τρόπο, λειτουργεί ως ένα *συγκολλητικό* επίπεδο λογισμικού, επιτρέποντας στους προγραμματιστές να κατασκευάζουν γρήγορα λειτουργικούς αγωγούς επεξεργασίας δεδομένων, αξιοποιώντας έτοιμα και δοκιμασμένα δομικά στοιχεία. Η εκφραστικότητά του έχει παρατηρηθεί στη βιβλιογραφία, καθώς εργασίες που απαιτούν εκτενή υλοποίηση σε γλώσσες όπως η Java μπορούν συχνά να εκφραστούν με μία μόνο εντολή κελύφους [25].

Σύγχρονη Χρήση και Σημασία Παρά την εμφάνιση νεότερων γλωσσών και εργαλείων, το κέλυφος διατηρεί αμείωτη τη σπουδαιότητά του και παραμένει αναπόσπαστο τμήμα της σύγχρονης πληροφορικής υποδομής. Κατατάσσεται συστηματικά ανάμεσα στις πιο δημοφιλείς γλώσσες προγραμματισμού [21], ενώ πρόσφατες μελέτες καταδεικνύουν σημαντική αύξηση της χρήσης του [40]. Το κέλυφος είναι πανταχού παρόν σε κρίσιμους τομείς, όπως η διαχείριση συστημάτων, η αυτοματοποίηση επαναλαμβανόμενων εργασιών, η μαζική επεξεργασία δεδομένων, η βιοπληροφορική, καθώς και η ενορχήστρωση υπολογιστικών ροών σε κατανεμημένα περιβάλλοντα και υποδομές νέφους. Ιδιαίτερα στο σύγχρονο οικοσύστημα ανάπτυξης λογισμικού, που κυριαρχείται από τεχνολογίες όπως το Docker

```
1 #!/bin/bash
2 mkfifo pipe1 pipe2
3 grep "Failed" log1 > pipe1 &
4 grep "Failed" log2 > pipe2 &
5 cat pipe1 pipe2 | sort | uniq -c
```

Κώδικας 2.1: Οκνηρία σε ρεαλιστικό σενάριο επεξεργασίας καταλόγων. Η εντολή `cat` διαβάζει πρώτα από το `t1` και μόνο αφού ολοκληρώσει την ανάγνωση αυτού του FIFO προχωρά στο `t2`. Επομένως, η δεύτερη διεργασία `grep` μπλοκάρει, επειδή δεν υπάρχει ακόμη διεργασία που να διαβάζει από το `t2`, και συνεχίζει μόνο όταν τελειώσει η πρώτη `grep`.

και πλατφόρμες όπως το Kubernetes, το κέλυφος αποτελεί το καθιερωμένο πρότυπο για τον ορισμό σημείων εισόδου, την προετοιμασία περιβαλλόντων και την εκτέλεση σεναρίων συνεχούς ολοκλήρωσης [45]. Ως εκ τούτου, τα προγράμματα κελύφους δεν αποτελούν απλώς βοηθητικά εργαλεία, αλλά εκτελούνται συχνά σε υποδομές μεγάλης κλίμακας, επηρεάζοντας άμεσα την αξιοπιστία, την ταχύτητα και τη συνολική απόδοση κρίσιμων συστημάτων παραγωγής. Η εκτεταμένη αυτή χρήση καθιστά την εις βάθος κατανόηση των μηχανισμών του, καθώς και τη βελτιστοποίηση της εκτέλεσής του, ιδιαίτερα σημαντική για τη σύγχρονη έρευνα συστημάτων.

Βασικές Αφαιρέσεις και Ροές Δεδομένων Η εκφραστική ισχύς και η ευελιξία του κελύφους απορρέουν από ένα σύνολο απλών, αλλά ιδιαίτερα ισχυρών αφαιρέσεων, οι οποίες ενσαρκώνουν τη φιλοσοφία του UNIX για τη σύνθεση μικρών και εξειδικευμένων εργαλείων. Κεντρικό ρόλο διαδραματίζουν οι ροές δεδομένων, οι οποίες υλοποιούνται ως σειριακές ακολουθίες bytes χωρίς εγγενή δομή και μεταφέρονται μεταξύ διεργασιών μέσω μηχανισμών όπως οι αγωγοί ή τα αρχεία. Οι ανώνυμοι αγωγοί επιτρέπουν την άμεση σύνδεση της τυπικής εξόδου (`stdout`) μιας εντολής με την τυπική είσοδο (`stdin`) μιας άλλης, δημιουργώντας σωληνώσεις (`pipelines`), ενώ οι επώνυμοι αγωγοί (UNIX FIFOs) προσφέρουν μόνιμα κανάλια επικοινωνίας στο σύστημα αρχείων. Σε αυτό το πλαίσιο, ο πυρήνας του λειτουργικού συστήματος αναλαμβάνει τον ρόλο της διαχείρισης, της δρομολόγησης και του συγχρονισμού των δεδομένων, επιτρέποντας την ταυτόχρονη εκτέλεση των συνδεδεμένων εντολών χωρίς ο χρήστης να χρειάζεται να διαχειριστεί ρητά νήματα ή μηχανισμούς συγχρονισμού.

Επιπρόσθετα, κάθε εντολή στο περιβάλλον του κελύφους αντιμετωπίζεται ως μια αυτόνομη και ανεξάρτητη μονάδα υπολογισμού, η οποία διαβάζει δεδομένα, τα επεξεργάζεται και παράγει αποτελέσματα. Το σύνολο των διαθέσιμων εντολών είναι πρακτικά απεριόριστο, καθώς οποιοδήποτε εκτελέσιμο αρχείο στο σύστημα μπορεί να κληθεί ως εντολή. Οι εντολές μπορεί να είναι υλοποιημένες σε οποιαδήποτε γλώσσα προγραμματισμού (C, Python, Rust, κ.λπ.) ή να διατίθενται μόνο σε δυαδική μορφή, με άγνωστη εσωτερική δομή και συμπεριφορά. Το χαρακτηριστικό αυτό, αν και προσφέρει μεγάλη ευελιξία, καθιστά ιδιαίτερα δύσκολη την πρόβλεψη της επίδοσης και των απαιτήσεων σε πόρους χωρίς τη διενέργεια εμπειρικών μετρήσεων.

Για τον συντονισμό αυτών των μονάδων, το κέλυφος παρέχει μια σειρά από τελεστές σύνθεσης, όπως ο σειριακός τελεστής (`;`), ο τελεστής εκτέλεσης στο παρασκήνιο (`&`), ο τελεστής αγωγού (`|`) και οι λογικοί τελεστές (`&&`, `||`). Μέσω αυτών, ο προγραμματιστής μπορεί να ορίσει με ακρίβεια τις χρονικές και λογικές εξαρτήσεις μεταξύ των διεργασιών.

Ωστόσο, η διαχείριση της ροής δεδομένων παρουσιάζει και σημαντικές προκλήσεις. Πολλές εντολές σχεδιάζονται να είναι «οκνηρές» ή ακολουθούν αυστηρά σειριακή λογική κατανάλωσης, διαβάζοντας δεδομένα μόνο όταν είναι απολύτως έτοιμες να τα επεξεργαστούν. Ένα χαρακτηριστικό παράδειγμα αυτής της συμπεριφοράς παρουσιάζεται στο Κώδικας 2.1. Αν και αυτή η συμπεριφορά είναι συχνά επιθυμητή για την ελαχιστοποίηση της χρήσης μνήμης, σε σύνθετους αγωγούς μπορεί να οδηγήσει σε φαινόμενα μπλοκαρίσματος και υποεκμετάλλευσης των διαθέσιμων υπολογιστικών πόρων, όπως φαίνεται στο ακόλουθο παράδειγμα.

Διάκριση Κελύφους και Εντολών Για την ορθή ανάλυση της απόδοσης, είναι θεμελιώδους σημασίας ο σαφής διαχωρισμός των αρμοδιοτήτων μεταξύ του ίδιου του διερμηνέα κελύφους και των επιμέρους εντολών που αυτός καλεί. Ο διερμηνέας (π.χ. Bash, Zsh) είναι υπεύθυνος για τη συντακτική ανάλυση του κώδικα, την ανάπτυξη μεταβλητών, τη διαχείριση των περιγραφέντων αρχείων και τη δημιουργία διεργασιών μέσω των κλήσεων συστήματος `fork` και `exec`. Αντιθέτως, ο κύριος υπολογιστικός φόρτος και η επεξεργασία των δεδομένων πραγματοποιούνται από τις εξωτερικές εντολές.

Ένα κρίσιμο στοιχείο που συχνά παραβλέπεται είναι η επίδραση των παραμέτρων εκτέλεσης στη συμπεριφορά των εντολών. Οι παράμετροι αυτές δεν μεταβάλλουν μόνο τη λειτουργικότητα, αλλά μπορούν να επηρεάσουν σημαντικά το προφίλ απόδοσης και τη δυνατότητα παραλληλισμού. Για παράδειγμα, μια εντολή που κανονικά επεξεργάζεται δεδομένα ως ροή μπορεί, με την προσθήκη κατάλληλων επιλογών, να απαιτήσει την πλήρη φόρτωση του αρχείου στη μνήμη ή να πραγματοποιήσει επιτόπιες τροποποιήσεις¹, μεταβάλλοντας ριζικά τον τρόπο αλληλεπίδρασης με το σύστημα αρχείων και τις υπόλοιπες διεργασίες.

Περιορισμοί και Προκλήσεις Παρά τη χρησιμότητα και την ευρεία διάδοσή του, το μοντέλο προγραμματισμού του κελύφους χαρακτηρίζεται από σημαντικούς περιορισμούς που δυσχεραίνουν την ανάπτυξη αποδοτικών και αξιόπιστων εφαρμογών:

- Περιορισμένη Κλιμάκωση και Παραλληλισμός:** Αν και το κέλυφος υποστηρίζει βασικές μορφές παραλληλισμού μέσω αγωγών και παρασκηνιακής εκτέλεσης, η αποτελεσματική αξιοποίηση των σύγχρονων πολυπύρηνων αρχιτεκτονικών παραμένει δύσκολη. Ο χρήστης συχνά καταφεύγει σε πολύπλοκους χειροκίνητους χειρισμούς, χρησιμοποιώντας τελεστές και εντολές όπως `&` και `wait[22]` ή σε εξωτερικά εργαλεία, όπως το `GNU parallel[51]`, διαδικασία που είναι επιρρεπής σε σφάλματα.
- Δυναμική και Αδιαφανής Συμπεριφορά:** Η εκτέλεση ενός προγράμματος κελύφους εξαρτάται σε μεγάλο βαθμό από την κατάσταση του περιβάλλοντος, την ύπαρξη και τα δικαιώματα αρχείων, καθώς και από ρυθμίσεις του συστήματος που μπορεί να μεταβληθούν δυναμικά. Η ιδιότητα αυτή, σε συνδυασμό με την αδιαφάνεια των εξωτερικών εντολών, καθιστά τη στατική ανάλυση και την πρόβλεψη της συμπεριφοράς ιδιαίτερα δυσχερή.
- Επιρρέπεια σε Σφάλματα και Περιορισμένη Ασφάλεια:** Το κέλυφος στερείται πολλών μηχανισμών ασφαλείας που συναντώνται σε σύγχρονες γλώσσες προγραμματισμού, όπως ισχυρό σύστημα τύπων ή δομημένη διαχείριση εξαιρέσεων. Ως αποτέλεσμα, απλά προγραμματιστικά λάθη, όπως η χρήση μη ορισμένων μεταβλητών σε κρίσιμες εντολές, μπορούν να επιφέρουν μη αναστρέψιμες συνέπειες. Χαρακτηριστικό παράδειγμα αποτελεί το κρίσιμο σφάλμα που εντοπίστηκε στο σενάριο εγκατάστασης της πλατφόρμας Steam για Linux [12], όπου η έλλειψη ελέγχου της μεταβλητής `STEAMROOT` οδήγησε στην ακούσια εκτέλεση της εντολής:

```
rm -rf "$STEAMROOT/"*
```

Σε περιπτώσεις όπου η μεταβλητή ήταν κενή, η εντολή μετατρεπόταν σε `rm -rf "/"*`, προκαλώντας τη διαγραφή ολόκληρου του συστήματος αρχείων του χρήστη.

Οι παραπάνω ιδιότητες συνθέτουν ένα περιβάλλον στο οποίο η συστηματική ανάλυση επιδόσεων και η αυτόματη βελτιστοποίηση αποτελούν ανοικτά και απαιτητικά ερευνητικά προβλήματα, ενισχύοντας την ανάγκη για τη δημιουργία ρεαλιστικών και αναπαραγωγικών μεθοδολογιών αξιολόγησης, όπως αυτή που προτείνεται στην παρούσα εργασία.

¹ Χαρακτηριστικό παράδειγμα αποτελεί η εντολή `sed`: στην τυπική της χρήση λειτουργεί ως φίλτρο, επιτρέποντας τον παραλληλισμό σε έναν αγωγό. Ωστόσο, η χρήση της σημαίας `-i` επιβάλλει την εγγραφή στο αρχικό αρχείο, απαιτώντας τη δημιουργία προσωρινών αρχείων και περιορίζοντας τις δυνατότητες παράλληλης επεξεργασίας.

2.2 Προσπάθειες Επιτάχυνσης και Βελτιστοποίησης

Η ανάγκη για βελτίωση της απόδοσης των προγραμμάτων κελύφους έχει οδηγήσει στην ανάπτυξη πληθώρας συστημάτων που επιχειρούν να υπερβούν τους περιορισμούς της σειριακής εκτέλεσης. Στη σχετική βιβλιογραφία συναντώνται προσεγγίσεις που στοχεύουν στη μείωση της μετακίνησης δεδομένων [58], στην αυτόματη [25, 32, 40] ή χειροκίνητη [6, 51] παραλληλοποίηση, καθώς και στην κατανομή της εκτέλεσης σε πολλαπλούς κόμβους [27, 47].

Ενδεικτικά, συστήματα όπως το POSH [58] επιχειρούν να μεταφέρουν τμήματα του υπολογισμού κοντά στα δεδομένα, ενώ προσεγγίσεις όπως τα DiSH [47] και FRACTAL [27] επεκτείνουν το κέλυφος σε καταναεμημένα περιβάλλοντα, επιτρέποντας την εκτέλεση σύνθετων προγραμμάτων σε συστοιχίες υπολογιστών. Παράλληλα, εργαλεία όπως το PASH [25, 32] και το *hS* [40] εστιάζουν στην αυτόματη εκμετάλλευση της διαθέσιμης παραλληλίας μέσω δυναμικής ανάλυσης και μετασχηματισμών κατά την εκτέλεση.

Στην παρούσα εργασία δεν επιχειρείται μια συνολική σύγκριση όλων των παραπάνω κατηγοριών συστημάτων. Αντίθετα, επιλέγονται τέσσερα αντιπροσωπευτικά εργαλεία που εκφράζουν διαφορετικές φιλοσοφίες επιτάχυνσης, τόσο καθιερωμένες όσο και πειραματικές: το Shark, το GNU parallel, το PASH και το *hS*. Τα συστήματα αυτά καλύπτουν ένα ευρύ φάσμα προσεγγίσεων, από στατικούς μετασχηματισμούς και χειροκίνητη παραλληλοποίηση, έως δυναμική μεταγλώττιση και εκτέλεση εκτός σειράς, και επιλέγονται με στόχο τη συγκριτική μελέτη της συμπεριφοράς τους σε ρεαλιστικά προγράμματα μέσω της σουίτας KOALA.

Shark Το Shark αποτελεί ένα από τα πρώτα συστήματα που αντιμετωπίζουν τα προγράμματα κελύφους ως αντικείμενα μεταγλώττισης [6], εφαρμόζοντας τεχνικές ανάλυσης και βελτιστοποίησης αντίστοιχες με εκείνες των παραδοσιακών γλωσσών προγραμματισμού. Η βασική του παραδοχή είναι ότι οι προσπελάσεις στο σύστημα αρχείων μπορούν να θεωρηθούν ανάλογες με τις αναφορές σε μεταβλητές, γεγονός που επιτρέπει την εφαρμογή ανάλυσης εξαρτήσεων και τεχνικών βελτιστοποίησης σε επίπεδο προγράμματος.

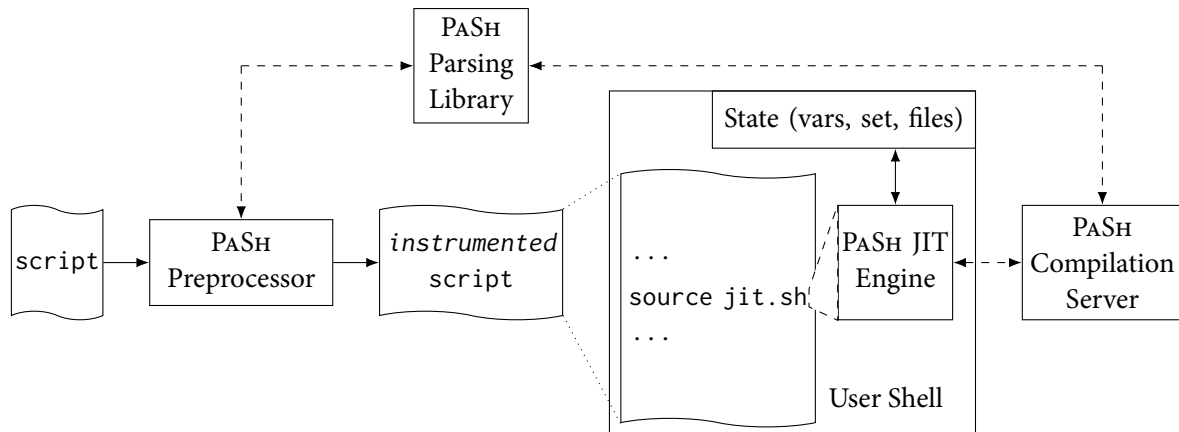
Με βάση αυτή την προσέγγιση, το Shark υλοποιεί ένα σύνολο βελτιστοποιήσεων με στόχο τη μείωση της χρήσης του συστήματος αρχείων και τη βελτίωση της τοπικότητας των δεδομένων. Ειδικότερα, εντοπίζει και εξαλείφει περιττά προσωρινά αρχεία, αντικαθιστά την επικοινωνία μέσω αρχείων με αγωγούς, και αφαιρεί άσκοπες ή πλεονάζουσες κλήσεις εντολών. Παράλληλα, μέσω της ανάλυσης εξαρτήσεων, αναδιοργανώνει ανεξάρτητες εντολές ώστε να εκτελούνται ταυτόχρονα, όταν αυτό είναι ασφαλές.

Ιδιαίτερη έμφαση δίνεται στη μείωση του κόστους δημιουργίας διεργασιών, το οποίο αποτελεί βασικό παράγοντα επιβάρυνσης στην εκτέλεση προγραμμάτων κελύφους. Για τον σκοπό αυτό, το Shark μετατρέπει συχνά χρησιμοποιούμενες εξωτερικές εντολές σε δυναμικές βιβλιοθήκες, οι οποίες φορτώνονται μία φορά και επαναχρησιμοποιούνται. Η τεχνική αυτή μειώνει σημαντικά το κόστος εκκίνησης των εντολών, αν και περιορίζει τη γενικότητα και την απομόνωση που παρέχει η εκτέλεση μέσω ξεχωριστών διεργασιών.

Επειδή το Shark δεν είναι διαθέσιμο ως υλοποιημένο σύστημα, στην παρούσα εργασία αξιοποιούνται οι συντακτικοί μετασχηματισμοί του, με στόχο τη μελέτη της επίδρασής τους σε ρεαλιστικά προγράμματα κελύφους, όπως αυτά που περιλαμβάνονται στη σουίτα KOALA.

GNU parallel Το GNU parallel αποτελεί ένα από τα πλέον διαδεδομένα εργαλεία χειροκίνητης παραλληλοποίησης στο οικοσύστημα του UNIX [51]. Επιτρέπει στον χρήστη να διασπά σύνολα δεδομένων και να εκτελεί πολλαπλές ανεξάρτητες διεργασίες ταυτόχρονα, αξιοποιώντας αποτελεσματικά τους διαθέσιμους υπολογιστικούς πόρους. Το εργαλείο υποστηρίζει τόσο παραλληλία δεδομένων όσο και παραλληλία εργασιών, ενώ παρέχει τη δυνατότητα εκτέλεσης διεργασιών σε απομακρυσμένους κόμβους μέσω ασφαλών συνδέσεων. Επιπλέον, παρέχει δυνατότητες ελέγχου της μορφής και της σειράς της εξόδου των παράλληλων διεργασιών.

Σε αντίθεση με τα αυτόματα συστήματα παραλληλοποίησης, το GNU parallel δεν αξιοποιεί



Σχήμα 2.1: Επισκόπηση του PASH. Το PASH ενορχηστρώνει τα σενάρια με κλήσεις στη μηχανή πάνω-στην-ώρα (JIT), η οποία κατά την εκτέλεση προωθεί τμήματα του προγράμματος στον διακομιστή μεταγλώττισης (PASH compilation server) για δυναμική επεξεργασία κατά την εκτέλεση.

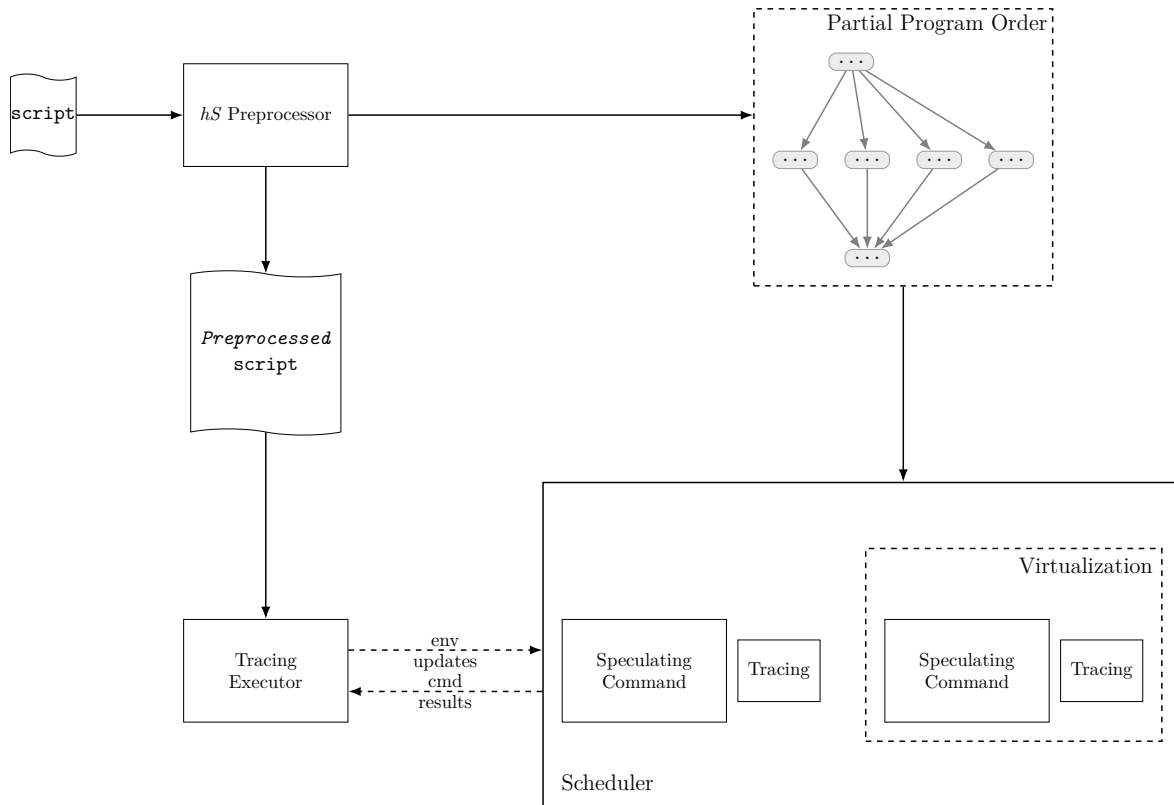
πληροφορία σχετικά με τη σημασιολογία ή τις εξαρτήσεις του προγράμματος. Η ευθύνη για τον ορθό διαχωρισμό των δεδομένων και τη διατήρηση της λειτουργικής ορθότητας μεταφέρεται στον χρήστη. Όπως έχει επισημανθεί στη σχετική βιβλιογραφία [25], λανθασμένες παραμετροποιήσεις μπορούν να οδηγήσουν τόσο σε σημαντική επιβράδυνση όσο και σε παραγωγή εσφαλμένων αποτελεσμάτων.

Το GNU parallel χρησιμοποιείται στην παρούσα εργασία ως βασικό σημείο αναφοράς, αντιπροσωπεύοντας την προσέγγιση της χειροκίνητης παραλληλοποίησης.

Το PASH [32, 73] αποτελεί ένα σύστημα αυτόματης παραλληλοποίησης προγραμμάτων κελύφους POSIX, το οποίο εκτελεί προγράμματα UNIX με παράλληλο τρόπο μέσω μηχανισμού μεταγλώττισης πάνω-στην-ώρα (Just-in-Time). Για την αντιμετώπιση της δυναμικής φύσης του κελύφους, το PASH εναλλάσσει φάσεις μεταγλώττισης και εκτέλεσης, συλλέγοντας σε πραγματικό χρόνο πληροφορίες για την κατάσταση του κελύφους, του συστήματος αρχείων και του περιβάλλοντος.

Το PASH αναγνωρίζει δυνητικά παραλληλοποιήσιμες περιοχές του προγράμματος και τις μετατρέπει σε γράφους ροής δεδομένων, στους οποίους κάθε εντολή αντιστοιχεί σε κόμβο και κάθε ροή δεδομένων σε ακμή. Η πληροφορία παραλληλοποισιμότητας κωδικοποιείται μέσω ενός συστήματος επισημειώσεων, το οποίο περιγράφει τις κατηγορίες παραλληλισμού των εντολών και λαμβάνει υπόψη τα ορίσματα και τις επιλογές τους, καθώς αυτά επηρεάζουν τη δυνατότητα παράλληλης εκτέλεσης. Στη συνέχεια, εφαρμόζονται μετασχηματισμοί στους γράφους αυτούς με στόχο την αποκάλυψη παραλληλίας δεδομένων και εργασιών, όπως η διάσπαση των ροών σε ανεξάρτητα τμήματα και η αναδιοργάνωση των εντολών. Ο βαθμός παραλληλίας είναι παραμετροποιήσιμος και ρυθμίζεται δυναμικά κατά την εκτέλεση με χρήση της σημαίας `-width` (πλάτος). Μετά την ολοκλήρωση των μετασχηματισμών, οι γράφοι μετατρέπονται εκ νέου σε κώδικα κελύφους, στον οποίο εισάγονται κατάλληλες εντολές παραλληλισμού (π.χ. `&`, `wait`) και επώνυμοι αγωγοί, ώστε να υλοποιείται ρητά η παράλληλη εκτέλεση και να διατηρείται η δομή του γράφου ροής δεδομένων. Στο σχήμα Σχήμα 2.1 φαίνεται μια επισκόπηση του PASH.

Για τη διασφάλιση της λειτουργικής ισοδυναμίας με τη σειριακή εκτέλεση, το PASH βασίζεται σε ένα μοντέλο ροής δεδομένων με επίγνωση σειράς [26], το οποίο διατηρεί τις απαραίτητες εξαρτήσεις μεταξύ εισόδων και εξόδων. Επιπλέον, πριν από κάθε μετασχηματισμό, αποθηκεύεται και αποκαθίσταται η κατάσταση του κελύφους, ώστε να αποφεύγονται παρενέργειες. Σε περιπτώσεις όπου η δυναμική μεταγλώττιση αποτυγχάνει ή η παραλληλοποίηση κρίνεται μη ασφαλής, το PASH επιστρέφει αυτόματα στη σειριακή εκτέλεση. Η προσέγγιση αυτή επιτρέπει σημαντικές επιταχύνσεις σε αγωγούς επεξεργασίας δεδομένων και επαναληπτικές δομές, ιδίως σε σενάρια με έντονη δυναμική συμπεριφορά. Ωστόσο, προϋποθέτει την ύπαρξη κατάλληλων επισημειώσεων εντολών και παρουσιάζει ευαισθησία στο κόστος της δυναμικής ανάλυσης και μεταγλώττισης.



Σχήμα 2.2: Επισκόπηση του hS. Το σύστημα εκτελεί μελλοντικές εντολές προκαταβολικά (speculation) σε απομονωμένα περιβάλλοντα (sandboxes) και εντοπίζει τις προσπελάσεις I/O (με χρήση ptrace) και την κατάσταση του περιβάλλοντος. Εάν δεν υπάρξουν συγκρούσεις εξαρτήσεων (conflicts), τα αποτελέσματα ενσωματώνονται στο σύστημα αρχείων (commit), διαφορετικά η εκτέλεση αναιρείται (rollback) και επαναλαμβάνεται [40].

hS Το hS εισάγει μια διαφορετική προσέγγιση αυτόματης επιτάχυνσης βασισμένη στην εκτέλεση εκτός σειράς [40]. Αντί να ακολουθεί αυστηρά τη συντακτική σειρά του προγράμματος, επιχειρεί να εκτελεί εντολές «καιροσκοπικά», προβλέποντας ότι δεν θα προκύψουν παραβιάσεις εξαρτήσεων.

Η προσέγγιση αυτή βασίζεται στην παρατήρηση ότι πολλά προγράμματα κελύφους είναι υπερβολικά σειριακά, παρά το γεγονός ότι περιέχουν ανεξάρτητες ή ασθενώς εξαρτώμενες εντολές. Το σύστημα προεπεξεργάζεται το πρόγραμμα και κατασκευάζει έναν γράφο μερικής διάταξης που αποτυπώνει τη συντακτική σειρά εκτέλεσης, ενώ στη συνέχεια, κατά την εκτέλεση, συλλέγει δυναμικά πληροφορίες για τις εξαρτήσεις μέσω ιχνηλάτησης προσπελάσεων στο σύστημα αρχείων και στο περιβάλλον. Με βάση αυτές τις πληροφορίες, εκτελεί μελλοντικές εντολές σε απομονωμένα περιβάλλοντα, χρησιμοποιώντας μηχανισμούς εικονικοποίησης και επικαλυπτόμενα συστήματα αρχείων. Οι παρενέργειες των εντολών καταγράφονται προσωρινά και ενσωματώνονται στο κύριο περιβάλλον μόνο όταν επιβεβαιωθεί ότι δεν παραβιάζονται εξαρτήσεις. Σε περίπτωση ανίχνευσης σύγκρουσης, οι εκτελέσεις αναιρούνται και η εκτέλεση συνεχίζεται σύμφωνα με τη σωστή σημασιολογική σειρά. Ο χρονοπρογραμματιστής του συστήματος εναλλάσσει εκτέλεση και επικύρωση, διασφαλίζοντας ότι η σειριακή σημασιολογία διατηρείται, ενώ παράλληλα εγγυάται ότι, στη χειρότερη περίπτωση, η απόδοση δεν αποκλίνει σημαντικά από τη βασική σειριακή εκτέλεση. Η τεχνική αυτή επιτρέπει την αξιοποίηση παραλληλίας εργασιών χωρίς την ανάγκη εκ των προτέρων επισημειώσεων ή προδιαγραφών εντολών, καθιστώντας το hS ένα σύστημα γενικής χρήσης. Ωστόσο, συνεπάγεται αυξημένο κόστος λόγω αποτυχημένων προβλέψεων, ιχνηλάτησης και μηχανισμών απομόνωσης, ενώ περιορίζεται από εντολές με μη εικονικοποιήσιμες παρενέργειες, όπως δικτυακή επικοινωνία ή εξαρτήσεις από χρονισμό και αναγνωριστικά διεργασιών.

Στην παρούσα εργασία χρησιμοποιείται μια πρώιμη υλοποίηση του hS, η οποία βρίσκεται σε πει-

ραματικό στάδιο ανάπτυξης, με στόχο να αναδειχθεί η δυνατότητα της σουίτας KOALA να υποστηρίζει και την αξιολόγηση μελλοντικών συστημάτων επιτάχυνσης.

Σύνοψη Τα παραπάνω συστήματα αναδεικνύουν ότι η επιτάχυνση προγραμμάτων κελύφους μπορεί να επιτευχθεί μέσω διαφορετικών μηχανισμών: στατικής ανάλυσης, χειροκίνητης ενορχήστρωσης, δυναμικής μεταγλώττισης και εκτέλεσης εκτός σειράς. Παρά τις σημαντικές επιτυχίες τους, η απόδοσή τους εξαρτάται σε μεγάλο βαθμό από τη δομή, τα δεδομένα και τη δυναμική συμπεριφορά των προγραμμάτων.

Η παρατήρηση αυτή καθιστά αναγκαία την ύπαρξη μιας εκτενούς και ρεαλιστικής σουίτας μετροπρογραμμάτων, ικανής να αποτυπώσει συστηματικά τις διαφοροποιήσεις αυτές, όπως η σουίτα KOALA.

2.3 Αξιολόγηση Υπολογιστικών Συστημάτων

Η πρόοδος στον σχεδιασμό και τη βελτιστοποίηση υπολογιστικών συστημάτων εξαρτάται άμεσα από τη δυνατότητα αξιόπιστης, επαναλήψιμης και συγκρίσιμης αξιολόγησης. Η ύπαρξη κοινά αποδεκτών σουιτών αξιολόγησης επιδόσεων επιτρέπει τη σύγκριση διαφορετικών προσεγγίσεων υπό παρόμοιες συνθήκες, τη διασταύρωση πειραματικών αποτελεσμάτων και τη συστηματική μελέτη συμβιβασμών μεταξύ απόδοσης, χρήσης πόρων και πολυπλοκότητας [37]. Ιστορικά, η καθιέρωση τέτοιων συνόλων μετροπρογραμμάτων συνέβαλε καθοριστικά στην ωρίμανση πολλών ερευνητικών πεδίων, μετατοπίζοντας την αξιολόγηση από αποσπασματικά πειράματα σε αναπαραγώγιμες και τυποποιημένες μεθοδολογίες.

Καθιερωμένες Σουίτες Αξιολόγησης Επιδόσεων Σε διαφορετικούς τομείς της επιστήμης υπολογιστών, η υιοθέτηση αντιπροσωπευτικών σουιτών αξιολόγησης υπήρξε καταλυτική για τη διαμόρφωση κοινών σημείων αναφοράς. Η οικογένεια SPEC [66] έθεσε το πρότυπο για την αξιολόγηση υλικού και μεταγλωττιστών, προσφέροντας αντιπροσωπευτικά φορτία για τη μέτρηση της απόδοσης του επεξεργαστή. Στον χώρο των διαχειριζόμενων περιβαλλόντων εκτέλεσης, η σουίτα DaCapo [7, 39] εισήγαγε σύνθετα και ρεαλιστικά φορτία εργασίας, επιτρέποντας τη μελέτη συλλεκτών απορριμμάτων και μεταγλωττιστών σε συνθήκες κοντά στη χρήση παραγωγής. Αντίστοιχα, τα μετροπρογράμματα Gabriel [19] παρείχαν κοινό πλαίσιο σύγκρισης για συστήματα LISP, συμβάλλοντας στη βελτίωση των μηχανισμών εκτέλεσης και των μεταγλωττιστών. Στον τομέα των πολυπύρηνων αρχιτεκτονικών, η σουίτα PARSEC [11] επικεντρώθηκε σε σύγχρονες εφαρμογές, επιτρέποντας τη μελέτη της κλιμάκωσης και της συμπεριφοράς της μνήμης σε ρεαλιστικά σενάρια. Αντίστοιχα, στις βάσεις δεδομένων, τα πρότυπα του οργανισμού TPC [57] καθιέρωσαν αυστηρές διαδικασίες μέτρησης και επικύρωσης αποτελεσμάτων, διασφαλίζοντας συγκρισιμότητα μεταξύ ερευνητικών και εμπορικών συστημάτων.

Κοινό χαρακτηριστικό των παραπάνω σουιτών είναι η έμφαση σε ολοκληρωμένες εφαρμογές και ρεαλιστικά δεδομένα, καθώς και η παροχή σαφώς ορισμένης διαδικασίας εκτέλεσης και επικύρωσης. Σε αντίθεση με τις μικρο-δοκιμές επίδοσης, οι οποίες απομονώνουν μεμονωμένες λειτουργίες, οι ολοκληρωμένες σουίτες αποτυπώνουν πολύπλοκες αλληλεπιδράσεις μεταξύ υποσυστημάτων και αναδεικνύουν φαινόμενα που δεν εμφανίζονται σε συνθετικά σενάρια. Η εμπειρία από τα παραπάνω πεδία καταδεικνύει ότι η ύπαρξη μιας καθιερωμένης σουίτας μπορεί να επιταχύνει ουσιαστικά την εξέλιξη ενός ερευνητικού χώρου.

Υφιστάμενες Προσεγγίσεις στο Κέλυφος UNIX Σε αντίθεση με τους παραπάνω τομείς, η έρευνα γύρω από το κέλυφος UNIX χαρακτηριζόταν για μεγάλο χρονικό διάστημα από την απουσία μιας καθιερωμένης σουίτας αξιολόγησης επιδόσεων.

Οι υπάρχουσες προσεγγίσεις μπορούν να ταξινομηθούν σε τέσσερις βασικές κατηγορίες. Πρώτον, εργαλεία όπως το ShellBench [36] και το UnixBench [35] βασίζονται σε συνθετικά σενάρια που

εστιάζουν σε επιμέρους χαρακτηριστικά, όπως η ταχύτητα κλήσεων συστήματος ή η δημιουργία διεργασιών. Αν και χρήσιμα για στοχευμένη ανάλυση χαμηλού επιπέδου, δεν αποτυπώνουν τη συμπεριφορά σύνθετων αγωγών επεξεργασίας δεδομένων ούτε οδηγούν σε γενικεύσιμα συμπεράσματα. Δεύτερον, σουίτες δοκιμών όπως η POSIX Test Suite [68] και η Smoosh [23] εστιάζουν στη λειτουργική ορθότητα και στη σημασιολογική συμμόρφωση με το πρότυπο POSIX. Παρότι απαραίτητες για την επικύρωση υλοποιήσεων, δεν παρέχουν πληροφορίες σχετικά με την απόδοση πραγματικών φορτίων εργασίας. Τρίτον, όσον αφορά την ανάλυση ρεαλιστικού κώδικα, πρόσφατες μελέτες όπως το "Bash in the Wild" [14] ανέλυσαν εκατομμύρια σενάρια από το GitHub για να χαρακτηρίσουν τη στατική χρήση της γλώσσας. Ωστόσο, η προσέγγιση αυτή αφορά μόνο την ανάλυση κώδικα (στατική συμπεριφορά) και δεν παρέχει εκτελέσιμα σενάρια με ρεαλιστικά δεδομένα για δυναμική αξιολόγηση της απόδοσης. Τέταρτον, ελλείψει καθιερωμένων προτύπων, πολλές ερευνητικές εργασίες βασίστηκαν σε αποσπασματικές συλλογές σεναρίων ή σε περιορισμένο αριθμό αυτοσχέδιων δοκιμών. Χαρακτηριστικά, οι δημιουργοί του Shark αναφέρουν ότι η αξιολόγησή τους βασίστηκε σε μικρό σύνολο δοκιμών, λόγω απουσίας κατάλληλης σουίτας αναφοράς [6]. Παρόμοιες πρακτικές εμφανίζονται και σε μεταγενέστερα συστήματα επιτάχυνσης κελύφους.

Η κατάσταση αυτή οδηγεί σε περιορισμένη αναπαραγωγικότητα, δυσκολία συγκρισιμότητας μεταξύ συστημάτων και κίνδυνο ακούσιας μεροληψίας μέσω επιλογής ευνοϊκών σεναρίων. Το πρόβλημα γίνεται εντονότερο στα συστήματα επιτάχυνσης κελύφους, όπου η απόδοση εξαρτάται έντονα από τη δομή των αγωγών, τον όγκο δεδομένων και τη δυναμική συμπεριφορά των εντολών. Μικρές μεταβολές στα δεδομένα ή στη σύνταξη μπορούν να οδηγήσουν σε σημαντικές αποκλίσεις επιδόσεων, οι οποίες δεν αποτυπώνονται σε απομονωμένες μικρο-δοκιμές.

Περιορισμοί και Ρόλος του ΚΟΑΛΑ Η σουίτα ΚΟΑΛΑ σχεδιάστηκε με στόχο να καλύψει αυτό το κενό, υιοθετώντας αρχές που έχουν αποδειχθεί επιτυχείς σε άλλα πεδία της έρευνας συστημάτων. Συγκεκριμένα, περιλαμβάνει ρεαλιστικά προγράμματα κελύφους προερχόμενα από πρακτικές εργασίες, καλύπτει πολλαπλά υπολογιστικά πεδία και παρέχει διαφορετικές κλίμακες εισόδου, επιτρέποντας τόσο ταχεία διερεύνηση όσο και εκτενή αξιολόγηση. Παράλληλα, συνοδεύεται από αυτοματοποιημένη υποδομή εγκατάστασης, εκτέλεσης και επικύρωσης αποτελεσμάτων, διασφαλίζοντας υψηλό βαθμό αναπαραγωγικότητας. Η ανάλυση της σουίτας δείχνει ότι καλύπτεται το σύνολο των βασικών χαρακτηριστικών του κελύφους POSIX, ενώ τα προγράμματα παρουσιάζουν σημαντική ποικιλία τόσο σε στατικά όσο και σε δυναμικά χαρακτηριστικά, όπως τον χρόνο επεξεργαστή (CPU), την κατανάλωση μνήμης και τη δραστηριότητα εισόδου/εξόδου. Τα φορτία εργασίας κλιμακώνονται από σύντομα σενάρια έως εκτελέσεις μεγάλης διάρκειας σε εκτεταμένα σύνολα δεδομένων. Με τον τρόπο αυτό, το ΚΟΑΛΑ επιτρέπει τη μετάβαση από αποσπασματικές μετρήσεις σε μια συνεκτική, αναπαραγώγιμη και επαναχρησιμοποιήσιμη μεθοδολογία αξιολόγησης. Το πλαίσιο αυτό καθιστά δυνατή τη συστηματική σύγκριση διαφορετικών προσεγγίσεων επιτάχυνσης στο ίδιο, ελεγχόμενο περιβάλλον.

Κεφάλαιο 3

Σχεδιασμός και Υλοποίηση της Σουίτας ΚΟΑΛΑ

Στο παρόν κεφάλαιο παρουσιάζεται εκτενώς η σουίτα ΚΟΑΛΑ. Αρχικά, αναφέρονται οι βασικές σχεδιαστικές της αρχές και η μεθοδολογία επιλογής των μετροπρογραμμάτων. Στη συνέχεια, αναλύεται η αρχιτεκτονική της σουίτας, η οποία περιλαμβάνει συνολικά 126 προγράμματα πραγματικής χρήσης από ποικίλες πηγές και υπολογιστικά πεδία, και τέλος περιγράφεται η υποδομή που εξασφαλίζει την αυτοματοποιημένη και αναπαραγωγίμη εκτέλεσή τους.

3.1 Σχεδιαστικές Αρχές

Ο σχεδιασμός του ΚΟΑΛΑ καθοδηγήθηκε από ένα σύνολο θεμελιωδών αρχών, οι οποίες αποτυπώνουν τις βασικές απαιτήσεις για μια σύγχρονη σουίτα αξιολόγησης συστημάτων κελύφους.

- **Αντιπροσωπευτικότητα:** Τα σενάρια της σουίτας αποτελούν πραγματικά προγράμματα που εκτελούν ουσιαστικούς υπολογισμούς πάνω σε πραγματικά δεδομένα. Η επιλογή αυτή επιτρέπει τη μελέτη της συμπεριφοράς των συστημάτων σε συνθήκες κοντά στη χρήση παραγωγής, σε αντίθεση με απομονωμένες μικρο-δοκιμές επίδοσης.
- **Ποικιλομορφία:** Η σουίτα καλύπτει ένα ευρύ φάσμα υπολογιστικών πεδίων, όπως ανάλυση δεδομένων, βιοπληροφορική, διαχείριση συστημάτων και μηχανική μάθηση, ενώ αξιοποιεί μεγάλο εύρος εντολών και χαρακτηριστικών του κελύφους. Με τον τρόπο αυτό αποφεύγεται η υπερπροσαρμογή της αξιολόγησης σε συγκεκριμένα μοτίβα προγραμμάτων.
- **Χαρακτηρισμός Φορτίων:** Εκτός από την παροχή μετροπρογραμμάτων, η σουίτα συνοδεύεται από συστηματικό χαρακτηρισμό τόσο των στατικών όσο και των δυναμικών ιδιοτήτων τους. Η προσέγγιση αυτή επιτρέπει την κατανόηση του τρόπου με τον οποίο διαφορετικά συστήματα επηρεάζουν την εκτέλεση πραγματικών προγραμμάτων.
- **Κλιμάκωση:** Η σουίτα παρέχει δεδομένα εισόδου σε πολλαπλές κλίμακες (ελάχιστη, μικρή, μεγάλη), επιτρέποντας τόσο γρήγορη διερεύνηση όσο και εκτεταμένες αξιολογήσεις μεγάλης διάρκειας. Τα πλήρη σύνολα δεδομένων φτάνουν σε μέγεθος εκατοντάδων gigabytes, επιτρέποντας την ανάλυση φορτίων εργασίας με έντονη δραστηριότητα εισόδου/εξόδου.
- **Αυτοματοποίηση και Αναπαραγωγιμότητα:** Η σουίτα ΚΟΑΛΑ παρέχει υποδομή που αυτοματοποιεί τη ρύθμιση του περιβάλλοντος, την εγκατάσταση εξαρτήσεων, την εκτέλεση και την επικύρωση αποτελεσμάτων, διευκολύνοντας την αναπαραγωγή πειραμάτων από διαφορετικούς ερευνητές.

Μεθοδολογία Επιλογής Μετροπρογραμμάτων

Η συγκρότηση της σουίτας βασίστηκε σε συστηματική διαδικασία επιλογής, με στόχο τη δημιουργία ενός αντιπροσωπευτικού και πρακτικά χρήσιμου συνόλου προγραμμάτων.

```

1  #!/bin/bash
2
3  d="./data/temperatures";
4  for y in $(seq $start $end); do
5      cat $d/$y | cut -c 89-92 | grep -v 999 |
6          sort -rn | head -n1 > max.$y
7      cat $d/$y | cut -c 89-92 | grep -v 999 |
8          sort -n | head -n1 > min.$y
9      cat $d/$y | cut -c 89-92 | grep -v 999 |
10         awk "{t += \$1; i++} END {print t/i}" > avg.$y
11 done

```

Κώδικας 3.1: Παράδειγμα ανάλυσης μετεωρολογικών δεδομένων στο ΚΟΑΛΑ. Υλοποίηση υπολογισμού μέγιστης, ελάχιστης και μέσης θερμοκρασίας μέσω επαναληπτικής δομής και σωληνώσεων.

Πηγές Προέλευσης Τα σενάρια προέρχονται από ποικίλες πηγές πραγματικής χρήσης, καλύπτοντας διαφορετικές χρονικές περιόδους και προγραμματιστικά στυλ:

- **Αποθετήρια ανοικτού κώδικα:** Έργα από πλατφόρμες όπως GitHub και GitLab, τα οποία χρησιμοποιούνται σε πραγματικά περιβάλλοντα.
- **Ακαδημαϊκή βιβλιογραφία:** Σενάρια που έχουν χρησιμοποιηθεί για την αξιολόγηση συστημάτων επιτάχυνσης κελύφους, όπως τα PASH, POSH και DISH.
- **Εκπαιδευτικά παραδείγματα:** Κλασικά παραδείγματα σύνθεσης εντολών από έργα όπως το Unix for Poets [33] και το Unix50 [54], τα οποία αναδεικνύουν τη φιλοσοφία του UNIX.

Κριτήρια Συμπερίληψης και Αποκλεισμού Για την επιλογή των σεναρίων εφαρμόστηκαν συγκεκριμένα κριτήρια, με πρωταρχικό στόχο τη συγκέντρωση ρεαλιστικών παραδειγμάτων εργασιών. Επιδιώχθηκε η συμπερίληψη προγραμμάτων, είτε απλών, είτε πολύπλοκων, τα οποία επεξεργάζονται πραγματικά δεδομένα και χαρακτηρίζονται από ετερογένεια και ποικιλομορφία, τόσο ως προς τα συντακτικά χαρακτηριστικά POSIX όσο και ως προς τα χαρακτηριστικά εκτέλεσής τους. Παράλληλα, αποφεύχθηκε σκόπιμα η ενσωμάτωση σεναρίων χωρίς διαθέσιμα δεδομένα ή λειτουργικές εξαρτήσεις, καθώς και η χρήση υπερβολικά απλοϊκών, τετριμμένων ή μη ρεαλιστικών προγραμμάτων, τα οποία δεν αντιπροσωπεύουν πραγματικές συνθήκες χρήσης.

3.2 Παράδειγμα Χρήσης

Για να καταδειχθεί ο τρόπος με τον οποίο το ΚΟΑΛΑ υποστηρίζει τη συγκριτική αξιολόγηση διαφορετικών συστημάτων επιτάχυνσης κελύφους, παρουσιάζεται ένα αντιπροσωπευτικό παράδειγμα από τη σουίτα.

Ο Κώδικας 3.1 παρουσιάζει ένα απλοποιημένο σενάριο ανάλυσης μετεωρολογικών δεδομένων, το οποίο μοιάζει με το πρόγραμμα `temp-analytics.sh` από το σύνολο μετροπρογραμμάτων `weather` της σουίτας. Σε αυτό υπολογίζονται η μέγιστη, η ελάχιστη και η μέση θερμοκρασία για πολλαπλά έτη. Το παράδειγμα συνδυάζει επαναληπτική δομή, σωληνώσεις και τυπικές εντολές επεξεργασίας δεδομένων, αναδεικνύοντας τις δυνατότητες αλλά και τους περιορισμούς διαφορετικών προσεγγίσεων βελτιστοποίησης.

Το σώμα του βρόχου αποτελείται από ανεξάρτητες σωληνώσεις που περιλαμβάνουν εντολές φιλτραρίσματος και υπολογισμού, όπως οι `grep`, `sort` και `awk`. Η σύνθεση αυτή επιτρέπει τη μελέτη διαφορετικών στρατηγικών επιτάχυνσης:

- το *Shark* μπορεί να εφαρμόσει συντακτικούς μετασχηματισμούς και να εκτελέσει τις ανεξάρτητες σωληνώσεις παράλληλα,
- το *GNU parallel* μπορεί να εισαγάγει παραλληλισμό σε επίπεδο επαναλήψεων ή και σωληνώσεων,
- το *PASH* μπορεί να παραλληλίσει τα επιμέρους στάδια κάθε σωλήνωσης.
- το *hS* μπορεί να εκτελέσει εντολές εκτός σειράς μέσω υποθετικής εκτέλεσης,

Το παράδειγμα αυτό δείχνει ότι η αξιολόγηση σε ρεαλιστικά σενάρια αποκαλύπτει τόσο τα οφέλη όσο και τους περιορισμούς κάθε προσέγγισης. Μέσω του ΚΟΑΛΑ, οι ερευνητές μπορούν να εκτελέσουν το ίδιο σενάριο με κοινά δεδομένα και ελεγχόμενο τρόπο εκτέλεσης, λαμβάνοντας άμεσα συγκρίσιμα αποτελέσματα χωρίς να απαιτείται η κατασκευή ιδιόκτητων φορτίων εργασίας.

3.3 Επισκόπηση της Σουίτας

Με βάση τις παραπάνω σχεδιαστικές αρχές, συγκροτήθηκε το πλήρες σύνολο της σουίτας ΚΟΑΛΑ. Σε αντίθεση με μικρο-δοκιμές επίδοσης, τα προγράμματα ΚΟΑΛΑ εκτελούν ολοκληρωμένες εργασίες, επιτρέποντας τη μελέτη σύνθετων αλληλεπιδράσεων μεταξύ εντολών, ροών δεδομένων και συστήματος αρχείων.

Ο Πίνακας 3.1 παρουσιάζει συνοπτικά το πλήρες σύνολο μετροπρογραμμάτων ΚΟΑΛΑ και τα βασικά χαρακτηριστικά τους. Τα προγράμματα οργανώνονται σε επιμέρους σύνολα, τα οποία μοιράζονται το είδος της εργασίας, την προέλευση, τα δεδομένα εισόδου ή τα υπολογιστικά χαρακτηριστικά. Στις επόμενες ενότητες αναλύονται τα βασικά περιγραφικά στοιχεία της σουίτας, ενώ αναλυτικότερος χαρακτηρισμός των στατικών και δυναμικών ιδιοτήτων τους ακολουθεί στα επόμενα κεφάλαια.

Υπολογιστικά Πεδία

Η σουίτα ΚΟΑΛΑ καλύπτει ένα ευρύ φάσμα υπολογιστικών πεδίων, αντανακλώντας τόσο παραδοσιακές όσο και σύγχρονες χρήσεις του κελύφους. Τα πεδία αυτά συμβολίζονται με αγγλικά αρχικά στον Πίνακα 3.1, ενώ παρακάτω παρουσιάζονται συνοπτικά με πλήρη ελληνική περιγραφή.

Τα προγράμματα ανάλυσης δεδομένων (**DA – Data Analytics**) περιλαμβάνουν σενάρια που εξάγουν, μετασχηματίζουν και συνοψίζουν μεγάλα σύνολα δεδομένων, όπως μετεωρολογικά αρχεία, δεδομένα δημόσιων μεταφορών και γονιδιωμικά σύνολα. Συνολικά περιλαμβάνονται 11 προγράμματα καταναμεμένα σε τρία σύνολα μετροπρογραμμάτων.

Τα προγράμματα διαχείρισης συστήματος (**SA – System Administration**) αντιστοιχούν σε τυπικές εργασίες ρύθμισης, συντήρησης και ανάλυσης καταγραφών συστήματος. Αποτελούν χαρακτηριστικό παράδειγμα της καθημερινής χρήσης του κελύφους σε πραγματικά περιβάλλοντα παραγωγής και περιλαμβάνουν 7 σενάρια χωρισμένα σε δύο σύνολα.

Οι ροές συνεχούς ολοκλήρωσης και ανάπτυξης (**CI – Continuous Integration**) περιλαμβάνουν σενάρια που αυτοματοποιούν διαδικασίες δόμησης λογισμικού, ανάλυσης και ελέγχου. Στη σουίτα ΚΟΑΛΑ αντιπροσωπεύονται από 23 προγράμματα καταναμεμένα σε δύο σύνολα.

Τα προγράμματα μηχανικής μάθησης (**ML – Machine Learning**) περιλαμβάνουν είτε σενάρια που υλοποιούν στάδια μάθησης είτε σενάρια που λειτουργούν ως «γλώσσα συγκόλλησης», συνδέοντας εξωτερικά εργαλεία και βιβλιοθήκες σε ενιαίες ροές εργασίας. Περιλαμβάνουν 27 προγράμματα καταναμεμένα σε τρία σύνολα.

Τα σενάρια αυτοματοποίησης (**AN – Automation**) περιλαμβάνουν 18 σενάρια, μεταξύ δύο συνόλων, που αυτοματοποιούν ετερογενείς εργασίες, όπως κρυπτογράφηση αρχείων, μετατροπή πολυμέσων ή ειδικές εργασίες επεξεργασίας περιεχομένου.

Τέλος, η κατηγορία των λοιπών προγραμμάτων (**MI – Miscellaneous**) περιλαμβάνει 40 σενάρια που δεν εντάσσονται σε συγκεκριμένο πεδίο, αλλά αναδεικνύουν την εκφραστικότητα του κελύφους μέσα από ετερογενείς εργασίες, όπως μετασχηματισμούς κειμένου ή υλοποίηση μηχανών αναζήτησης.

Πίνακας 3.1: Σύνοψη των μετροπρογραμμάτων της σουίτας ΚΟΑΛΑ. Ο πίνακας συνοψίζει όλα τα υποσύνολα μετροπρογραμμάτων της σουίτας ΚΟΑΛΑ. Στήλη 1: χαρακτηριστικά ταυτοποίησης, όπου παρουσιάζεται το όνομα κάθε συνόλου μετροπρογραμμάτων και (ενδεικτικά) προγράμματα που περιλαμβάνει. Στήλες 2–4: περιγραφικά χαρακτηριστικά, που συνοψίζουν το υπολογιστικό πεδίο εφαρμογής ($\mathcal{D}$), τον αριθμό προγραμμάτων του συνόλου (#.sh) και το συνολικό μέγεθος κώδικα (LoC). Στήλη 5: σύνοψη του μεγέθους των δεδομένων εισόδου κάθε προγράμματος. Στήλες 6–7: στατικά χαρακτηριστικά, δηλαδή ο αριθμός συντακτικών δομών του κελύφους (#Cons) και ο αριθμός διακριτών εντολών (#Cmd). Στήλες 8–11: δυναμικά χαρακτηριστικά εκτέλεσης, όπως ο χρόνος αξιολόγησης του κελύφους (t_S), ο χρόνος εκτέλεσης εντολών (t_C), η κατανάλωση μνήμης (Mem) και ο συνολικός όγκος εισόδου/εξόδου (I/O). Στήλες 12–13: χαρακτηριστικά συστήματος, συμπεριλαμβανομένου του αριθμού κλήσεων συστήματος (#SC) και ανοικτών περιγραφών αρχείων (#FD). Στήλη 14: πηγή προέλευσης προγραμμάτων.

Benchmark/Script	Surface			Inputs		Syntax			Dynamic			System		Source						
	$\mathcal{D}$	#.sh	LoC	Small	Full	#Cons	#Cmd	t_s	t_c	Mem	I/O	#SC	#FD							
analytics	SA/DA	4	53	1.94GB	78.9GB	10	21	10ms	84s	334MB	23.0GB	117.3m	84	[1, 58, 65]						
nginx.sh			19												10	13	~0	1s	10.3MB	1.79GB
...																				
bio	DA/BI	4	823	24.3GB	114GB	17	66	51s	6720s	25.1GB	352GB	35.3m	79	[9, 28, 30]						
data.sh			226												13	44	46s	3521s	25.1GB	287GB
...																				
ci-cd	CI/BS	21	2592	N/A		20	134	30ms	128s	461MB	2.35GB	3.5m	885	[10, 56]						
...																				
covid	DA/DE	5	53	3.34GB	5.08GB	5	6	~0	67s	1011MB	80.6GB	14.3m	150	[71]						
1.sh			9												5	6	~0	12s	13.2MB	17.5GB
...																				
file-mod	AN/MI	5	41	4.35GB	39.2GB	11	10	99ms	235s	164MB	13.9GB	1.5m	61	[58, 62, 65]						
encrypt.sh			11												10	6	~0	2s	2.82MB	5.10GB
img-conv.sh			11												11	6	99ms	170s	145MB	3.83GB
...																				
inference	ML/DA	3	61	3.85GB	11.7GB	15	27	40ms	1586s	7.16GB	83.7GB	37.9m	81	[18, 31]						
...																				
ml	ML/DA	1	47	4.71GB	15.0GB	7	1	~0	1156s	7.87GB	41.6GB	14.9m	10	[55]						
nlp	TP/ML	23	303	3k bks	115.9k bks	10	19	15s	851s	9.62MB	272GB	178.6m	526	[33]						
bigrams.sh			19												10	16	710ms	50s	7.93MB	22.5GB
...																				
oneliners	AN/TP	13	119	202MB	13.5GB	10	24	60ms	14s	199MB	3.77GB	4.5m	288							
spell.sh			11												6	7	~0	1s	8.38MB	523MB
top-n.sh			2												5	6	~0	960ms	8.25MB	408MB
uniq-ips.sh			2												4	3	~0	60ms	8.15MB	54.2MB
...														[59, 67]						
pkg	CI/AN	2	43	110 pkgs	2.0k pkgs	16	22	5s	201s	572MB	15.3GB	35.2m	132	[3, 75]						
pacaur.sh			29												14	19	5s	81s	490MB	14.6GB
proginf.sh			14												11	6	10ms	120s	572MB	709MB
repl	SA/MI	3	586	N/A		20	55	~0	24s	197MB	1.21GB	5.6m	79	[29, 58]						
...																				
unixfun	MI/TP	36	70	599MB	59.1GB	4	12	~0	5s	9.80MB	5.71GB	944.0k	733	[54]						
1.sh			2												3	2	~0	50ms	680KB	108MB
2.sh			2												3	3	~0	260ms	7.51MB	209MB
...																				
weather	DA/DE	2	74	893MB	146GB	11	20	260ms	958s	94.2MB	39.1GB	56.7m	50	[70]						
...																				
web-search	MI/TP	4	34	833MB	8.61GB	14	30	11s	1343s	1.32GB	17.1GB	112.6m	174	[8]						
...																				
Min		1	34	1.05MB	44.9MB	4	1	0	5.4	9.62MB	1.21GB	944.0k	10							
Mean		9	349.9	3.66GB	38.0GB	12.1	31.9	6.1	955.7	3.17GB	67.9GB	44.2m	238							
Median		4	65.5	1.54GB	13.5GB	11	21.5	0.1	218.2	398MB	20.1GB	25.0m	108							
Max		36	2592	24.3GB	146GB	20	134	52	6720.9	25.1GB	352GB	178.6m	885							

Πέρα από την ταξινόμηση κατά πεδίο εφαρμογής, η σουίτα συνδυάζει και διαφορετικά στυλ υπολογισμού, τα οποία εμφανίζονται μετά την κάθετο στη στήλη $\mathcal{D}$ του Πίνακα 3.1. Παραδείγματα αποτελούν τα σύνολα εξαγωγής δεδομένων (DE), τα σενάρια επεξεργασίας κειμένου πολλαπλών σταδίων

(TP) και τα σύνολα αυτοματοποίησης εργασιών (AN), τα οποία αντιπροσωπεύουν διαφορετικά πρότυπα χρήσης του κελύφους.

Σύνολα Μετροπρογραμμάτων

Η σουίτα ΚΟΑΛΑ αποτελείται από δεκατέσσερα σύνολα προγραμμάτων, τα οποία παρουσιάζονται με αλφαβητική σειρά και καλύπτουν διαφορετικά υπολογιστικά πρότυπα και στυλ χρήσης του κελύφους [37]. Κάθε σύνολο περιλαμβάνει προγράμματα με κοινή προέλευση, παρόμοια δεδομένα εισόδου ή αντίστοιχα υπολογιστικά χαρακτηριστικά.

Το σύνολο *analytics* περιλαμβάνει τέσσερα προγράμματα ανάλυσης αρχείων καταγραφής, τα οποία εκτελούν λειτουργίες φιλτραρίσματος και σύνοψης γεγονότων [58, 65]. Τα προγράμματα επεξεργάζονται συνολικά 78.9GB γραμμικών δεδομένων, όπως ίχνη TCP, αρχεία πρόσβασης Nginx και δεδομένα σάρωσης ZMap, προερχόμενα από πραγματικές δικτυακές καταγραφές [15, 17, 61, 72]. Η μικρή έκδοση των δεδομένων αποτελεί περικομμένο υποσύνολο των ίδιων αρχείων.

Το σύνολο *bio* αποτελείται από τέσσερα προγράμματα επεξεργασίας γονιδιωματικών και μεταγραφωματικών δεδομένων. Ένα πρόγραμμα εκτελεί ανάλυση πληθυσμιακής γονιδιωματικής [9, 30], ενώ τα υπόλοιπα τρία υλοποιούν στάδια της πλατφόρμας TERA-Seq για επεξεργασία αλληλουχιών RNA [28]. Τα σενάρια εμφανίζουν διακλάδωση και σύγκλιση ροών επεξεργασίας (με δομές *fan-out/fan-in*), παραλληλισμό τύπου ουράς εργασιών (*work-queue*) και έντονη πρόσβαση σε μεγάλα αρχεία, με συνολικά δεδομένα εισόδου 114GB [69].

Το σύνολο *ci-cd* περιλαμβάνει 21 προγράμματα δόμησης λογισμικού, όπως σενάρια δόμησης για τα λογισμικά Lua, Memcached, Redis και SQLite [10], καθώς και τη σουίτα δοκιμών του εργαλείου *make self* [56]. Τα προγράμματα αυτά χρησιμοποιούνται σε ροές συνεχούς ολοκλήρωσης και χαρακτηρίζονται από έντονες εξαρτήσεις και πολύπλοκη ροή ελέγχου, αλλά περιορισμένο όγκο δεδομένων εισόδου.

Το σύνολο *covid* περιέχει πέντε προγράμματα που υπολογίζουν στατιστικά στοιχεία δημόσιων συγκοινωνιών κατά την περίοδο της πανδημίας [71]. Τα σενάρια διατίθενται σε δύο εκδοχές: μία που βασίζεται σε σύνθεση κλασικών εργαλείων UNIX όπως *cut*, *sort* και *uniq*, και μία υλοποιημένη ως μονολιθικό πρόγραμμα *awk*. Τα δεδομένα εισόδου αποτελούνται από 5.08GB αρχείων CSV.

Το σύνολο *file-mod* περιλαμβάνει πέντε προγράμματα μετασχηματισμού αρχείων, όπως συμπίεση, κρυπτογράφηση και μετατροπή μορφότυπων. Δύο από αυτά επεξεργάζονται αρχεία καταγραφής δικτύου χρησιμοποιώντας *openssl* [49, 62], ενώ τα υπόλοιπα εκτελούν μετατροπές πολυμέσων σε εκατοντάδες αρχεία εικόνας και ήχου [58, 65].

Το σύνολο *inference* αποτελείται από τρία προγράμματα που εκτελούν εργασίες συμπερασμού χρησιμοποιώντας μεγάλα μοντέλα θεμελίωσης [20, 34, 76]. Παραδείγματα περιλαμβάνουν περιγραφή εικόνων [31], παραγωγή playlists [18] και ταξινόμηση ιερογλυφικών εικόνων [34]. Τα προγράμματα λειτουργούν ως περιβλήματα (*wrappers*) για εξωτερικές εργασίες μηχανικής μάθησης [52, 53, 64].

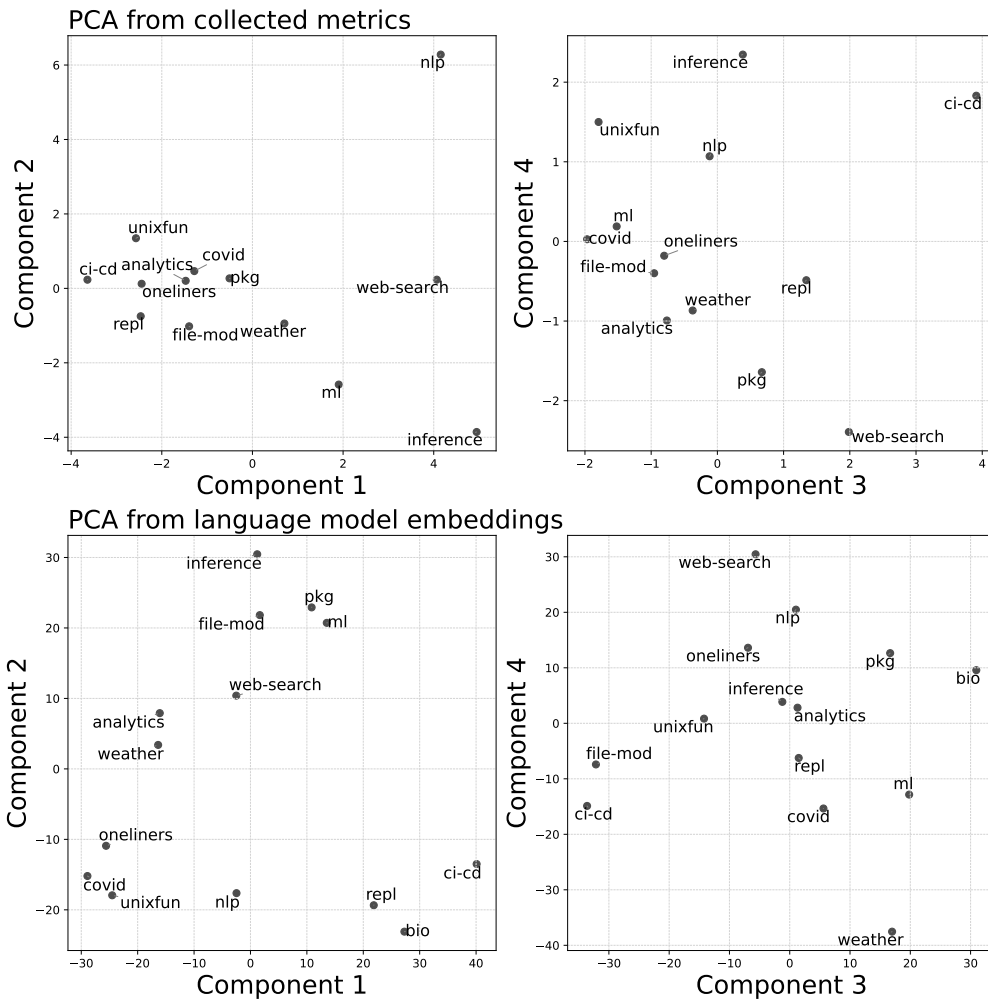
Το σύνολο *ml* αντιστοιχεί σε τυπικό αγωγό μηχανικής μάθησης, όπου ένα μονολιθικό πρόγραμμα Python έχει αποσυντεθεί σε πολλαπλά στάδια κελύφους (εισαγωγή δεδομένων, εκπαίδευση, ταξινόμηση και αξιολόγηση), επιτρέποντας τη μελέτη αρθρωτών ροών εργασίας [55].

Το σύνολο *nlp* περιλαμβάνει 23 σενάρια επεξεργασίας φυσικής γλώσσας από το tutorial “Unix for Poets” [33]. Τα περισσότερα από αυτά αποτελούνται από μία ή δύο γραμμές και μπορούν να συνδυαστούν σε μεγαλύτερους αγωγούς, λειτουργώντας πάνω σε συλλογή άνω των 115.000 βιβλίων [46].

Το σύνολο *oneliners* περιλαμβάνει 11 αλυσίδες αγωγών που προέρχονται τόσο από την κλασική βιβλιογραφία του Unix όσο και από σύγχρονες πηγές [1, 4, 5, 33, 44, 59, 67]. Τα σενάρια χρησιμοποιούν σύνθετους τελεστές ροής όπως απεικόνιση (*map*), φιλτράρισμα (*filter*) και αναγωγή (*reduction*) και αναδεικνύουν τη φιλοσοφία σύνθεσης του κελύφους.

Το σύνολο *pkg* αποτελείται από δύο προγράμματα που αυτοματοποιούν την εγκατάσταση πακέτων και την ανάλυση δικαιωμάτων λογισμικού [2, 50, 75]. Οι εκτελέσεις περιλαμβάνουν λήψη, δόμηση και ανάλυση εκατοντάδων πακέτων.

Το σύνολο *perl* περιλαμβάνει δύο ανεξάρτητα προγράμματα, ένα για έλεγχο ασφαλείας και ένα για αναπαραγωγή ανάπτυξης λογισμικού σε αποθετήριο Git. Και τα δύο δίνουν έμφαση σε σύνθετη



Σχήμα 3.1: Ανάλυση Κύριων Συνιστωσών (PCA). Πάνω: προβολή των μετροπρογραμμάτων με βάση στατικά και δυναμικά χαρακτηριστικά. Κάτω: προβολή βασισμένη σε διανυσματικές αναπαραστάσεις του πηγαίου κώδικα.

αλληλεπίδραση με το σύστημα αρχείων.

Το σύνολο `unixfun` περιέχει 36 προγράμματα από το Unix50 challenge [54], τα οποία μιμούνται κλασικούς υπολογισμούς επεξεργασίας κειμένου.

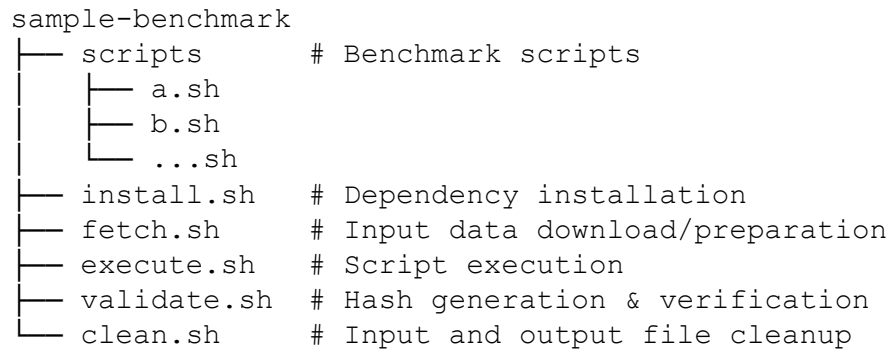
Το σύνολο `weather` περιλαμβάνει δύο φάσεις ανάλυσης ιστορικών δεδομένων καιρού [70]. Οι φάσεις αυτές αντιστοιχούν σε υπολογισμούς τύπου MapReduce και επιτρέπουν συγκρίσεις με συστήματα μεγάλης κλίμακας, ενώ τα δεδομένα προέρχονται από το NOAA [48] και αναπαράγουν το γνωστό διάγραμμα καιρού του Edward Tufte [16].

Τέλος, το σύνολο `web-search` υλοποιεί μια πλήρη ροή αναζήτησης ιστού (`crawl`, `index`, `query`) χρησιμοποιώντας αποκλειστικά εργαλεία κελύφους και προηγμένους τελεστές ροής δεδομένων.

Ποικιλομορφία της Σουίτας μέσω PCA

Πριν από την αναλυτική παρουσίαση των στατικών και δυναμικών χαρακτηριστικών της σουίτας, είναι χρήσιμο να εξεταστεί η συνολική ποικιλομορφία των προγραμμάτων.

Το Σχήμα 3.1 παρουσιάζει τα αποτελέσματα Ανάλυσης Κύριων Συνιστωσών (PCA), η οποία εφαρμόζεται σε δύο ανεξάρτητες αναπαραστάσεις των συνόλων μετροπρογραμμάτων. Η PCA μειώνει τη διάσταση των δεδομένων, διατηρώντας τις βασικές δομικές διαφορές μεταξύ των προγραμμάτων. Η πρώτη απεικόνιση βασίζεται σε στατικά και δυναμικά χαρακτηριστικά εκτέλεσης και δείχνει ότι τα σεσάρια κατανέμονται σε ευρύ χώρο, γεγονός που υποδηλώνει σημαντική ποικιλία συντακτικών



Σχήμα 3.2: Αρχιτεκτονική Διεπαφής Μετροπρογραμμάτων. Η ροή εργασιών αποτελείται από πέντε σπονδυλωτά σενάρια ελέγχου. Ο κεντρικός οδηγός `main.sh` ενορχηστρώνει την εκτέλεση, διασφαλίζοντας την αποδοτική διαχείριση των πόρων και την ακεραιότητα των δεδομένων.

και συμπεριφορικών προφίλ. Η δεύτερη απεικόνιση χρησιμοποιεί διανυσματικές αναπαραστάσεις πηγαίου κώδικα που παράγονται από προεκπαιδευμένο μοντέλο, αποτυπώνοντας υψηλού επιπέδου πληροφορία σχετικά με τη δομή και τη σημασιολογία των προγραμμάτων. Η αντίστοιχη κατανομή ενισχύει το συμπέρασμα ότι η σουίτα ΚΟΑΛΑ καλύπτει ευρύ φάσμα συντακτικών και σημασιολογικών μοτίβων.

3.4 Υποδομή και Διαμόρφωση Εκτέλεσης

Η αρχιτεκτονική του ΚΟΑΛΑ βασίζεται σε μια τυποποιημένη διεπαφή (interface) που διέπει τον κύκλο ζωής κάθε προγράμματος. Η προσέγγιση αυτή διασφαλίζει τον ντετερμινισμό των μετρήσεων και την ομοιομορφία μεταξύ ετερογενών φόρτων εργασίας, ορίζοντας ρητά τις φάσεις προετοιμασίας, εκτέλεσης και επαλήθευσης.

Η προδιαγραφή αυτή υλοποιείται μέσω πέντε σεναρίων υποδομής, τα οποία παρουσιάζονται στο Σχήμα 3.2. Συγκεκριμένα:

- Το `install.sh` εγκαθιστά τις απαραίτητες εξαρτήσεις λογισμικού.
- Το `fetch.sh` αναλαμβάνει τη λήψη ή παραγωγή των δεδομένων εισόδου και δέχεται όρισμα που καθορίζει το μέγεθός τους (`-min`, `-small`, `-full`).
- Το `execute.sh` εκτελεί τα προγράμματα του συνόλου και συλλέγει βασικές μετρικές, όπως χρόνο εκτέλεσης και κατανάλωση πόρων.
- Το `validate.sh` επαληθεύει την ορθότητα της εξόδου συγκρίνοντας τα παραγόμενα αποτελέσματα με προκαθορισμένες τιμές αναφοράς μέσω υπολογισμού κατακερματισμών.
- Το `clean.sh` απομακρύνει προσωρινά αρχεία και παραγόμενα αποτελέσματα.

Ενδεικτικά, στο Παράρτημα παρατίθενται τα σενάρια υποδομής για το σύνολο `weather`, στα Αποσπάσματα A.1, A.2, A.3, A.4 και A.5, αντίστοιχα.

Το ΚΟΑΛΑ περιλαμβάνει επίσης έναν κεντρικό οδηγό εκτέλεσης, το `main.sh`, ο οποίος καλεί διαδοχικά τα παραπάνω σενάρια για ένα ή περισσότερα μετροπρογράμματα. Ο οδηγός δέχεται παραμέτρους είτε μέσω σημαιών γραμμής εντολών είτε μέσω μεταβλητών περιβάλλοντος και τις προωθεί στα επιμέρους σενάρια. Ενδεικτικά, η μεταβλητή `ΚΟΑΛΑ_SHELL` καθορίζει τον διερμηνέα κελύφους που θα χρησιμοποιηθεί (π. χ. `bash`, `zsh` ή συγκεκριμένη διαδρομή εκτελέσιμου), με προεπιλογή το `sh`.

Επιπλέον, παρέχονται σημαίες για εξειδικευμένη συλλογή δεδομένων και έλεγχο της εκτέλεσης. Η σημαία `-resources` ενεργοποιεί την καταγραφή μετρικών δυναμικής ανάλυσης—όπως ο χρόνος

Πίνακας 3.2: Εκτιμώμενος χρόνος ενσωμάτωσης ανά σύνολο. Ο πίνακας παρουσιάζει τον εκτιμώμενο χρόνο ενσωμάτωσης (σε ώρες) για κάθε υποσύνολο μετροπρογραμμάτων της σουίτας ΚΟΑΛΑ. Οι εκτιμήσεις βασίζονται στην εμπειρία των ερευνητών κατά τη διαδικασία ενσωμάτωσης και περιλαμβάνουν την προσαρμογή των μετροπρογραμμάτων και την προετοιμασία των σεναρίων υποδομής.

Υποσύνολο Μετροπρογραμμάτων	~ <i>T</i> (ώρες)
analytics	40
bio	60
ci-cd	75
covid	65
file-mod	60
ml	30
inference	35
nlp	10
oneliners	10
pkg	30
repl	25
unixfun	10
weather	30
web-search	40
Σύνολο	520

εκτέλεσης, η κατανάλωση μνήμης και η ένταση δραστηριότητας εισόδου/εξόδου (I/O)—οι οποίες παρουσιάζονται αναλυτικά σε επόμενο κεφάλαιο. Για απλή χρονομέτρηση διατίθεται η σημαία `-time` (ή `-t`). Τέλος, μέσω της επιλογής `-scripts` (ή `-s`), η οποία ακολουθείται από τα ονόματα των επιθυμητών σεναρίων, ο χρήστης δύναται να εκτελέσει επιλεκτικά συγκεκριμένα σενάρια από ένα σύνολο μετροπρογραμμάτων.

Ένας χρήστης μπορεί να εκτελέσει το πρόγραμμα `temp-analytics.sh` (που μοιάζει με το παράδειγμα του Σχήματος 3.1) από το σύνολο `weather`, με το `main.sh` και τη μικρή κλίμακα δεδομένων ως εξής:

```
./main.sh --small weather --scripts temp-analytics
```

Η σχεδίαση της υποδομής στοχεύει στη γρήγορη συλλογή προκαταρκτικών αποτελεσμάτων και στη διευκόλυνση συγκρίσεων μεταξύ συστημάτων. Παραμένει εσκεμμένα ελαφριά και επεκτάσιμη, επιτρέποντας στους χρήστες να την προσαρμόσουν στις ιδιαίτερες ανάγκες τους.

Εκτέλεση σε Απομονωμένο Περιβάλλον

Η σουίτα ΚΟΑΛΑ παρέχει προαιρετική υποστήριξη εκτέλεσης σε απομονωμένο περιβάλλον μέσω Docker. Διατίθεται αρχείο `Dockerfile` το οποίο δημιουργεί περιβάλλον βασισμένο σε Debian και εγκαθιστά τις βασικές εξαρτήσεις της σουίτας. Υποστηρίζεται επίσης η δυναμική δημιουργία εικόνων ειδικά προσαρμοσμένων σε κάθε σύνολο μετροπρογραμμάτων, με ενσωματωμένη κλήση του `main.sh`. Η χρήση container επιτρέπει αναπαραγώγιμα πειράματα χωρίς τροποποίηση του συστήματος του χρήστη. Η σουίτα αποφεύγει τη χρήση προνομιακών εντολών, περιορίζοντας τις απαιτήσεις σε δικαιώματα διαχειριστή μόνο όπου είναι απολύτως αναγκαίο.

Ενσωμάτωση Νέου Μετροπρογράμματος

Η προσθήκη νέου μετροπρογράμματος στο ΚΟΑΛΑ απαιτεί την προσαρμογή των πέντε βασικών σεναρίων υποδομής. Συγκεκριμένα, περιλαμβάνει: (1) τον ορισμό εξαρτήσεων στο `install.sh`, (2) την προετοιμασία δεδομένων πολλαπλών μεγεθών στο `fetch.sh`, (3) την αυτοματοποιημένη εκτέλεση στο `execute.sh`, (4) τον ορισμό λογικής επικύρωσης στο `validate.sh`, και (5) τον καθαρισμό προσωρινών αρχείων στο `clean.sh`.

Ο απαιτούμενος χρόνος ενσωμάτωσης κυμάνθηκε από 10 έως 80 εργατοώρες ανά υποσύνολο. Μικρά και αυτοτελή σύνολα όπως τα `oneliners`, `unixfun` και `nlp` ενσωματώθηκαν σε περίπου 10–12 ώρες. Αντίθετα, σύνθετα ή έντονα εξαρτώμενα από δεδομένα σύνολα όπως τα `file-mod`, `bio` και `ci-cd` απαιτήσαν σημαντικά περισσότερη προσπάθεια λόγω μεγέθους, εξαρτήσεων και αυστηρών απαιτήσεων ορθότητας. Σημαντικό μέρος του χρόνου αφιερώθηκε στην προσαρμογή δεδομένων και στη διασφάλιση της ορθότητας υπό διαφορετικά περιβάλλοντα εκτέλεσης, γεγονός που αναδεικνύει τη σημασία μιας επαναχρησιμοποιήσιμης και τυποποιημένης υποδομής.

Κεφάλαιο 4

Συντακτική Ανάλυση της Σουίτας ΚΟΑΛΑ

Το παρόν κεφάλαιο εστιάζει στη συντακτική ανάλυση των μετροπρογραμμάτων της σουίτας ΚΟΑΛΑ, αναδεικνύοντας τη δομική τους πολυπλοκότητα και την ποικιλομορφία των κατασκευών του κελύφους που συναντώνται στην πράξη. Σε αντίθεση με τα περιβάλλοντα μίας γλώσσας, το κέλυφος λειτουργεί συχνά ως «συγκολλητικός ιστός», συνδυάζοντας στοιχεία από πολλαπλές γλώσσες. Ως εκ τούτου, η ανάλυση διαχωρίζει τα χαρακτηριστικά που αφορούν καθαρά το τμήμα του κελύφους από εκείνα των εξωτερικών προγραμμάτων που καλούνται μέσω αυτού και συχνά υλοποιούν τον πυρήνα του υπολογισμού.

Η κατανόηση αυτών των χαρακτηριστικών είναι θεμελιώδης για τη μελέτη συστημάτων επιτάχυνσης, καθώς πολλές τεχνικές βελτιστοποίησης βασίζονται στο συντακτικό δέντρο για τον εντοπισμό παραλληλισμού και την αναδιάταξη εντολών.

4.1 Μεθοδολογία

Η εξαγωγή των χαρακτηριστικών πραγματοποιείται μέσω της βιβλιοθήκης `libdash` [41], η οποία χρησιμοποιεί τον ορισμό αφηρημένης σύνταξης του `SMOOSH` [23] για το πρότυπο `POSIX`. Η προσέγγιση αυτή διασφαλίζει ορθότερη ανάλυση σε σχέση με απλές μεθόδους σάρωσης (π.χ. κανονικές εκφράσεις), επιτρέποντας την ακριβή αναγνώριση δομών ελέγχου και τον σαφή διαχωρισμό μεταξύ κλήσεων και ορισμών.

Η μεθοδολογία ακολουθεί δύο άξονες:

- Για το **τμήμα του κελύφους**, καταμετράται το σύνολο των εμφανίσεων κάθε κόμβου στο Αφηρημένο Συντακτικό Δέντρο (AST).
- Για το **τμήμα των εντολών**, η ανάλυση καταγράφει αποκλειστικά τους κόμβους που αφορούν εντολές, ενσωματωμένες λειτουργίες (`built-ins`) και συναρτήσεις.

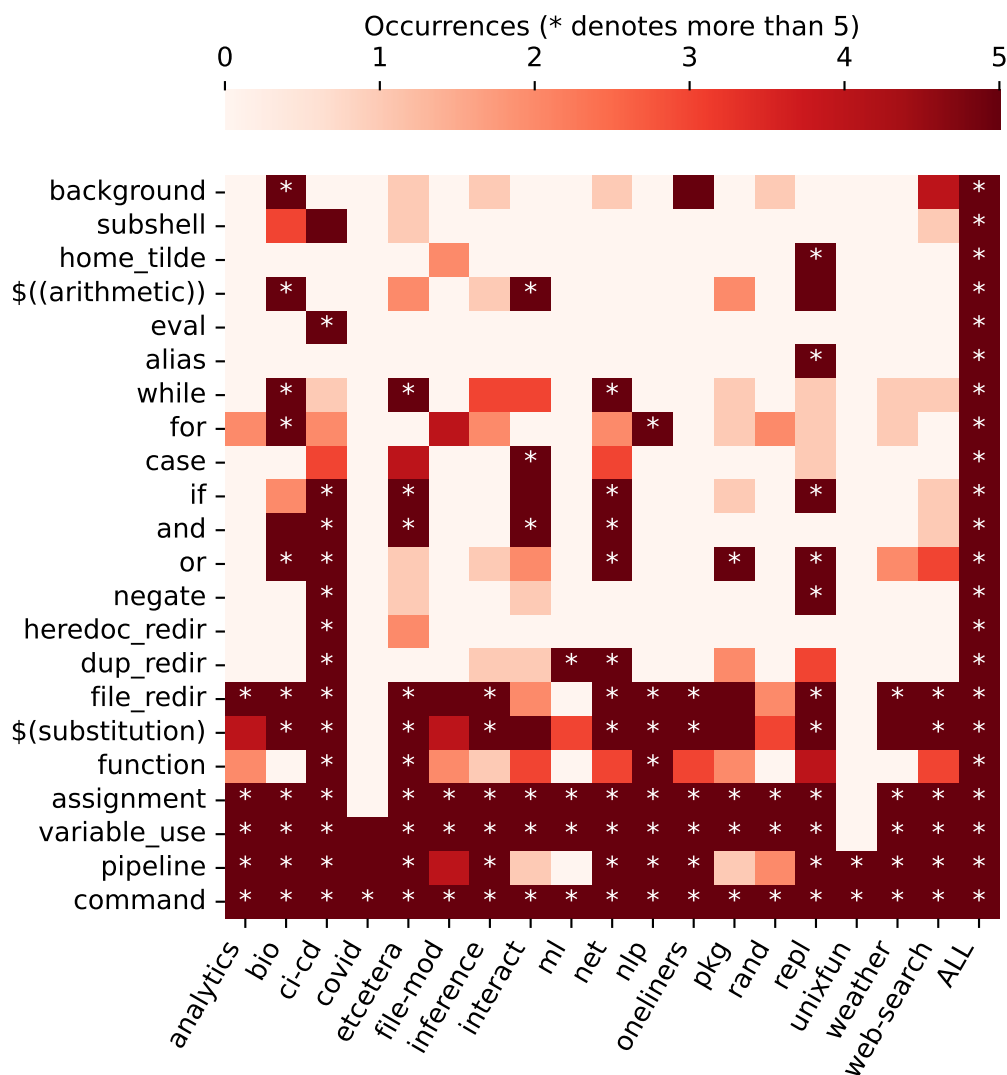
Αυτό παράγει συντηρητικά αποτελέσματα που εξαιρούν δυναμικές κατασκευές (π.χ. `$CMD arg`), αποτρέποντας τη σύγχυση μεταξύ της στατικής δομής και της επαναλαμβανόμενης συμπεριφοράς κατά τον χρόνο εκτέλεσης.

4.2 Συντακτικά Χαρακτηριστικά και Μοτίβα

Η κατανομή των συντακτικών δομών αποτυπώνεται οπτικά στο Σχήμα 4.1. Κάθε κελί υποδεικνύει τη συχνότητα εμφάνισης μιας δομής (κάθετος άξονας) σε ένα μετροπρόγραμμα (οριζόντιος άξονας). Συνολικά, τα μετροπρογράμματα ΚΟΑΛΑ αξιοποιούν όλο το φάσμα των συντακτικών κατασκευών του `POSIX` κελύφους, σε ποικίλους συνδυασμούς που αντικατοπτρίζουν διαφορετικά στυλ υπολογισμού.

Βασικές Δομές και Ροές

Τα περισσότερα σύνολα χρησιμοποιούν σωληνώσεις (`pipelines`) της μορφής `a | b`, περνώντας την έξοδο της μιας εντολής ως είσοδο στην επόμενη. Οι σωληνώσεις αποτελούν κρίσιμο χαρακτηριστικό που βελτιστοποιείται από πολλά συστήματα παραλληλισμού δεδομένων. Επιπλέον, σχεδόν όλα τα



Σχήμα 4.1: Θερμικός χάρτης συντακτικών χαρακτηριστικών. Κάθε κελί υποδεικνύει τη συχνότητα εμφάνισης μιας δομής σε ένα μετροπρόγραμμα (με * να υποδηλώνει περισσότερες από 5 εμφανίσεις). Η στήλη ALL αθροίζει την παρουσία των δομών σε όλη τη σουίτα [37].

σενάρια κάνουν χρήση μεταβλητών, αναθέσεων και ανακατευθύνσεων ροών ή αρχείων (`file_redir`), ενώ συχνή είναι και η υποκατάσταση εντολών (`command substitution`).

Πολυπλοκότητα και Διαφοροποίηση ανά Πεδίο Όπως διαφαίνεται στο Σχήμα 4.1, κανένα σύνολο προγραμμάτων δεν μοιράζεται το ίδιο συντακτικό προφίλ. Η ανάλυση αναδεικνύει σαφή μοτίβα διαφοροποίησης:

Απλά Σενάρια vs. Σύνθετες Ροές Σύνολα όπως τα `unixfun`, `nlp`, `covid` και `analytics` χαρακτηρίζονται από αραιή παρουσία πολύπλοκων δομών (ανοιχτόχρωμες περιοχές), βασιζόμενα κυρίως σε γραμμικές σωληνώσεις και αναλυτές αρχείων.

Δομές Ελέγχου και Λογισμικό Υποδομής Αντίθετα, σύνολα όπως τα `ci-cd`, `bio` και `pkg` εμφανίζουν πυκνή χρήση δομών ελέγχου (`if`, `while`, `case`), υποκελυφών (`subshells`) και λογικών τελεστών. Ειδικότερα, το `ci-cd` και το `file-mod` ορίζουν συναρτήσεις, καταρρίπτοντας την αντίληψη ότι τα σενάρια

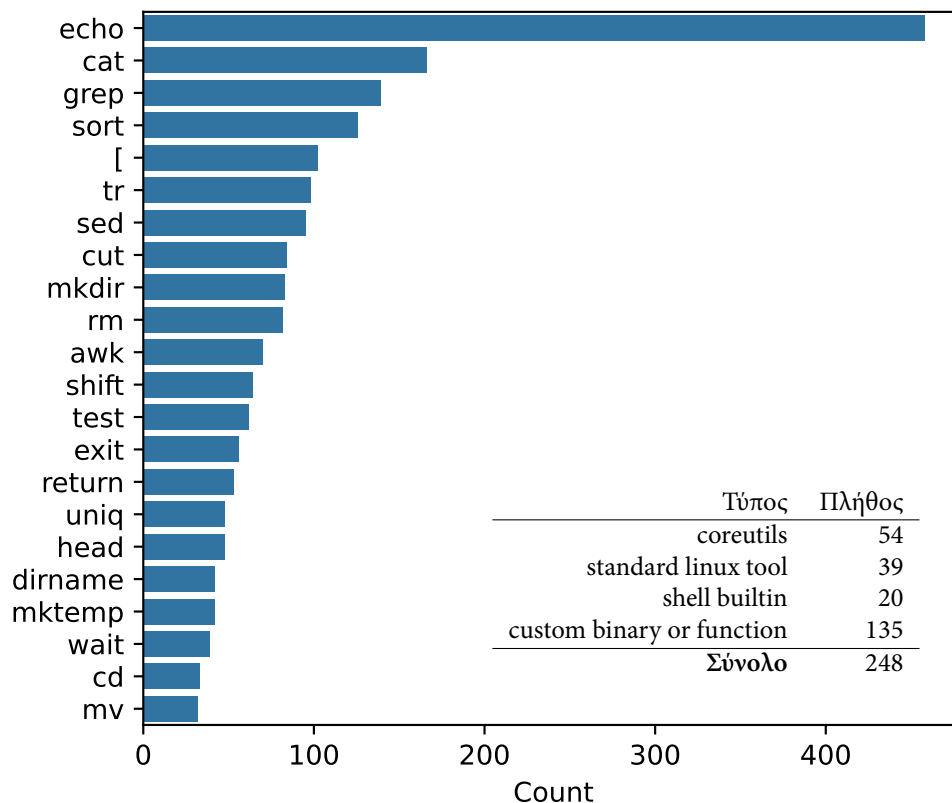
κελύφους στερούνται αρθρωτής δομής.

Ρητός Παραλληλισμός Δύο σύνολα (bio, web-search) αξιοποιούν ρητά τον παραλληλισμό μέσω των τελεστών & και wait. Αυτό το χαρακτηριστικό είναι ιδιαίτερα σημαντικό για συστήματα που στοχεύουν στη βελτιστοποίηση της απόδοσης, καθώς υποδηλώνει προσπάθεια εκμετάλλευσης των πόρων σε επίπεδο σεναρίου.

Σύγκριση με Πραγματικά Σενάκια Χρήσης Η αναλογία αυτών των χαρακτηριστικών ευθυγραμμίζεται με μελέτες χρήσης κελύφους σε ευρεία κλίμακα [14]. Η κατανομή των δομών στη σουίτα ΚΟΑΛΑ αντιστοιχεί στις σύγχρονες τάσεις, επιβεβαιώνοντας ότι η σουίτα είναι αντιπροσωπευτική των προγραμμάτων που απαντώνται στον πραγματικό κόσμο.

4.3 Ανάλυση Εντολών και Συνιστωσών

Η εξέταση της κατανομής των εντολών (Σχήμα 4.2) αποκαλύπτει τα δομικά στοιχεία των σεναρίων. Η σουίτα περιλαμβάνει πολλές από τις πιο κοινές εντολές: η echo κυριαρχεί με 441 εμφανίσεις, ενώ ακολουθούν τα cat, grep, sort και tr με περίπου τις μισές εμφανίσεις.



Σχήμα 4.2: Κατανομή εντολών. Συχνότητα εμφάνισης των πιο κοινών εντολών (αριστερά) και ομαδοποίηση των μοναδικών εργαλείων (ένθετος πίνακας) [37].

Τα αποτελέσματα αυτά συμφωνούν σε μεγάλο βαθμό με μελέτες εντολών σε πραγματικά περιβάλλοντα [14], με εξαίρεση τις εντολές χειρισμού διαδρομών (*path manipulation*), καθώς οι περισσότερες διαδρομές στο ΚΟΑΛΑ είναι στατικές.

Ο πίνακας του Σχήματος 4.2 ομαδοποιεί τις εντολές σε τέσσερις κατηγορίες:

- GNU Coreutils (π.χ. echo, cat).
- Ενσωματωμένες εντολές κελύφους (*shell built-ins*, π.χ. cd, return).
- Τυπικά εργαλεία Linux (*standard tools*, π.χ. grep, sed, awk).
- Προσαρμοσμένα εκτελέσιμα ή συναρτήσεις (*custom binaries or functions*).

Ενώ οι τρεις πρώτες κατηγορίες αποτελούν την πλειονότητα ως προς τη συχνότητα χρήσης, τα προσαρμοσμένα εκτελέσιμα και οι συναρτήσεις αντιστοιχούν σε 135 εντολές, δηλαδή στο 54.4% του συνόλου των μοναδικών εντολών. Το γεγονός αυτό, που είχε επισημανθεί και στην αρχική ανάλυση, δυσχεραίνει τη στατική ανάλυση καθώς τα συστήματα επιτάχυνσης καλούνται να διαχειριστούν αυτά τα εργαλεία ως «μαύρα κουτιά».

4.4 Σύνοψη

Η συντακτική ανάλυση επιβεβαιώνει ότι η σουίτα είναι συντακτικά πλήρης και αντιπροσωπεύει ένα ευρύ φάσμα προγραμματιστικών μοτίβων, από απλές σωληνώσεις έως σύνθετα συστήματα δόμησης λογισμικού. Ο συνδυασμός εξωτερικών εντολών, σωληνώσεων, υποκελυφών και ανακατευθύνσεων καθιστά το κέλυφος μια μοναδικά ισχυρή αλλά και πολύπλοκη πλατφόρμα. Η ποικιλομορφία που καταγράφεται στο ΚΟΑΛΑ στοχεύει ακριβώς στην αποτύπωση αυτής της διακύμανσης, προσφέροντας ένα ρεαλιστικό σημείο αναφοράς για την αξιολόγηση συστημάτων κελύφους.

Κεφάλαιο 5

Δυναμική Ανάλυση και Πειραματική Διαμόρφωση

Στο προηγούμενο κεφάλαιο παρουσιάστηκε η συντακτική ανάλυση του ΚΟΑΛΑ, η οποία ανέδειξε τη μεγάλη ποικιλία συντακτικών δομών και προγραμματιστικών μοτίβων που εμφανίζονται στα μετροπρογράμματα. Ωστόσο, η στατική ανάλυση από μόνη της δεν επαρκεί για την κατανόηση της συμπεριφοράς ενός φορτίου εργασίας. Η πραγματική επιβάρυνση ενός προγράμματος κελύφους καθορίζεται από τον τρόπο με τον οποίο αλληλεπιδρά με το λειτουργικό σύστημα κατά την εκτέλεση, τη χρήση πόρων, τη σχέση μεταξύ κελύφους και εξωτερικών εντολών, καθώς και την ένταση δραστηριότητας εισόδου/εξόδου.

Για τον λόγο αυτό, το παρόν κεφάλαιο παρουσιάζει τη δυναμική ανάλυση του ΚΟΑΛΑ, όπως υλοποιήθηκε στο πλαίσιο της ανάπτυξης και χαρακτηρισμού της σουίτας [37]. Η ανάλυση βασίζεται σε λεπτομερή παρακολούθηση της εκτέλεσης κάθε μετροπρογράμματος και καταγράφει χρονικές μετρικές, κατανάλωση μνήμης, όγκο δραστηριότητας εισόδου/εξόδου και αλληλεπίδραση με τον πυρήνα του λειτουργικού συστήματος. Αξίζει να σημειωθεί εξ αρχής ότι, για την παραγωγή των διαγραμμάτων που ακολουθούν, χρησιμοποιήθηκαν τα μικρά σύνολα δεδομένων εισόδου (`-small`), παρέχοντας μια ισορροπημένη βάση σύγκρισης για όλη τη σουίτα.

5.1 Πειραματική Διαμόρφωση και Μεθοδολογία

Η δυναμική ανάλυση του ΚΟΑΛΑ στοχεύει στον ποσοτικό χαρακτηρισμό της συμπεριφοράς των προγραμμάτων κελύφους κατά την πραγματική εκτέλεση. Σε αντίθεση με τη στατική ανάλυση, η οποία περιορίζεται στη συντακτική δομή του κώδικα, η δυναμική ανάλυση επιτρέπει την παρατήρηση της αλληλεπίδρασης των μετροπρογραμμάτων με τον πυρήνα του λειτουργικού συστήματος, της χρήσης πόρων και της κατανομής του χρόνου εκτέλεσης μεταξύ κελύφους και εξωτερικών εντολών.

Για τη διεξαγωγή των πειραμάτων και την εξαγωγή των δυναμικών χαρακτηριστικών, χρησιμοποιήθηκε ένα σύστημα που διέθετε επεξεργαστή 8 πυρήνων (AMD Ryzen 7 9700X στα 3.80GHz με υποστήριξη hyper-threading), 32GB μνήμη RAM και δίσκο 1TB NVMe. Το λειτουργικό σύστημα ήταν Ubuntu 24.04.1, ενώ ως διερμηνέας κελύφους χρησιμοποιήθηκε το `bash` (έκδοση 5.2.21) με ενεργοποιημένη τη σημαία `-posix`.

Για την εξαγωγή αυτών των χαρακτηριστικών υιοθετείται ένας συνδυασμός συμπληρωματικών τεχνικών μέτρησης, οι οποίες επιτρέπουν τόσο τη συλλογή ακριβών συνολικών μεγεθών όσο και τη συνεχή παρακολούθηση της εκτέλεσης σε πραγματικό χρόνο. Συγκεκριμένα, η μεθοδολογία βασίζεται σε δύο βασικούς άξονες, τις μετρήσεις συστήματος μετά την ολοκλήρωση της εκτέλεσης και τη συνεχή παρακολούθηση κατά τη διάρκεια της εκτέλεσης.

Μετρήσεις Συστήματος μετά την Εκτέλεση

Η πρώτη τεχνική βασίζεται στην ανάγνωση πληροφοριών που παρέχει ο ίδιος ο πυρήνας του Linux μέσω του εικονικού συστήματος αρχείων `/proc`. Μετά την ολοκλήρωση κάθε μετροπρογράμματος, ανακτώνται μετρικές από τα αρχεία:

- `/proc/<pid>/stat`
- `/proc/<pid>/io`

όπου `<pid>` είναι το αναγνωριστικό διεργασίας του μετροπρογράμματος που εκτελέστηκε.

Το αρχείο `stat` παρέχει λεπτομερή στοιχεία σχετικά με τον χρόνο χρήσης της κεντρικής μονάδας επεξεργασίας, διαχωρίζοντας τον χρόνο που δαπανήθηκε σε λειτουργία χρήστη και σε λειτουργία πυρήνα. Από τις τιμές αυτές υπολογίζεται ο συνολικός χρόνος επεξεργασίας, ο οποίος επιτρέπει την αποτίμηση του πραγματικού υπολογιστικού κόστους ανεξάρτητα από καθυστερήσεις αναμονής ή αδράνειας.

Το αρχείο `io` παρέχει συγκεντρωτικές πληροφορίες σχετικά με τις λειτουργίες εισόδου και εξόδου. Ιδιαίτερο ενδιαφέρον παρουσιάζουν τα πεδία που αντιστοιχούν στον συνολικό όγκο δεδομένων που διαβάστηκαν και γράφτηκαν, επιτρέποντας τον χαρακτηρισμό της έντασης δραστηριότητας εισόδου/εξόδου κάθε φόρτου εργασίας. Η προσέγγιση αυτή είναι ανεξάρτητη από τις επιμέρους εντολές και καταγράφει τη συνολική συμπεριφορά του μετροπρογράμματος όπως αυτή παρατηρείται από τον πυρήνα.

Οι μετρήσεις αυτές συμπληρώνονται σε ξεχωριστή εκτέλεση από καταγραφή κλήσεων συστήματος με το εργαλείο `strace`. Συγκεκριμένα, χρησιμοποιείται η επιλογή `-f`, ώστε να παρακολουθούνται και οι κλήσεις συστήματος που προέρχονται από διεργασίες-παιδιά, επιτρέποντας την εξαγωγή του συνολικού αριθμού αλληλεπιδράσεων με τον πυρήνα για ολόκληρο τον φόρτο εργασίας και όχι μόνο για την αρχική διεργασία.

Χρησιμοποιήθηκε επίσης δοκιμαστικά το `lsprof` για την καταγραφή ανοικτών περιγραφέντων αρχείων. Ωστόσο, η μέθοδος αυτή βασίζεται σε στιγμιότυπα της κατάστασης του συστήματος και όχι σε συνεχή παρακολούθηση, γεγονός που μπορεί να οδηγήσει σε απώλεια βραχύβιων γεγονότων εισόδου/εξόδου. Για τον λόγο αυτό τα αποτελέσματά του δεν αξιοποιήθηκαν στις τελικές μετρικές, οι οποίες βασίζονται κυρίως στα δεδομένα του `/proc`, στο `strace` και στη δυναμική παρακολούθηση μέσω `psutil`.

Ο συνδυασμός των παραπάνω επιτρέπει την αποτύπωση της συνολικής σχέσης κάθε μετροπρογράμματος με το λειτουργικό σύστημα, προσφέροντας μετρικές που δεν είναι δυνατόν να εξαχθούν μόνο από τον χρόνο εκτέλεσης.

Συνεχής Παρακολούθηση κατά την Εκτέλεση

Παρότι οι μετρήσεις που εξάγονται από το `/proc` μετά την ολοκλήρωση της εκτέλεσης παρέχουν ακριβή συγκεντρωτικά δεδομένα, δεν επαρκούν για την κατανόηση της χρονικής εξέλιξης και των στιγμιαίων αιχμών στην κατανάλωση πόρων. Για τον λόγο αυτό, εφαρμόζεται μια δεύτερη τεχνική, η οποία βασίζεται σε συνεχή δειγματοληψία κατά τη διάρκεια της εκτέλεσης (runtime profiling).

Ειδικότερα, υλοποιήθηκε ένας μηχανισμός παρακολούθησης σε γλώσσα Python, ο οποίος πραγματοποιεί περιοδικό έλεγχο με διάστημα 10 ms (0.01 s). Ο μηχανισμός αξιοποιεί τη βιβλιοθήκη `psutil` σε συνδυασμό με τη συνάρτηση υψηλής ακρίβειας `time.perf_counter`. Η συγκεκριμένη συχνότητα επιλέχθηκε ως ο βέλτιστος συμβιβασμός μεταξύ υψηλής χρονικής ανάλυσης και χαμηλής υπολογιστικής επιβάρυνσης (overhead), διασφαλίζοντας την επαρκή καταγραφή των απότομων μεταβολών χωρίς να επηρεάζεται αρνητικά η εκτέλεση των ίδιων των μετροπρογραμμάτων.

Κατά τη δειγματοληψία συλλέγονται διαρκώς οι ακόλουθες πληροφορίες:

- στιγμιαία χρήση μνήμης (για τον εντοπισμό της μέγιστης αιχμής),
- πραγματικός χρόνος εκτέλεσης (wall-clock time),
- στιγμιαίος αριθμός ανοικτών περιγραφέντων αρχείων (file descriptors).

Η συνεχής παρακολούθηση επιτρέπει τον εντοπισμό φαινομένων που θα περνούσαν απαρατήρητα σε μια απλή μέτρηση έναρξης-λήξης. Τέτοια φαινόμενα περιλαμβάνουν τις απότομες αιχμές στη χρήση μνήμης ή τις μεταβολές στη συμπεριφορά των προγραμμάτων κατά τη μετάβασή τους από αμιγώς υπολογιστικές φάσεις σε φάσεις έντονης δραστηριότητας εισόδου/εξόδου.

Συνδυασμός Μετρήσεων και Εξαγωγή Αποτελεσμάτων

Η τελική μεθοδολογία βασίζεται στη συνδυασμένη χρήση των δύο προαναφερθεισών τεχνικών. Οι μετρήσεις από το `/proc` και το `strace` παρέχουν ακριβή συνολικά μεγέθη, αντανακλώντας τη συμπεριφορά του συστήματος όπως την αντιλαμβάνεται ο πυρήνας, ενώ η συνεχής παρακολούθηση μέσω του `rsutil` αποτυπώνει πιστά τη χρονική εξέλιξη.

Για την εξαγωγή των τελικών αποτελεσμάτων που παρουσιάζονται στα επόμενα υποκεφάλαια, εφαρμόζεται μια συγκεκριμένη λογική συνάθροισης ανά σύνολο μετροπρογραμμάτων. Όπως αναλύεται και στην αντίστοιχη δημοσίευση της σουίτας [37], τα στατιστικά που αφορούν τον χρόνο επεξεργαστή (CPU time) και τον όγκο δραστηριότητας εισόδου/εξόδου (I/O) αθροίζονται για όλα τα επιμέρους προγράμματα που ανήκουν στο ίδιο σύνολο. Αντίθετα, για μετρικές χωρητικότητας, όπως η χρήση μνήμης και οι περιγραφείς αρχείων, υιοθετείται η μέγιστη τιμή (*high-water mark*) που καταγράφηκε σε οποιοδήποτε σημείο κατά τη διάρκεια ολόκληρης της εκτέλεσης του συνόλου.

Αυτός ο συνδυασμός μετρήσεων και συνάθροισης καθιστά δυνατή την εξαγωγή ενός πλήρους και αντιπροσωπευτικού προφίλ εκτέλεσης για κάθε μετροπρόγραμμα της σουίτας, καλύπτοντας αφενός την υπολογιστική διάσταση και αφετέρου την αλληλεπίδραση με το υποκείμενο σύστημα.

5.2 Αποτελέσματα Δυναμικού Χαρακτηρισμού

Πέραν της συντακτικής ποικιλομορφίας, η οποία αναλύθηκε εκτενώς στο προηγούμενο κεφάλαιο, η αξία μιας σουίτας αξιολόγησης κρίνεται πρωτίστως από τη συμπεριφορά των σεναρίων της κατά την εκτέλεση. Σε αυτό το κεφάλαιο παρατίθενται τα αποτελέσματα της δυναμικής ανάλυσης του ΚΟΑΛΑ, με έμφαση σε μετρικές όπως ο χρόνος εκτέλεσης, η κατανάλωση μνήμης και η ένταση δραστηριότητας εισόδου/εξόδου. Η ανάλυση αποδεικνύει ότι τα μετροπρόγραμμα ΚΟΑΛΑ καλύπτουν ένα ιδιαίτερα ευρύ φάσμα συμπεριφορών, καθώς οι τιμές των μετρικών διαφέρουν κατά πολλές τάξεις μεγέθους ανάμεσα στα διάφορα υπολογιστικά πεδία.

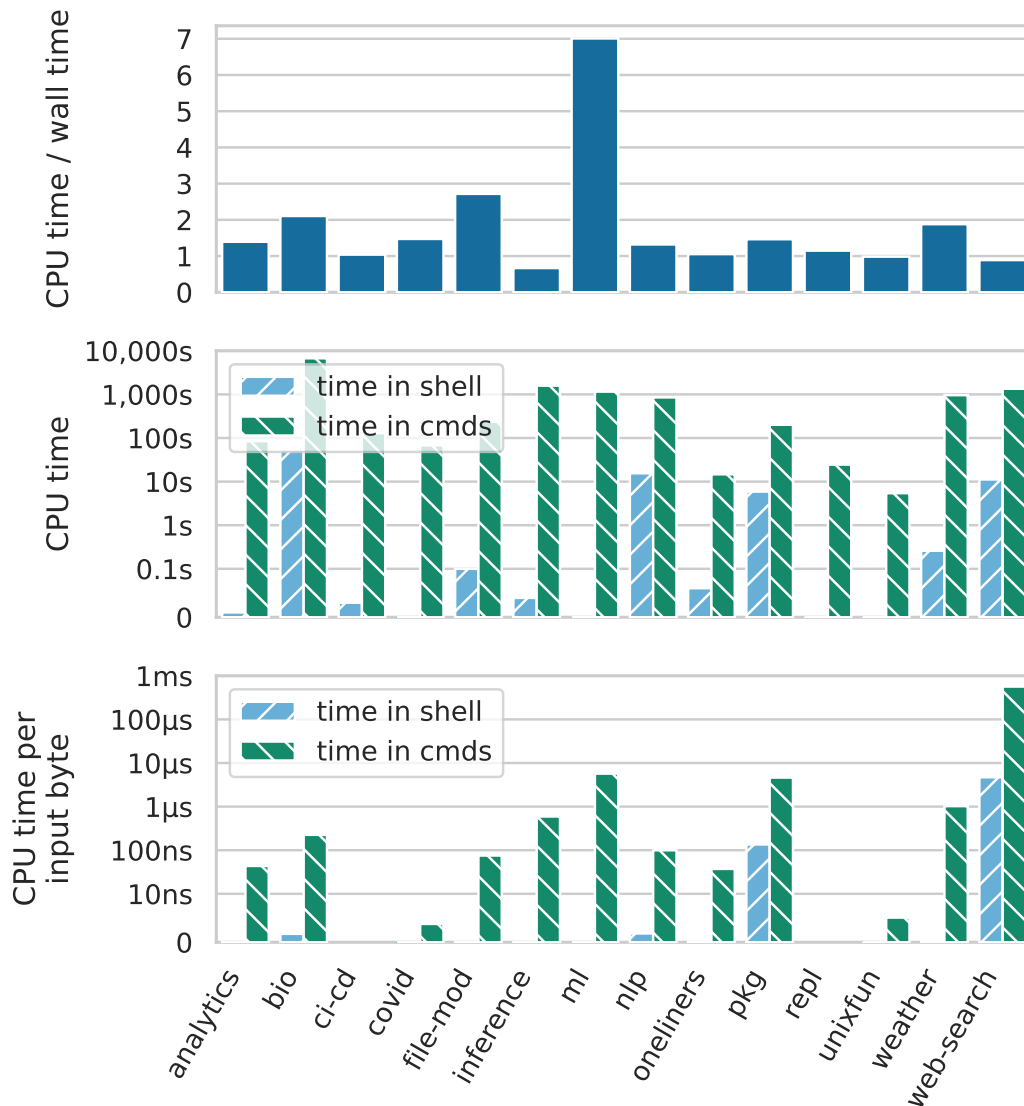
Ανάλυση Χρόνου Εκτέλεσης

Ο χρόνος εκτέλεσης αποτελεί τη θεμελιώδη μετρική για την αξιολόγηση της απόδοσης. Όπως αποτυπώνεται συνολικά στο Σχήμα 5.1, η σουίτα ΚΟΑΛΑ περιλαμβάνει τόσο βραχύβια σεσάρια ελέγχου όσο και χρονοβόρες διαδικασίες μαζικής επεξεργασίας δεδομένων, αναδεικνύοντας διαφορετικά προφίλ συμφόρησης πόρων.

Μεταβλητότητα Διάρκειας και Κατανομή Χρόνου Η απόλυτη διάρκεια εκτέλεσης των μετροπρογραμμάτων (Σχήμα 5.1, μεσαίο γράφημα) παρουσιάζει εκτεταμένη διακύμανση [37]. Στο κάτω άκρο του φάσματος, σύνολα όπως το `unixfun` ολοκληρώνονται ταχύτατα, επιτρέποντας γρήγορους κύκλους αξιολόγησης. Αντίθετα, στο άνω άκρο, φόρτοι όπως τα `bio`, `m1` και `web-search` απαιτούν παρτεταμένους χρόνους εκτέλεσης, ξεπερνώντας τα 1.000 δευτερόλεπτα αθροιστικού χρόνου CPU.

Ένα κρίσιμο εύρημα αφορά τον διαχωρισμό του υπολογιστικού χρόνου μεταξύ του ίδιου του διεργασιών κελύφους και των εξωτερικών εκτελέσιμων:

- **Φόρτοι κυριαρχούμενοι από εξωτερικές εντολές:** Σεσάρια όπως τα `bio`, `inference` και `m1` αναλώνουν σχεδόν το σύνολο του χρόνου τους στις εξωτερικές εφαρμογές [37]. Σε αυτές τις περιπτώσεις, το κέλυφος λειτουργεί αμιγώς ως εντοχρηστωτής, και η όποια επιτάχυνση προϋποθέτει παραλληλισμό σε επίπεδο ροής δεδομένων (*data-flow*).
- **Μικτοί φόρτοι εργασίας:** Σε σύνολα όπως τα `nlp`, `ci-cd` και `rep1`, ένα υπολογίσιμο ποσοστό του χρόνου δαπανάται στην εκτέλεση λογικής εντός του ίδιου του κελύφους (μπλε τμήματα ράβδων) [37]. Τα σεσάρια αυτά αποτελούν ιδανικούς στόχους για συστήματα βελτιστοποίησης που αποσκοπούν στη μείωση του κόστους διερμηνείας (*interpretation overhead*).

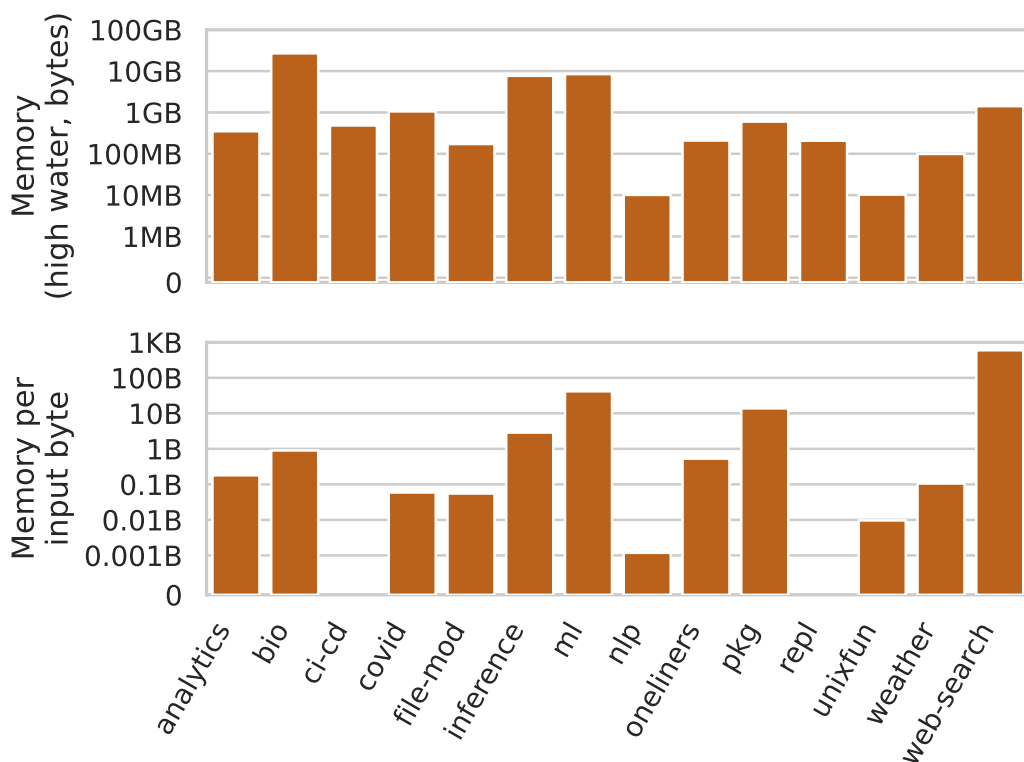


Σχήμα 5.1: Ανάλυση χρόνου εκτέλεσης. Πάνω: Λόγος χρόνου CPU προς πραγματικό χρόνο (wall-clock time). Μέση: Απόλυτος χρόνος CPU, διαχωρισμένος σε χρόνο κελύφους (μπλε) και εξωτερικών εντολών (πράσινο), σε συμμετρική λογαριθμική κλίμακα. Κάτω: Χρόνος CPU κανονικοποιημένος ανά byte δεδομένων εισόδου [37].

Αποδοτικότητα Παραλληλισμού Ο λόγος του συνολικού χρόνου CPU προς τον πραγματικό χρόνο (wall-clock time), ο οποίος απεικονίζεται στο άνω γράφημα, λειτουργεί ως δείκτης παραλληλισμού. Τιμές μεγαλύτερες της μονάδας υποδεικνύουν την ταυτόχρονη αξιοποίηση πολλαπλών πυρήνων, είτε μέσω ρητών δομών του κελύφους είτε εσωτερικά από τα ίδια τα εργαλεία. Για παράδειγμα, το *ml* αγγίζει τιμές κοντά στο 7, ενώ το *file-mod* πλησιάζει το 3, καταδεικνύοντας υψηλή αποδοτικότητα παράλληλης εκτέλεσης. Στον αντίποδα, τιμές κοντά ή κάτω από τη μονάδα (π.χ. στα *covid* και *web-search*) υποδηλώνουν αυστηρά σειριακή εκτέλεση ή σημαντικές καθυστερήσεις λόγω αναμονής (I/O blocking) [37].

Υπολογιστική Πυκνότητα Μια συμπληρωματική διάσταση της ανάλυσης προσφέρεται από το κάτω γράφημα, το οποίο κανονικοποιεί τον υπολογιστικό χρόνο ως προς το μέγεθος των δεδομένων εισόδου (CPU time ανά input byte).

Η μετρική αυτή αποτυπώνει την «υπολογιστική πυκνότητα» κάθε φόρτου [37]. Είναι χαρακτηριστικό ότι σύνολα όπως τα *bio* και *covid*, παρά τον μεγάλο απόλυτο χρόνο εκτέλεσής τους, καταγράφουν εξαιρετικά χαμηλές τιμές στο κανονικοποιημένο γράφημα. Αυτό επιβεβαιώνει ότι η χρονική



Σχήμα 5.2: Ανάλυση κατανάλωσης μνήμης. Πάνω: Απόλυτη μέγιστη χρήση μνήμης σε συμμετρική λογαριθμική κλίμακα. Κάτω: Μνήμη κανονικοποιημένη ανά byte δεδομένων εισόδου [37].

τους διάρκειες οφείλεται αποκλειστικά στον τεράστιο όγκο δεδομένων που διαχειρίζονται, και όχι στην πολυπλοκότητα των λειτουργιών ανά byte. Αντιθέτως, σενάρια όπως τα *web-search* και *ml* χαρακτηρίζονται από ιδιαίτερα υψηλή υπολογιστική ένταση ανά μονάδα δεδομένων. Σημειώνεται ότι μετροπρογράμματα χωρίς διακριτά αρχεία εισόδου (π.χ. *ci-cd* και *repl*) εξαιρούνται από τον συγκεκριμένο υπολογισμό.

Ανάλυση Κατανάλωσης Μνήμης

Η μέγιστη κατανάλωση μνήμης αποτελεί κρίσιμο παράγοντα αξιολόγησης, ιδιαίτερα σε περιβάλλοντα περιορισμένων πόρων, όπως τα *serverless containers* ή τα ενσωματωμένα συστήματα. Το Σχήμα 5.2 αποτυπώνει την απόλυτη χρήση μνήμης καθώς και την κανονικοποιημένη απαίτηση ανά byte δεδομένων εισόδου.

Απόλυτη Κατανάλωση Μνήμης Όπως προκύπτει από το άνω γράφημα, η διακύμανση στη χρήση μνήμης εκτείνεται σε πέντε τάξεις μεγέθους. Παραδοσιακοί φόρτοι επεξεργασίας ροών, όπως τα *unixfun* και *analytics*, χαρακτηρίζονται από ελάχιστο αποτύπωμα — με ορισμένα σενάρια να απαιτούν μόλις 600–700 KB— καθώς διεκπεραιώνουν την είσοδο και έξοδο δεδομένων αυστηρά σειριακά, αποφεύγοντας τη φόρτωση ογκωδών δομών.

Στον αντίποδα, σύγχρονοι φόρτοι εργασίας όπως τα *bio*, *inference* και *ml* προσεγγίζουν ή και υπερβαίνουν τα 10 GB (με το *bio* να αγγίζει τα 25 GB ακόμη και στο μικρό σύνολο δεδομένων). Αυτή η κατακόρυφη αύξηση υπαγορεύεται από την ανάγκη φόρτωσης εκτενών μοντέλων μηχανικής μάθησης (βάρη νευρωνικών δικτύων) ή τη διατήρηση πολύπλοκων γονιδιωματικών δομών στη μνήμη [37]. Είναι θεμελιώδες να τονιστεί ότι, σε πλήρη αντιστοιχία με τα ευρήματα της ανάλυσης του χρόνου CPU, η υψηλή κατανάλωση μνήμης πηγάζει αποκλειστικά από τα εξωτερικά εκτελέσιμα και τα περιβάλλοντα εκτέλεσής τους (π.χ. PyTorch) και όχι από τον ίδιο τον διερμηνέα κελύφους.

Κανονικοποιημένη Μνήμη και Διατήρηση Κατάστασης (Statefulness) Το κάτω γράφημα εμβαθύνει στη συμπεριφορά των μετροπρογραμμάτων, κανονικοποιώντας τη μέγιστη μνήμη ως προς το μέγεθος των δεδομένων εισόδου. Η προσέγγιση αυτή αναδεικνύει τον βαθμό διατήρησης εσωτερικής κατάστασης (statefulness) του εκάστοτε υπολογισμού.

Σύνολα όπως τα `m1` και `web-search` δεσμεύουν δυσανάλογα μεγάλη μνήμη συγκριτικά με τον όγκο των αρχείων που διαβάζουν (απαιτώντας δεκάδες έως εκατοντάδες bytes μνήμης ανά byte εισόδου), γεγονός που επιβεβαιώνει τη δημιουργία απαιτητικών εσωτερικών ευρετηρίων ή την εκπαίδευση μοντέλων. Αντίθετα, φόρτοι όπως τα `covid` και `file-mod` καταγράφουν ελάχιστη μνήμη ανά μονάδα εισόδου, επιβεβαιώνοντας ότι διέπονται από μεγάλες ροές δεδομένων: επεξεργάζονται τεράστιους όγκους δεδομένων διατηρώντας αμελητέα ποσότητα πληροφορίας στη μνήμη RAM ανά πάσα χρονική στιγμή. Σημειώνεται ότι σενάρια χωρίς ρητή παροχή δεδομένων εισόδου (π.χ. `ci-cd` και `perl`) εξαιρούνται από τη συγκεκριμένη γραφική απεικόνιση.

Ανάλυση Εισόδου/Εξόδου (I/O)

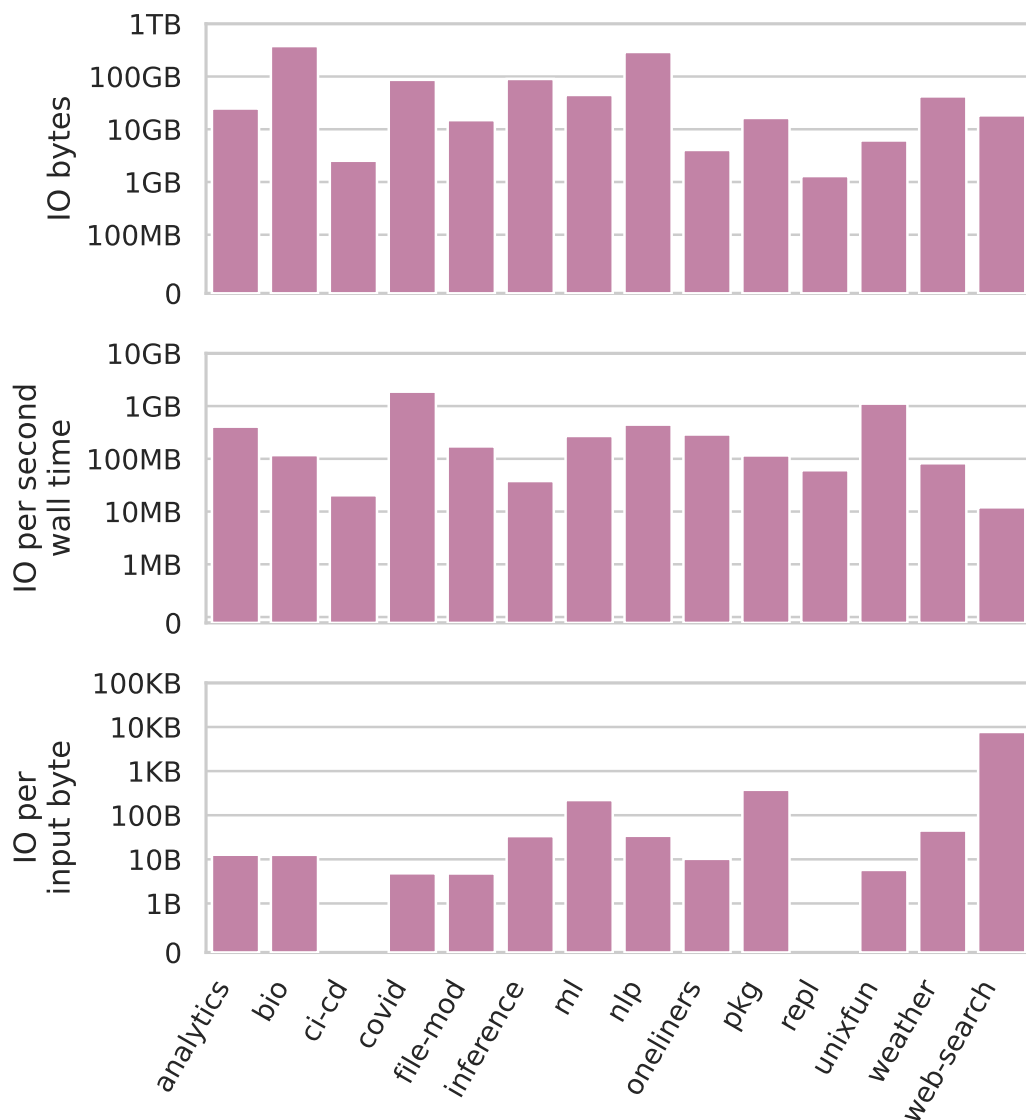
Δεδομένου ότι το προγραμματιστικό μοντέλο του UNIX εδράζεται παραδοσιακά στη ροή και τον χειρισμό αρχείων, η συμπεριφορά εισόδου/εξόδου (I/O) συνιστά κεντρικό πυλώνα του δυναμικού χαρακτηρισμού. Το Σχήμα 5.3 συνοψίζει τον απόλυτο όγκο δεδομένων I/O, τον ρυθμό διαμεταγωγής (throughput), καθώς και τον κανονικοποιημένο όγκο I/O ανά byte εισόδου.

Απόλυτος Όγκος I/O Όπως απεικονίζεται στο άνω γράφημα, η ένταση δεδομένων παρουσιάζει σημαντικές αποκλίσεις μεταξύ των διαφορετικών πεδίων. Σενάρια όπως τα `ci-cd` και `perl` καταγράφουν περιορισμένη δραστηριότητα I/O (π.χ. ~75.9 MB για το πρώτο), καθώς η φύση τους είναι κατεξοχήν υπολογιστική (π.χ. μεταγλώττιση λογισμικού). Αντιθέτως, φόρτοι μαζικής επεξεργασίας δεδομένων, όπως το `bio`, αγγίζουν έως και τα 336 GB αθροιστικού I/O κατά την εκτέλεση του πλήρους συνόλου δεδομένων, στοιχείο που τους καθιστά έντονα εξαρτώμενους από την απόδοση του υποσυστήματος αποθήκευσης (storage-bound). Ο μέσος όγκος εισόδου/εξόδου για το σύνολο της σουίτας υπολογίζεται περίπου στα 66.6 GB.

Ρυθμός Διαμεταγωγής (I/O Throughput) Το μεσαίο γράφημα αναλύει τον ρυθμό I/O ανά δευτερόλεπτο πραγματικού χρόνου εκτέλεσης, διακρίνοντας με σαφήνεια τους φόρτους που περιορίζονται από τον δίσκο (I/O-bound) από εκείνους που περιορίζονται από τον επεξεργαστή (CPU-bound). Ενδεικτικά, μετροπρογράμματα όπως τα `covid` και `unixfun` σαρώνουν και εγγράφουν δεδομένα με ρυθμούς που προσεγγίζουν το 1 GB/s, καταδεικνύοντας ότι η απόδοσή τους εξαρτάται άμεσα από το εύρος ζώνης (bandwidth) του αποθηκευτικού μέσου. Στον αντίποδα, φόρτοι όπως τα `ci-cd`, `file-mod` και `inference` παρουσιάζουν συγκριτικά χαμηλό ρυθμό διαμεταγωγής, επιβεβαιώνοντας ότι ο κύριος όγκος του χρόνου τους αναλώνεται σε υπολογιστικά εντατικές διεργασίες στην ΚΜΕ.

Κανονικοποιημένο I/O ανά Byte Εισόδου Η κανονικοποίηση του συνολικού I/O ως προς το μέγεθος των αρχικών δεδομένων εισόδου (κάτω γράφημα) παρέχει κρίσιμες πληροφορίες για τον κύκλο ζωής των δεδομένων κατά την εκτέλεση.

Σενάρια όπως τα `web-search`, `pkg` και `m1` παράγουν δυσανάλογα μεγάλο όγκο I/O (απαιτώντας αρκετά KB δραστηριότητας I/O για κάθε 1 byte εισόδου). Η συμπεριφορά αυτή δικαιολογείται από τη συνεχή δημιουργία ενδιάμεσων προσωρινών αρχείων, την εκτεταμένη παραγωγή ευρετηρίων (indexing) ή τις πολλαπλές σαρώσεις των ίδιων δεδομένων (multi-pass algorithms). Αντίθετα, κλασικά εργαλεία επεξεργασίας ροών στα σύνολα `covid`, `file-mod` και `unixfun` προσεγγίζουν την ιδανική αναλογία (κοντά στο 1 προς 1), καθώς κατά κανόνα διαβάζουν γραμμικά την είσοδο και εξάγουν ένα συμπυκνωμένο αποτέλεσμα, χωρίς να πολλαπλασιάζουν τον όγκο των δεδομένων στον δίσκο. Σύμφωνα με την καθιερωμένη πρακτική, μετροπρογράμματα χωρίς ρητή είσοδο μεγάλων αρχείων (π.χ. `ci-cd` και `perl`) παραλείπονται από το κανονικοποιημένο γράφημα.



Σχήμα 5.3: Ανάλυση Εισόδου/Εξόδου (I/O). Παρουσίαση σε συμμετρική λογαριθμική κλίμακα. Πάνω: Απόλυτος συνολικός όγκος I/O (Bytes). Μέση: Ρυθμός I/O ανά δευτερόλεπτο πραγματικού χρόνου εκτέλεσης (IO per second wall time). Κάτω: Συνολικό I/O κανονικοποιημένο ανά byte δεδομένων εισόδου

Αλληλεπίδραση με τον Πυρήνα και Περιγραφείς Αρχείων

Πέρα από τον συνολικό όγκο δραστηριότητας εισόδου/εξόδου, ιδιαίτερο ενδιαφέρον παρουσιάζει ο βαθμός αλληλεπίδρασης κάθε μετροπρογράμματος με τον πυρήνα του λειτουργικού συστήματος. Για τον σκοπό αυτό, καταγράφεται ο συνολικός αριθμός κλήσεων συστήματος (*system calls*) μέσω του εργαλείου *strace*, καθώς και ο μέγιστος αριθμός ταυτόχρονα ανοικτών περιγραφών αρχείων (*file descriptors*) κατά τη διάρκεια της εκτέλεσης, μέσω της συνεχούς παρακολούθησης με τη βιβλιοθήκη *psutil*.

Ο Πίνακας 5.1 συνοψίζει τα σχετικά αποτελέσματα. Ο συνολικός αριθμός κλήσεων συστήματος αποτελεί άμεσο δείκτη της έντασης επικοινωνίας με τον πυρήνα (π.χ. δημιουργία νέων διεργασιών, δέσμευση μνήμης, προσπελάσεις αποθηκευτικού μέσου). Αντίστοιχα, η μέγιστη τιμή των ανοικτών περιγραφών αρχείων παρέχει μια ισχυρή ένδειξη της αρχιτεκτονικής πολυπλοκότητας, αποτυπώνοντας τον βαθμό χρήσης σωληνώσεων, ανακατευθύνσεων και ταυτοχρονισμού (*concurrency*) του εκάστοτε φόρτου εργασίας.

Πίνακας 5.1: Συνολικός αριθμός κλήσεων συστήματος και μέγιστος αριθμός ανοικτών περιγραφέων αρχείων ανά σύνολο μετροπρογραμμάτων. Για το μικρό -small μέγεθος δεδομένων εισόδου.

Benchmark	System Calls	File Descriptors
analytics	117.322.520	84
bio	35.258.065	79
ci-cd	3.460.149	885
covid	14.345.907	150
file-mod	1.472.282	61
inference	37.947.319	81
ml	14.852.620	10
nlp	178.596.496	526
oneliners	4.451.080	288
pkg	35.194.614	132
repl	5.640.039	79
unixfun	943.977	733
weather	56.712.143	50
web-search	112.613.314	174

Τα δεδομένα αναδεικνύουν εντυπωσιακή διαφοροποίηση μεταξύ των φόρτων εργασίας. Σύνολα όπως τα nlp, analytics και web-search καταγράφουν εκατοντάδες εκατομμύρια κλήσεις συστήματος. Η συμπεριφορά αυτή υποδηλώνει συνεχή διασύνδεση και μεταφορά δεδομένων μεταξύ πλήθους μικρών διεργασιών μέσω ανώνυμων σωληνώσεων (pipes), συνεπάγοντας υψηλό κόστος διεργασιακής επικοινωνίας (Inter-Process Communication overhead). Στον αντίποδα, φόρτοι όπως τα file-mod και unixfun παρουσιάζουν συγκριτικά εξαιρετικά χαμηλό συνολικό αριθμό κλήσεων (πέριξ του ενός εκατομμυρίου).

Η ανάλυση των ανοικτών περιγραφέων αρχείων αποκαλύπτει εξίσου ετερογενή προφίλ. Μετροπρογράμματα όπως τα ci-cd και unixfun απαιτούν τη διατήρηση τεράστιου αριθμού ταυτόχρονα ανοικτών περιγραφέων (885 και 733, αντίστοιχα). Στην περίπτωση του ci-cd, το φαινόμενο αυτό ερμηνεύεται από τη μαζική και παράλληλη προσπέλαση αρχείων πηγαίου κώδικα και βιβλιοθηκών κατά τη φάση μεταγλώττισης (compilation), ενώ στο unixfun δικαιολογείται από την παρουσία εκτεταμένων δέντρων σωληνώσεων. Αντίθετα, βαριά υπολογιστικοί φόρτοι (compute-bound), όπως το ml, λειτουργούν με μόλις 10 ανοικτούς περιγραφείς, εστιάζοντας τους πόρους τους αποκλειστικά στη διαχείριση της μνήμης και του επεξεργαστή.

5.3 Σύνοψη Δυναμικών Ευρημάτων

Η δυναμική ανάλυση τεκμηριώνει αναντίρρητα ότι η σουίτα ΚΟΑΛΑ δεν αποτελεί μια ομοιογενή συλλογή απλών σεναρίων, αλλά ένα σύνθετο και ετερογενές οικοσύστημα συμπεριφορών [37]. Οι καταγεγραμμένες μετρικές αποδεικνύουν ότι τα μετροπρογράμματα καλύπτουν όλο το φάσμα των υπολογιστικών απαιτήσεων:

- **Χρόνος Εκτέλεσης:** Η σουίτα περιλαμβάνει από βραχύβια σενάρια ελέγχου ελάχιστων δευτερολέπτων έως βαριές εκτελέσεις οι οποίες, στα πλήρη σύνολα δεδομένων τους, διαρκούν αρκετές ώρες. Παράλληλα, η ποσοστιαία κατανομή του υπολογιστικού χρόνου μεταξύ του διερμηνέα κελύφους και των εξωτερικών εκτελέσιμων ποικίλλει δραστικά ανάλογα με το πεδίο.
- **Κατανάλωση Μνήμης:** Το αποτύπωμα μνήμης κυμαίνεται από εξαιρετικά ελαφριές διαδικασίες ροής (streaming), που απαιτούν λιγότερο από 1 MB, έως διεργασίες διατήρησης κατάστασης (stateful) και φορτώσεις μοντέλων που δεσμεύουν δεκάδες GB μνήμης.

- **Δραστηριότητα I/O:** Το προφίλ εισόδου/εξόδου εκτείνεται από αμιγώς CPU-bound εφαρμογές με ελάχιστη χρήση δίσκου, έως I/O-bound εργασίες μαζικής σάρωσης αρχείων, με ρυθμούς διαμεταγωγής που προσεγγίζουν το 1 GB/s και αθροιστικό όγκο δεδομένων εκατοντάδων GB.

Η εγγενής αυτή ποικιλομορφία διασφαλίζει ότι τα συστήματα βελτιστοποίησης που αξιολογούνται μέσω του ΚΟΑΛΑ υποβάλλονται σε ρεαλιστικές δοκιμασίες, αντιμετωπίζοντας όλα τα πιθανά σημεία συμφόρησης (bottlenecks) —από τα όρια της υπολογιστικής ισχύος και του εύρους ζώνης μνήμης, έως τις καθυστερήσεις του αποθηκευτικού μέσου και το κόστος αλληλεπίδρασης με τον πυρήνα. Υπό αυτό το πρίσμα, η σουίτα προσφέρει στην ερευνητική κοινότητα ένα ολοκληρωμένο και αυστηρό πλαίσιο για τη συγκριτική αξιολόγηση και την ανάδειξη των δυνατοτήτων των σύγχρονων εργαλείων επιτάχυνσης κελύφους.

Κεφάλαιο 6

Αξιολόγηση Συστημάτων Βελτιστοποίησης

Στο παρόν κεφάλαιο αξιοποιείται η μεθοδολογία και η σουίτα μετροπρογραμμάτων KOALA, όπως αυτές αναπτύχθηκαν στα προηγούμενα κεφάλαια, με σκοπό τη συγκριτική αξιολόγηση τεσσάρων αντιπροσωπευτικών συστημάτων επιτάχυνσης κελύφους: των Shark, GNU parallel, PASH και hS. Όπως τεκμηριώθηκε στο θεωρητικό υπόβαθρο (Κεφάλαιο 2), τα εν λόγω συστήματα ενσωματώνουν θεμελιωδώς διαφορετικές στρατηγικές βελτιστοποίησης.

Βασικός στόχος της παρούσας αξιολόγησης δεν είναι αποκλειστικά η καταγραφή της απόλυτης επίδοσης (performance) των συστημάτων, αλλά και η συστηματική αποτίμηση της προσπάθειας υιοθέτησής τους (adoption effort). Ο απαιτούμενος χρόνος και ο βαθμός δυσκολίας κατά τον μετασχηματισμό των σεναρίων συνιστούν καθοριστικούς παράγοντες για την πρακτική εφαρμοσιμότητα μιας λύσης [37]. Τα πειράματα διεξήχθησαν σε ελεγχόμενο περιβάλλον εκτέλεσης (στιγμιότυπο AWS c6i.4xlarge) εξοπλισμένο με 32GB μνήμης RAM και επεξεργαστή 16 πυρήνων στα 3.5 GHz, υπό το λειτουργικό σύστημα Ubuntu 24.04.1.

6.1 Κινητήριο Παράδειγμα: Το Σενάριο weather

Για την πληρέστερη κατανόηση του τρόπου με τον οποίο κάθε σύστημα προσεγγίζει τη βελτιστοποίηση στην πράξη, εισάγεται ως σημείο αναφοράς το μετροπρόγραμμα ανάλυσης μετεωρολογικών δεδομένων weather (Κώδικας 3.1). Ο αρχικός κώδικας (Κώδικας 3.1) συνθέτει μια επαναληπτική δομή ελέγχου (βρόχος for) με τρεις ανεξάρτητες σωληνώσεις (pipelines), οι οποίες διαβάζουν το ίδιο αρχείο εισόδου. Ο συνδυασμός αυτών των χαρακτηριστικών αναδεικνύει ιδανικά τόσο τις δυνατότητες παραλληλισμού όσο και τις δομικές απαιτήσεις μετασχηματισμού που επιβάλλει η εκάστοτε στρατηγική επιτάχυνσης.

6.2 Shark: Στατική Βελτιστοποίηση Εισόδου/Εξόδου

Η προσέγγιση του Shark [6] στοχεύει στην επιτάχυνση μέσω της ελαχιστοποίησης του κόστους εισόδου/εξόδου (I/O) και της εξάλειψης περιττών διεργασιών. Δεδομένου ότι η πρωτότυπη υλοποίησή του δεν υποστηρίζει πλήρως τη σύγχρονη σύνταξη του bash, οι απαιτούμενοι μετασχηματισμοί στα επιλεγμένα μετροπρογράμματα εφαρμόστηκαν χειροκίνητα. Η διαδικασία αυτή απαίτησε σημαντική επένδυση χρόνου για τη στατική ανάλυση του κώδικα (αρκετές ώρες ανά σύνολο), προκειμένου να διασφαλιστεί απολύτως η ορθότητα των παραγόμενων αποτελεσμάτων.

Μετασχηματισμός στο weather

Εφαρμόζοντας τις αρχές βελτιστοποίησης του Shark (Κώδικας 6.1), υλοποιούνται δύο κύριες παρεμβάσεις. Αρχικά, απαλείφεται η περιττή κλήση της εντολής cat, αντικαθιστώντας την με άμεση ανακατεύθυνση εισόδου (<). Εν συνεχεία, εκμεταλλευόμενοι τη στατική γνώση ότι οι τρεις σωληνώσεις γράφουν σε διακριτά αρχεία εξόδου, τις τοποθετούμε στο παρασκήνιο (μέσω του τελεστή &) και εισάγουμε την εντολή συγχρονισμού wait, επιτυγχάνοντας με αυτόν τον τρόπο ρητό παραλληλισμό σε επίπεδο γραμμής εντολών.

```

1  #!/bin/bash
2  d="./data/temperatures"
3  for y in $(seq $start $end); do
4      cut -c 89-92 < "$d/$y" | grep -v 999 |
5          sort -rn | head -n 1 > "max.$y" &
6      cut -c 89-92 < "$d/$y" | grep -v 999 |
7          sort -n | head -n 1 > "min.$y" &
8      cut -c 89-92 < "$d/$y" | grep -v 999 |
9          awk '{t+=$1; i++;} END {print t/i}' > "avg.$y" &
10     wait
11 done

```

Κώδικας 6.1: Το σενάριο `weather` μετασχηματισμένο βάσει του `Shark`. Η διαδικασία απαίτησε χειροκίνητη παρέμβαση για την αφαίρεση της `cat` και την προσθήκη τελεστών παραλληλισμού στο παρασκήνιο.

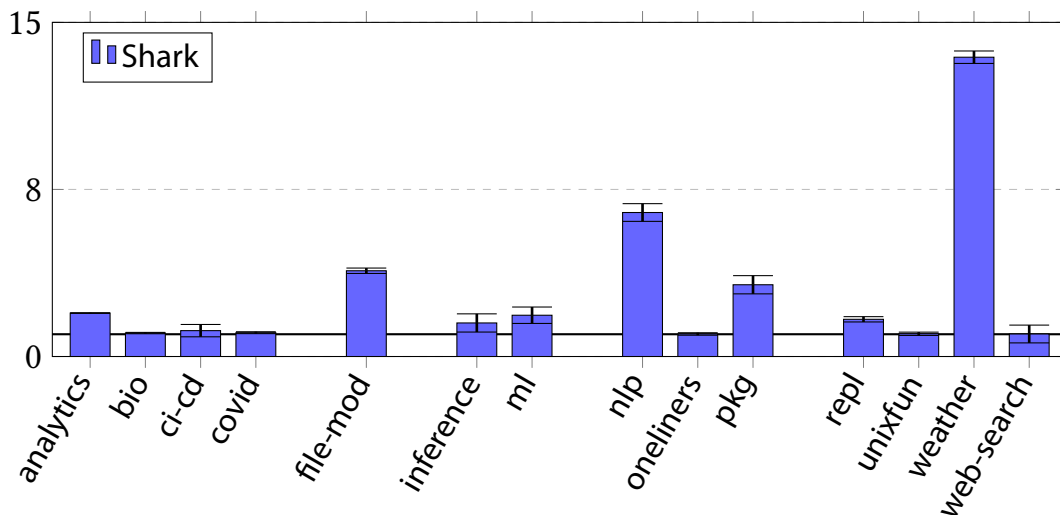
Δυσκολία Υιοθέτησης

Ενώ το παράδειγμα του `weather` απαίτησε ήπιες παρεμβάσεις, το μετροπρόγραμμα `count_trigrams` αναδεικνύει το πραγματικό κόστος υιοθέτησης του `Shark` σε ρεαλιστικά σενάρια ανάλυσης δεδομένων. Το αρχικό σενάριο (Κώδικας C.1) ακολουθεί μια κλασική προσέγγιση «δημιουργίας προσωρινών αρχείων» (`tempfiles`), η οποία είναι φιλική προς τον προγραμματιστή και εύκολη στην αποσφαλμάτωση.

Για να επιτευχθούν οι στόχοι βελτιστοποίησης του `Shark`—συγκεκριμένα η μετατροπή των ενδιάμεσων αρχείων σε ροές και ο παραλληλισμός—απαιτήθηκε ριζικός ανασχεδιασμός της αρχιτεκτονικής του κώδικα, όπως φαίνεται στον Κώδικα C.2.

Η διαδικασία μετασχηματισμού ανέδειξε τέσσερις βασικές προκλήσεις:

1. **Διαχείριση Πολυπλοκότητας Ροών:** Η αντικατάσταση των προσωρινών αρχείων με επώνυμες σωληνώσεις (FIFOs) επέβαλε τη χρήση της εντολής `tee` για τον διαχωρισμό της εισόδου σε πολλαπλά κανάλια. Αυτό εισάγει το πρόβλημα του συγχρονισμού Παραγωγού-Καταναλωτή. Όπως φαίνεται στον Κώδικα C.2, οι εντολές κατανάλωσης (`tail`) πρέπει να εκκινούν στο παρασκήνιο πριν την εγγραφή στα FIFOs, διαφορετικά το σενάριο οδηγείται σε αδιέξοδο (`deadlock`).
2. **Χειροκίνητος Παραλληλισμός:** Το `Shark` δεν προσφέρει αυτόματο μηχανισμό διαμοιρασμού φόρτου. Στο βελτιστοποιημένο σενάριο, αναγκαστήκαμε να υλοποιήσουμε έναν μηχανισμό διαχείρισης εργασιών (`job control`) με χρήση μετρητών (`job_count`) και της εντολής `wait`, ώστε να εκμεταλλευτούμε τους πολλαπλούς πυρήνες του συστήματος χωρίς να εξαντλήσουμε τους πόρους (*fork bomb*).
3. **Διόγκωση Κώδικα:** Η έκταση του κώδικα υπερδιπλασιάστηκε, ενώ η αναγνωσιμότητα μειώθηκε δραματικά. Η λογική της επεξεργασίας κρύβεται πλέον πίσω από λεπτομέρειες υλοποίησης του συστήματος αρχείων και διαχείρισης διεργασιών.
4. **Πρόκληση Ορθότητας και Χρονικό Κόστος:** Η διασφάλιση της ορθότητας (`correctness`) κατά τον μετασχηματισμό καθίσταται εξαιρετικά επίπονη, καθώς η χειροκίνητη διαχείριση ασύγχρονων ροών αυξάνει τον κίνδυνο για συνθήκες ανταγωνισμού (`race conditions`). Ως εκ τούτου, ο χρόνος που απαιτείται για την ανάπτυξη και την επαλήθευση του βελτιστοποιημένου κώδικα είναι σημαντικός, καθιστώντας την προσέγγιση ασύμφορη για σενάρια που απαιτούν γρήγορη υλοποίηση.



Σχήμα 6.1: Σχετικές επιταχύνσεις του Shark στα μετροπρογράμματα της σουίτας KOALA.

Ανάλυση Απόδοσης

Το Shark επέτυχε εντυπωσιακές βελτιώσεις απόδοσης στα σύνολα `weather` (13.43×) και `nlp` (6.46×). Παρά ταύτα, η συγκεκριμένη προσέγγιση παρουσιάζει σημαντικούς πρακτικούς περιορισμούς:

- **Υψηλό Κόστος Υιοθέτησης:** Η χειροκίνητη αντικατάσταση ενδιάμεσων προσωρινών αρχείων με επώνυμες σωληνώσεις (FIFOs), όπως επιχειρήθηκε στο σύνολο `nlp`, αυξάνει δραματικά την πολυπλοκότητα του κώδικα, καθιστώντας τον δυσανάγνωστο και επιρρεπή σε αδιέξοδα (deadlocks) κατά τον συγχρονισμό.
- **Σειριακές Εξαρτήσεις:** Σε φόρτους εργασίας με αυστηρή ακολουθιακή λογική, όπως τα `ci-cd` και `web-search`, τα περιθώρια στατικού παραλληλισμού είναι ελάχιστα. Ως αποτέλεσμα, η προσέγγιση προσφέρει οριακές μόνο επιταχύνσεις (1.01×–1.16×), ανεξαρτήτως της προσπάθειας μετασχηματισμού που καταβάλλεται.

Συμπερασματικά, το Shark αποδεικνύει ότι η στατική βελτιστοποίηση I/O μπορεί να προσφέρει σημαντικά κέρδη σε σενάρια μαζικής επεξεργασίας δεδομένων, ωστόσο η απαίτηση για βαθιά γνώση προγραμματισμού συστήματος και η έλλειψη αυτοματοποίησης το καθιστούν λιγότερο ελκυστικό για τον μέσο χρήστη του κελύφους που αναζητά άμεση επιτάχυνση.

6.3 GNU parallel: Χειροκίνητος Παραλληλισμός

Το GNU parallel [51], αν και παρέχει ισχυρές δυνατότητες παραλληλισμού, προϋποθέτει την πλέον ριζική και χρονοβόρα χειροκίνητη αναδιάρθρωση του πηγαιού κώδικα. Κατά τη διαδικασία αυτή, ο προγραμματιστής καλείται να απομονώσει τη λογική του σεναρίου σε διακριτές συναρτήσεις, να εξάγει ορθά τις απαραίτητες μεταβλητές στο περιβάλλον (μέσω της εντολής `export`) και να διαχειριστεί τις διαφυγές χαρακτήρων. Η τελευταία απαίτηση οδηγεί συχνά σε ιδιαίτερα περίπλοκες δομές εισαγωγικών, καθιστώντας την αποσφαλμάτωση μια εξαιρετικά επίπονη διαδικασία.

Μετασχηματισμός στο `weather`

Στην περίπτωση του `weather`, η λογική του βρόχου ενθυλακώνεται στη συνάρτηση `process_year`, ενώ η τροφοδοσία των ετών υλοποιείται μέσω της εντολής `seq`, η οποία διοχετεύει τα δεδομένα απευθείας στο `parallel` (Κώδικας 6.2).

```

1  #!/bin/bash
2  d="./data/temperatures"
3  process_year() {
4      y=$1
5      cut -c 89-92 "$d/$y" | grep -v 999 |
6          sort -rn | head -n 1 > "max.$y"
7      cut -c 89-92 "$d/$y" | grep -v 999 |
8          sort -n | head -n 1 > "min.$y"
9      cut -c 89-92 "$d/$y" | grep -v 999 |
10         awk '{t+= $1; i++;} END{print t/i}' > "avg.$y"
11 }
12 export -f process_year
13 export d
14 seq "$start" "$end" | parallel --jobs "$(nproc)" process_year

```

Κώδικας 6.2: Το σενάριο `weather` μετασχηματισμένο βάσει του GNU `parallel`. Η διαδικασία απαιτήσε ριζική τροποποίηση της αρχιτεκτονικής του κώδικα.

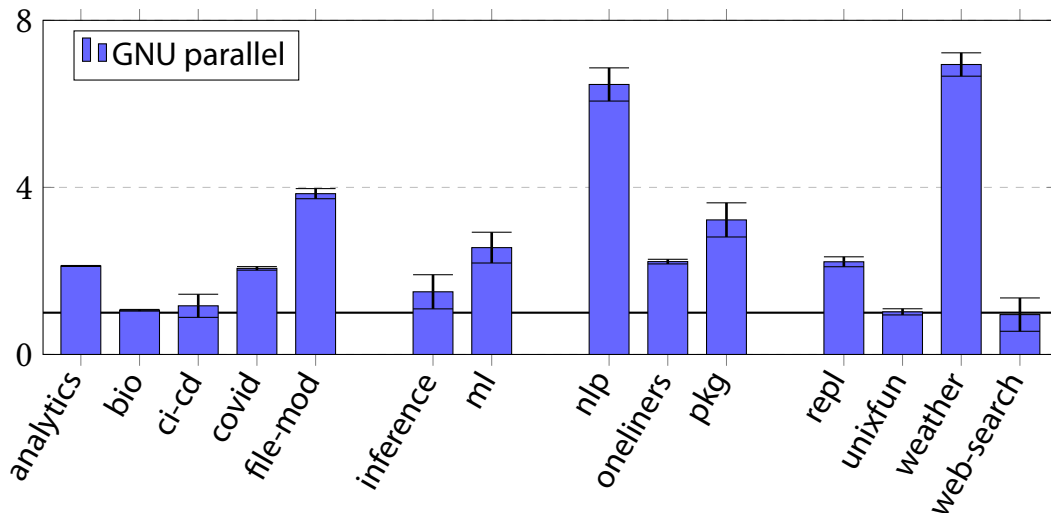
Δυσκολία Υιοθέτησης

Παρά την αυτοματοποίηση που προσφέρει, η ορθή χρήση του GNU `parallel` σε σύνθετα σενάρια αποδεικνύεται στην πράξη μια μη τετριμμένη διαδικασία. Όπως αναλύεται στη σχετική βιβλιογραφία [26], η μετατροπή σειριακών σωληνώσεων σε παράλληλες απαιτεί συχνά πολλαπλές επαναλήψεις για την επίτευξη ορθότητας και βέλτιστης επίδοσης.

Διαχείριση Ενδιάμεσων Δεδομένων και Σωληνώσεων Σε αντίθεση με την απλή σύνταξη του κελύφους, το GNU `parallel` απαιτεί από τον χρήστη να διαχειριστεί ρητά τη ροή των δεδομένων. Στο παράδειγμα του σεναρίου από το σύνολο `covid` (Κώδικας C.4), παρατηρείται η ανάγκη χρήσης προσωρινών καταλόγων (`mktemp`), ορισμού μεγέθους τεμαχίων δεδομένων (`chunk_size`) και η χρήση μηχανισμών καθαρισμού (`trap`) για τα προσωρινά αρχεία. Η απλή σωληνώση του αρχικού σεναρίου (Κώδικας C.3) μετατρέπεται σε ένα σενάριο που απαιτεί εξαγωγή συναρτήσεων (`export -f`) και συντονισμό μέσω της παραμέτρου `-pipe`.

Πολυπλοκότητα Σύνταξης και Ορθότητα Μια ακόμη σημαντικότερη πρόκληση ανακύπτει σε σενάρια που απαιτούν συγχρονισμό ή διατήρηση της σειράς των δεδομένων, όπως το `spell`. Στο αρχικό σενάριο (Κώδικας C.5), η ροή είναι γραμμική. Ωστόσο, στην παράλληλη εκδοχή (Κώδικας C.6), η ανάγκη για διατήρηση της σειράς (ώστε να λειτουργήσουν σωστά εντολές όπως η `uniq`) επιβάλλει τη χρήση της σημαίας `-k` (*keep order*) [26]. Η παράλειψή της οδηγεί σε λανθασμένα αποτελέσματα λόγω του μη-ντετερμινιστικού χρονοπρογραμματισμού των εργασιών.

Επιπλέον, η διαχείριση πολλαπλών ροών εισόδου/εξόδου απαιτεί συχνά τη χειροκίνητη δημιουργία επώνυμων σωληνώσεων (`mkfifo`) και πολύπλοκες δομές εντολών που περιλαμβάνουν χαρακτηριστικές διαφυγής για να περάσουν σωστά τα ορίσματα στο υποκείμενο κέλυφος. Όπως φαίνεται στον Κώδικα C.6, η εντολή γίνεται δυσανάγνωστη και δύσκολη στη συντήρηση, ενώ η απόδοση εξαρτάται δραματικά από «μαγικές» σταθερές, όπως το μέγεθος του μπλοκ (`-block`), όπου λανθασμένη ρύθμιση μπορεί να οδηγήσει σε δραματικές διαφορές στους χρόνους εκτέλεσης (π.χ. από λεπτά σε δευτερόλεπτα) [26].



Σχήμα 6.2: Σχετικές επιταχύνσεις του GNU parallel στα μετροπρογράμματα της σουίτας ΚΟΑΛΑ.

Ανάλυση Απόδοσης

Ιδιαίτερα σημαντική επιτάχυνση καταγράφηκε σε απολύτως παραλληλοποιήσιμα (embarrassingly parallel) σενάρια, όπως τα **weather** (7×), **nlp** (6.46×) και **file-mod** (3.84×). Αντιθέτως, σε φόρτους εργασίας όπως τα **repl** και **ci-cd** παρατηρήθηκε επιβράδυνση (0.95×) εξαιτίας του σημαντικού χρόνου εκκίνησης (startup overhead) του εργαλείου, καθιστώντας τον χρόνο που επενδύθηκε στην αναδιάρθρωση μη αποδοτικό. Παράλληλα, η εξολοκλήρου χειροκίνητη διαχείριση ενέχει σοβαρούς κινδύνους για την ορθότητα των αποτελεσμάτων (π.χ. στην περίπτωση παράλειψης της σημαίας `-k` για τη διατήρηση της αρχικής σειράς εξόδου).

6.4 ΡΑΣΗ: Παραλληλισμός Ροών Δεδομένων

Σε πλήρη αντίθεση με τις προαναφερθείσες προσεγγίσεις, ο μεταγλωττιστής πάνω-στην-ώρα (JIT) του ΡΑΣΗ [73] προσφέρει μια εμπειρία *μηδενικής προσπάθειας* υιοθέτησης. Ο χρήστης απαλλάσσεται πλήρως από την ανάγκη τροποποίησης έστω και μίας γραμμής του αρχικού κώδικα (όπως αυτός παρατίθεται στον Κώδικα 3.1). Η εκτέλεση πραγματοποιείται με απόλυτη διαφάνεια, απαιτώντας αποκλειστικά τον ορισμό της αντίστοιχης μεταβλητής περιβάλλοντος στον οδηγό της σουίτας: `KOALA_SHELL="pash"`.

Ο Εσωτερικός Μετασχηματισμός στο **weather**

Παρότι ο χρήστης εκτελεί το αρχικό σενάριο αμετάβλητο, το ΡΑΣΗ παρεμβάλλεται ως μεταγλωττιστής πάνω-στην-ώρα. Διαβάζοντας τις διαθέσιμες επισημειώσεις, αντιλαμβάνεται δυναμικά ότι η εντολή `cut` δεν διατηρεί εσωτερική κατάσταση, ενώ η `sort` απαιτεί λειτουργία συγχώνευσης (`merge`). Βάσει αυτών, κατασκευάζει και εκτελεί εσωτερικά έναν πολύπλοκο γράφο ροής δεδομένων, διασπώντας την είσοδο σε επώνυμες σωληνώσεις (FIFOs) και συγχωνεύοντας τα επιμέρους αποτελέσματα, όπως φαίνεται στο απλοποιημένο παράδειγμα (Κώδικας 6.3). Το πλήρες πρόγραμμα κελύφους που παράγεται από το ΡΑΣΗ για το σενάριο **weather** παρουσιάζεται στον Κώδικα B.1 στο Παράρτημα 8.2.

Ανάλυση Απόδοσης και το Πρόβλημα των Επισημειώσεων

Το ΡΑΣΗ επέτυχε σημαντικές επιταχύνσεις στα σύνολα **oneliners** (2.14×) και **covid** (1.82×). Εντούτοις, αναδείχθηκαν κρίσιμοι περιορισμοί:

```

1 mkfifo f{0..13}
2 cat "/data/temperatures/2000" >f0 &
3 split f0 f1 f2 &
4 cut -c 89-92 <f1 >f3 &
5 cut -c 89-92 <f2 >f4 &
6 grep -v 999 <f3 >f5 &
7 grep -v 999 <f4 >f6 &
8 eager <f5 >f7 &
9 eager <f6 >f8 &
10 sort -rn <f7 >f9 &
11 sort -rn <f8 >f10 &
12 eager <f9 >f11 &
13 eager <f10 >f12 &
14 sort -rn -m f11 f12 >f13 &
15 head -n 1 <f13 >"max.2000" &
16 # ...
17 # ...
18 # --- Alternatively, for the AVG calculation (Merge -> Sequential Awk) ---
19 merge f12 f13 | awk '{t+=$1; i++;} END {if (i>0) print t/i}' > "avg.2000"
20 wait
21 rm f{0..13}

```

Κώδικας 6.3: Απλοποιημένη σημασιολογική αναπαράσταση του σεναρίου *weather* με χρήση του PaSh. Παρουσιάζεται ο γράφος ροής δεδομένων (DFG) που δημιουργεί και εκτελεί αυτόματα το PaSh στο παρασκήνιο για μία εκ των σωληνώσεων του *weather*.

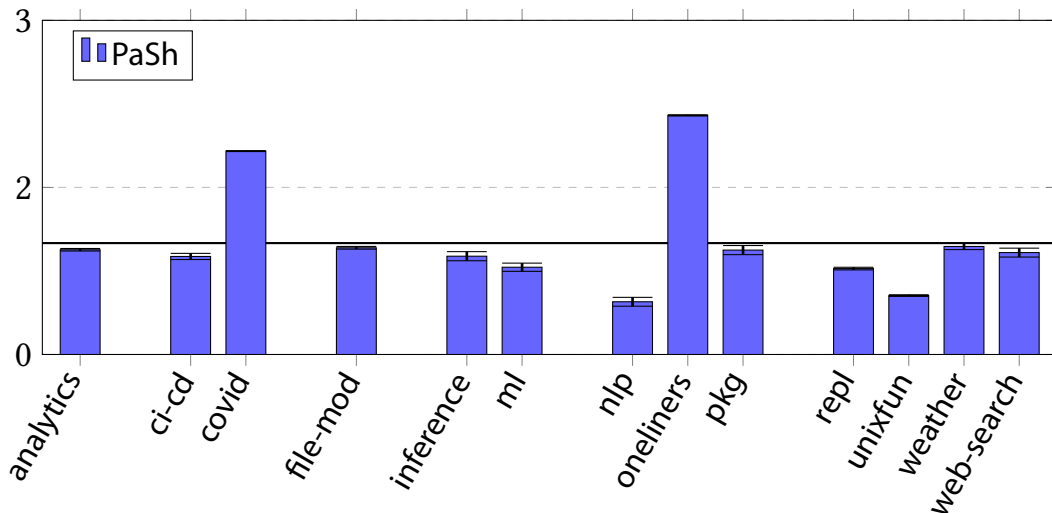
- **Η Εξάρτηση από τις Επισημειώσεις:** Σε πεδία όπως η βιοπληροφορική (*bio*), η παντελής έλλειψη επισημειώσεων για τα εξειδικευμένα εργαλεία αναγκάζει το PaSh να μεταπέσει σε ασφαλή σειριακή εκτέλεση (1.00×). Αν και η συγγραφή προσαρμοσμένων επισημειώσεων θα μπορούσε να αποφέρει τεράστια επιτάχυνση, αυτό πρακτικά μεταθέτει το «κόστος υιοθέτησης» από την αναδιάρθρωση του κώδικα στη χρονοβόρα μελέτη εγχειριδίων χρήσης για τη δημιουργία αυστηρών προδιαγραφών διεπαφής.
- **Κόστος Δυναμικής Μεταγλώττισης:** Στην περίπτωση του *weather*, αν και ο κώδικας είναι πλήρως παραλληλοποιήσιμος, το υπολογιστικό κόστος κατασκευής του γράφου και των σωληνώσεων (FIFOs) σε κάθε επανάληψη του βρόχου μειώνει τα κέρδη απόδοσης, περιορίζοντας την τελική επιτάχυνση στο οριακό 1.08×.

6.5 *hS*: Εκτέλεση Εκτός Σειράς

Το *hS* [40] συνιστά επίσης μια δυναμική προσέγγιση μηδενικής προσπάθειας, η οποία επιχειρεί να επιλύσει το πρόβλημα των «άγνωστων» εντολών παρακάμπτοντας ολοκληρωτικά την ανάγκη για επισημειώσεις. Η εκτέλεση επιτυγχάνεται με απόλυτη διαφάνεια μέσω του ορισμού: *KOALA_SHELL="hs"*. Το *hS* αναλαμβάνει την ιχνηλάτηση (μέσω μηχανισμών όπως το *FUSE* ή το *ptrace*) των προσπελάσεων στο σύστημα αρχείων, κάνοντας υποθετική εκτέλεση των σωληνώσεων.

Ο Χρονοδρομολογητής στο *weather*

Σε αντίθεση με τη δημιουργία FIFOs που υιοθετεί το PaSh, ο χρονοδρομολογητής (*scheduler*) του *hS* εκκινεί τις διαδοχικές σωληνώσεις (*pipelines* H1, H2, H3) εντός απομονωμένων περιβαλλόν-



Σχήμα 6.3: Σχετικές επιταχύνσεις του PaSh στα μετροπρογράμματα της σουίτας KOALA.

```

1 #!/bin/bash
2
3 d="./data/temperatures";
4 for y in $(seq $start $end); do
5     # pipeline H1
6     hs-speculate cat $d/$y | cut -c 89-92 | grep -v 999 |
7     sort -rn | head -n1 > max.$y
8     # pipeline H2
9     hs-speculate cat $d/$y | cut -c 89-92 | grep -v 999 |
10    sort -n | head -n1 > min.$y
11    # pipeline H3
12    hs-speculate cat $d/$y | cut -c 89-92 | grep -v 999 |
13    awk "{t += \$1; i++} END {print t/i}" > avg.$y
14 done

```

Κώδικας 6.4: Απλοποιημένη σημασιολογική αναπαράσταση του σεναρίου weather με χρήση του *hS*. Το σύστημα επιχειρεί υποθετική εκτέλεση των σωληνώσεων.

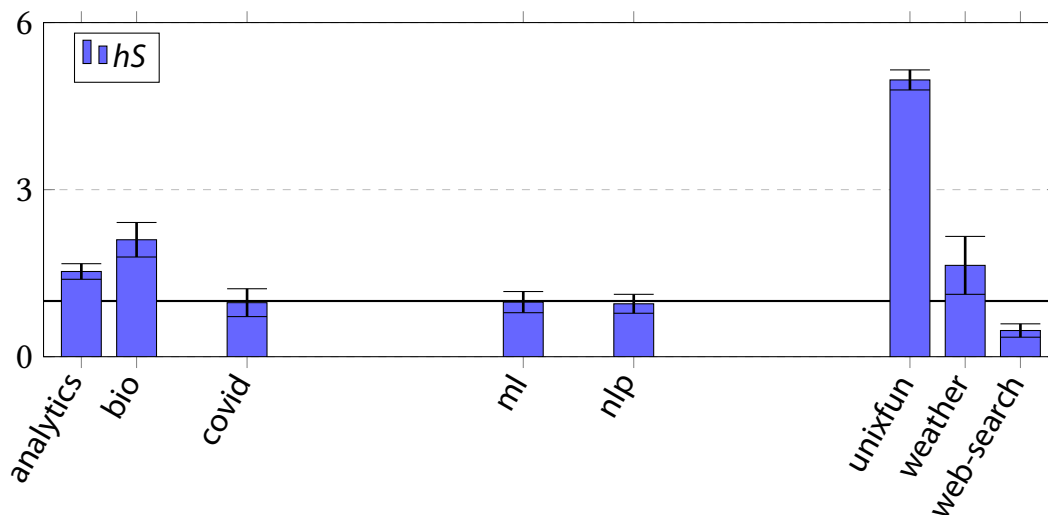
των εκτέλεσης (sandboxes) πριν καν ολοκληρωθούν οι προηγούμενες εξαρτήσεις τους. Ο Πίνακας 6.1 αποτυπώνει αναλυτικά αυτή τη συμπεριφορά για τον βρόχο του weather.

Ανάλυση Απόδοσης

Το *hS* έδειξε σημαντικές επιδόσεις στα σύνολα **weather** (4.97×) και **analytics** (2.60×), καθώς οι προβλέψεις του επιβεβαιώνονταν σταθερά, αποκρύπτοντας πλήρως τις καθυστερήσεις εισόδου/εξόδου. Ωστόσο, σε φόρτους εργασίας με ισχυρές εξαρτήσεις δεδομένων ή ταχύτερες μικρές διεργασίες (π.χ. **web-search**, **pkg**), οι συνεχείς ακυρώσεις και επαναφορές (*rollbacks*) και η υπολογιστική επιβάρυνση (*overhead*) της ιχνηλάτησης προκάλεσαν σοβαρές επιβραδύνσεις (έως 0.26×), αναδεικνύοντας το τίμημα της ανεξέλεγκτης υποθετικής εκτέλεσης.

Χρόνος	Ενέργεια Χρονοδρομολογητή <i>hS</i> (Στο παρασκήνιο)
t_0	Εκκίνηση H1(2000): Εκτελείται κανονικά (Frontier).
t_1	Υποθετική εκτέλεση (Speculation): Έναρξη των H2(2000) και H3(2000) σε απομονωμένα περιβάλλοντα (οι αναγνώσεις/εγγραφές ιχνηλατούνται).
t_2	Το κέλυφος προχωρά στο επόμενο έτος ($y=2001$). Ξεκινά η υποθετική H1(2001) ενώ οι H2/H3 του 2000 τρέχουν ακόμη.
t_3	Η H1(2000) ολοκληρώνεται.
t_4	Η H2(2000) ολοκληρώνεται. Δεν εντοπίζονται συγκρούσεις εισόδου/εξόδου (I/O conflicts) με την H1 → Οριστικοποίηση (Commit) στο Σύστημα Αρχείων.
t_5	Η H3(2000) ολοκληρώνεται. Δεν υπάρχουν συγκρούσεις → Οριστικοποίηση .
t_6	Ο βρόχος συνεχίζεται ταχύτατα, έχοντας πρακτικά επικαλύψει τις καθυστερήσεις (latency) των επιμέρους εντολών.

Πίνακας 6.1: Εσωτερική διαχείριση του κώδικα του σεναρίου *weather* από τον χρονοδρομολογητή του *hS*. Οι H1, H2 και H3 αντιστοιχούν στις τρεις σειριακές σωληνώσεις του βρόχου. Για το παράδειγμα, θεωρούμε ως έτος εκκίνησης (start) το 2000.



Σχήμα 6.4: Σχετικές επιταχύνσεις του *hS* στα μετροπρογράμματα της σουίτας ΚΟΑΛΑ.

6.6 Συγκριτική Σύνοψη Αξιολόγησης

Η αξιολόγηση μέσω της σουίτας ΚΟΑΛΑ τεκμηριώνει ότι κανένα σύστημα δεν αποτελεί την απόλυτη και καθολική λύση, ιδίως όταν συνυπολογιστεί η προσπάθεια υιοθέτησης:

- Τα εργαλεία GNU parallel και Shark προσφέρουν τη μέγιστη δυνατή απόδοση (έως 13×), ωστόσο προαπαιτούν εξαντλητική χειροκίνητη αναδιάρθρωση και εις βάθος κατανόηση των εξαρτήσεων των δεδομένων, γεγονός που τα καθιστά πρακτικά δύσχρηστα για κώδικα μεγάλης κλίμακας.
- Ο μεταγλωττιστής PASH διασφαλίζει απόλυτη ορθότητα και απαιτεί μηδενική παρέμβαση στον πηγαίο κώδικα, αλλά η αποτελεσματικότητά του μειώνεται δραματικά όταν τα υποκείμενα εργαλεία στερούνται χειροκίνητων επισημειώσεων.
- Το σύστημα *hS* υπερβαίνει τον σκόπελο των επισημειώσεων, όμως η επιβάρυνση της ιχνηλάτησης σε επίπεδο χρήστη και οι πιθανές ακυρώσεις λόγω εξαρτήσεων υποβαθμίζουν σημαντικά τα

όποια οφέλη σε πλήθος σεναρίων.

Κεφάλαιο 7

Συζήτηση

Η συστηματική ανάλυση μέσω της σουίτας ΚΟΑΛΑ αναδεικνύει ότι η επιτάχυνση προγραμμάτων κελύφους δεν αποτελεί ένα μονοδιάστατο πρόβλημα παραλληλισμού, αλλά έναν σύνθετο χώρο σχεδιαστικών επιλογών που εξαρτάται από τη φύση των φορτίων εργασίας, τη σημασιολογία των εντολών και τη δομή των ίδιων των σεναρίων. Ο συνδυασμός στατικού και δυναμικού χαρακτηρισμού των μετροπρογραμμάτων, καθώς και η εφαρμογή πολλαπλών συστημάτων επιτάχυνσης, προσφέρουν μια ολοκληρωμένη εικόνα του χώρου βελτιστοποίησης του κελύφους.

7.1 Διαφορετικές Στρατηγικές Επιτάχυνσης και τα Όριά τους

Η εφαρμογή των συστημάτων Shark, GNU parallel, PASH και hS στη σουίτα ΚΟΑΛΑ δείχνει ότι κάθε προσέγγιση επιταχύνει διαφορετικές κατηγορίες προγραμμάτων και αξιοποιεί διαφορετικές ευκαιρίες παραλληλισμού.

- **Συντακτικοί μετασχηματισμοί (Shark):** Το Shark επιτυγχάνει σημαντικές επιταχύνσεις σε σενάρια με ανεξάρτητες επαναλήψεις ή αγωγούς εντολών, φθάνοντας έως και 13.43× σε ορισμένες περιπτώσεις. Τα οφέλη μειώνονται όταν οι αγωγοί δεδομένων είναι ήδη βελτιστοποιημένοι ή όταν οι εξαρτήσεις μεταξύ σταδίων περιορίζουν τους διαθέσιμους μετασχηματισμούς.
- **Χειροκίνητος παραλληλισμός (GNU parallel):** Η χρήση του GNU parallel αποδίδει ιδιαίτερα καλά σε εργασίες έντονης εισόδου/εξόδου ή σε βρόχους χωρίς αλληλεξαρτήσεις, με μέσες επιταχύνσεις περίπου 2.6×. Ωστόσο, απαιτεί χειροκίνητη τροποποίηση των προγραμμάτων, γεγονός που περιορίζει τη γενικότητα της προσέγγισης.
- **Παραλληλοποίηση με γνώση εντολών (PASH):** Το PASH επιταχύνει σενάρια που περιέχουν αγωγούς εντολών, όμως η αποτελεσματικότητά του εξαρτάται από τη διαθεσιμότητα σχολιασμών και από το κατά πόσο το πρόγραμμα εμπίπτει στο μοντέλο παραλληλοποίησης που υποστηρίζει.
- **Εκτέλεση εκτός σειράς (hS):** Το hS προσφέρει οφέλη σε περιπτώσεις με ανεξάρτητα στάδια υπολογισμού, αλλά η απόδοσή του επηρεάζεται από το κόστος απομόνωσης και ιχνηλάτησης, και την ύπαρξη εξαρτήσεων, οδηγώντας σε αστάθεια ή ακόμη και επιβραδύνσεις σε ορισμένα προγράμματα.

Συνολικά, φαίνεται ότι δεν υπάρχει μία καθολικά βέλτιστη τεχνική επιτάχυνσης· κάθε σύστημα αξιοποιεί διαφορετικά χαρακτηριστικά του κελύφους και εμφανίζει διαφορετικούς περιορισμούς.

7.2 Το Κέλυφος ως Γλώσσα Ενορχήστρωσης

Η συντακτική ανάλυση δείχνει ότι τα προγράμματα κελύφους λειτουργούν κυρίως ως μηχανισμός ενορχήστρωσης εξωτερικών εργαλείων.

Παρότι οι πιο συχνές εντολές προέρχονται από POSIX εργαλεία και GNU Coreutils, το 54% των μοναδικών εντολών στα μετροπρογράμματα αντιστοιχεί σε προσαρμοσμένα εκτελέσιμα ή συναρτήσεις χρηστών. Το γεγονός αυτό επιβεβαιώνει ότι τα πραγματικά προγράμματα κελύφους λειτουργούν κυρίως ως συνδετικός μηχανισμός που ενοποιεί ετερογενή προγράμματα. Η ιδιότητα αυτή δυσκολεύει τις στατικές τεχνικές βελτιστοποίησης, καθώς η συμπεριφορά μεγάλου μέρους των εντολών παραμένει αδιαφανής για το σύστημα επιτάχυνσης.

7.3 Πολυμορφία Δυναμικής Συμπεριφοράς

Ο δυναμικός χαρακτηρισμός αποκαλύπτει ότι τα φορτία εργασίας του ΚΟΑΛΑ καλύπτουν ευρύ φάσμα συμπεριφορών ως προς τον χρόνο εκτέλεσης και τη χρήση πόρων.

- Ο χρόνος χρήσης επεξεργαστή κυμαίνεται από δευτερόλεπτα έως ώρες.
- Η χρήση μνήμης και το συνολικό μέγεθος εισόδου/εξόδου διαφέρουν κατά αρκετές τάξεις μεγέθους.
- Ορισμένα προγράμματα κυριαρχούνται από εξωτερικές εντολές, ενώ άλλα δαπανούν σημαντικό χρόνο στον ίδιο τον διερμηνέα του κελύφους.

Η ποικιλία αυτή εξηγεί γιατί τεχνικές που εστιάζουν αποκλειστικά στον παραλληλισμό δεδομένων δεν αποδίδουν ομοιόμορφα σε όλα τα μετροπρογράμματα. Το κέλυφος χρησιμοποιείται όχι μόνο για επεξεργασία δεδομένων αλλά και για ενορχήστρωση σύνθετων εργασιών συστήματος.

7.4 Η Αξία της Συστηματικής Αξιολόγησης

Ένα βασικό συμπέρασμα της εργασίας είναι ότι η απουσία καθιερωμένων μετροπρογραμμάτων αποτελούσε σημαντικό εμπόδιο στην έρευνα επιτάχυνσης του κελύφους.

Η σουίτα ΚΟΑΛΑ παρέχει:

- ρεαλιστικά φορτία εργασίας με πραγματικά δεδομένα,
- αυτοματοποιημένη εγκατάσταση και επικύρωση αποτελεσμάτων,
- αναπαραγώγιμη υποδομή αξιολόγησης,
- ευρεία κάλυψη συντακτικών και δυναμικών χαρακτηριστικών.

Η εφαρμογή της σουίτας σε πολλαπλά συστήματα έδειξε ότι αποτελέσματα που εμφανίζονται εντυπωσιακά σε μικρά τεχνητά παραδείγματα συχνά διαφοροποιούνται όταν αξιολογούνται σε ρεαλιστικά σενάρια. Επομένως, η ύπαρξη μιας συστηματικής και κοινά αποδεκτής σουίτας είναι απαραίτητη για δίκαιες συγκρίσεις και αξιόπιστη πρόοδο στο πεδίο.

7.5 Συμπέρασμα

Η ανάλυση μέσω της σουίτας καταδεικνύει ότι το κέλυφος UNIX αποτελεί ένα ιδιαίτερα ετερογενές περιβάλλον εκτέλεσης. Τα διαφορετικά φορτία εργασίας, οι εξωτερικές εντολές και η ποικιλία δυναμικών συμπεριφορών καθιστούν δύσκολη την ύπαρξη μιας ενιαίας στρατηγικής επιτάχυνσης. Η συμβολή του ΚΟΑΛΑ έγκειται στο ότι επιτρέπει, για πρώτη φορά, συγκρίσιμη και συστηματική αξιολόγηση συστημάτων επιτάχυνσης σε ρεαλιστικά σενάρια, αναδεικνύοντας τόσο τις δυνατότητες όσο και τα όριά τους.

Κεφάλαιο 8

Μελλοντικό Έργο και Συμπεράσματα

Η παρούσα εργασία παρουσίασε τη σουίτα ΚΟΑΛΑ, μια συστηματική σουίτα αξιολόγησης που επιτρέπει τη ρεαλιστική και αναπαραγώγιμη μελέτη του κελύφους UNIX. Η ανάλυση των δομικών χαρακτηριστικών και των πρακτικών περιορισμών των υφιστάμενων συστημάτων επιτάχυνσης παρέχει το απαραίτητο θεμέλιο για τον σχεδιασμό νέων βελτιστοποιήσεων. Η σύνθεση της στατικής και δυναμικής ανάλυσης επιβεβαιώνει ότι η αποτελεσματική επιτάχυνση απαιτεί μια ολιστική προσέγγιση που λαμβάνει υπόψη τόσο τη σύνταξη όσο και την πραγματική συμπεριφορά εκτέλεσης.

Η σουίτα αποδεικνύει ότι η ύπαρξη κοινής και αναπαραγώγιμης υποδομής αξιολόγησης αποτελεί κρίσιμο στοιχείο για την πρόοδο της έρευνας, επιτρέποντας ουσιαστικές συγκρίσεις μεταξύ συστημάτων κάτω από κοινές συνθήκες.

8.1 Μελλοντικό Έργο

Η υποδομή και τα ευρήματα της παρούσας εργασίας ανοίγουν πολλαπλές κατευθύνσεις για μελλοντική έρευνα.

Επέκταση της Σουίτας

Παρότι η σουίτα ΚΟΑΛΑ καλύπτει ευρύ φάσμα εφαρμογών, νέες κατηγορίες σεναρίων μπορούν να ενισχύσουν την αντιπροσωπευτικότητα της σουίτας:

- **Σενάρια έντονης δικτυακής αλληλεπίδρασης:** Σενάρια που βασίζονται σε συνεχή επικοινωνία με απομακρυσμένες υπηρεσίες ή σε συγχρονισμό δεδομένων θα επέτρεπαν τη μελέτη συστημάτων όπου η καθυστέρηση δικτύου κυριαρχεί.
- **Μη ντετερμινιστικές εκτελέσεις:** Σενάρια που εξαρτώνται από τυχαιότητα ή από δυναμική κατάσταση συστήματος μπορούν να αποκαλύψουν δυσκολίες σε τεχνικές παράλληλης εκτέλεσης και εκτέλεσης εκτός σειράς.
- **Διαδραστικές συνεδρίες:** Η μοντελοποίηση πραγματικής χρήσης του κελύφους θα επιτρέψει τη μελέτη του χρόνου απόκρισης (latency) και της εμπειρίας χρήστη.

Αξιολόγηση Νέων Αρχιτεκτονικών Συστημάτων

Η ποικιλία των μετροπρογραμμάτων καθιστά το ΚΟΑΛΑ κατάλληλο για την αξιολόγηση νέων κατηγοριών συστημάτων πέρα από την τοπική επιτάχυνση:

- **Κατανεμημένα shells:** Συστήματα όπως το DISH [47] και το POSH [58] επιτρέπουν εκτέλεση προγραμμάτων κελύφους σε πολλαπλούς κόμβους. Μετροπρογράμματα με μεγάλα δεδομένα προσφέρουν φυσικό πεδίο αξιολόγησης του κόστους και των οφελών της κατανεμημένης εκτέλεσης.
- **Ανοχή σφαλμάτων και συνέχιση εκτέλεσης:** Συστήματα όπως το FRACTAL [27] εισάγουν μηχανισμούς επαναφοράς και συνέχειας. Μακρόχρονα μετροπρογράμματα μπορούν να χρησιμοποιηθούν για τη μέτρηση της πραγματικής επιβάρυνσης αυτών των τεχνικών.

Σχεδιασμός Επόμενης Γενιάς Βελτιστοποιητών

Τα ευρήματα της στατικής και δυναμικής ανάλυσης υποδεικνύουν βασικές κατευθύνσεις για νέες τεχνικές:

- **Προηγμένη στατική ανάλυση:** Προσεγγίσεις όπως αυτή που παρουσιάζεται στο [38] στοχεύουν στην καλύτερη κατανόηση της σημασιολογίας των σεναρίων κελύφους, γεγονός που μπορεί να μειώσει την ανάγκη για χειροκίνητους σχολιασμούς.
- **Μείωση κόστους παρακολούθησης:** Η εμπειρία από συστήματα όπως το *hS* υποδεικνύει την ανάγκη για μετακίνηση της ιχνηλάτησης από το επίπεδο χρήστη (FUSE) σε επίπεδο πυρήνα (eBPF), μειώνοντας την υπολογιστική επιβάρυνση κατά την εκτέλεση εκτός σειράς.
- **Βελτιστοποίηση εισόδου/εξόδου:** Η αποδοτική διαχείριση της ροής δεδομένων είναι εξίσου σημαντική με τον παραλληλισμό.

8.2 Τελικά Συμπεράσματα

Η παρούσα διπλωματική κατέδειξε ότι τα προγράμματα κελύφους αποτελούν ένα πλούσιο και απαιτητικό υπολογιστικό περιβάλλον, με μεγάλη ποικιλία τόσο στη σύνταξη όσο και στη δυναμική τους συμπεριφορά. Η συστηματική ανάλυση και αξιολόγηση μέσω του ΚΟΑΛΑ επέτρεψε τη χαρτογράφηση των πραγματικών χαρακτηριστικών των σεναρίων κελύφους, τη δίκαιη σύγκριση διαφορετικών συστημάτων επιτάχυνσης και την ανάδειξη των σχεδιαστικών συμβιβασμών που απαιτούνται. Το βασικότερο συμπέρασμα είναι ότι η πρόοδος στην έρευνα για το κέλυφος εξαρτάται από την ύπαρξη κοινών, ρεαλιστικών και αναπαραγωγίμων πλαισίων αξιολόγησης. Το ΚΟΑΛΑ διατίθεται ως εργαλείο ανοικτού κώδικα, ενώ η μόνιμη αρχειοθέτηση των δεδομένων εισόδου και των εξαρτήσεων διασφαλίζει τη μακροπρόθεσμη αναπαραγωγιμότητα των αποτελεσμάτων. Η παρούσα εργασία φιλοδοξεί να αποτελέσει σημείο αναφοράς, θέτοντας τις βάσεις για πιο συστηματική και συγκρίσιμη μελλοντική έρευνα στο πεδίο.

Βιβλιογραφία

- [1] Adam Drake. *Command-line Tools Can Be 235x Faster Than Your Hadoop Cluster*. Accessed: 2025-06-01. 2014.
- [2] *Arch User Repository (AUR)*. Accessed: 2025-01-13.
- [3] Armando Cerna. *Pacaur building script*. Accessed: 2025-01-13.
- [4] Bentley, Jon. “Programming pearls: a spelling checker”. Στο: *CACM* 28.5 (Μάι. 1985), 456–462. ISSN: 0001-0782. DOI: {10.1145/3532.315102}. URL: %7Bhttps://doi.org/10.1145/3532.315102%7D.
- [5] Bentley, Jon and Knuth, Don and McIlroy, Doug. “Programming pearls: a literate program”. Στο: *CACM* 29.6 (Ιούν. 1986), 471–483. ISSN: 0001-0782. DOI: {10.1145/5948.315654}. URL: %7Bhttps://doi.org/10.1145/5948.315654%7D.
- [6] Emery D. Berger. *Optimizing Shell Scripting Languages*. Αδημοσίευτη ερευνητική εργασία UMCS TR-2003-009. University of Massachusetts Amherst, 2003.
- [7] Stephen M. Blackburn κ.ά. “The DaCapo benchmarks: java benchmarking development and analysis”. Στο: *Proceedings of the 21st Annual ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages, and Applications*. OOPSLA ’06. Portland, Oregon, USA: Association for Computing Machinery, 2006, σσ. 169–190. ISBN: 1595933484. DOI: 10.1145/1167473.1167488. URL: https://doi.org/10.1145/1167473.1167488.
- [8] Brown University Department of Computer Science. *CSCI 1380: Distributed Computer Systems*. Accessed: 2025-06-04. 2025.
- [9] Cappellini, Enrico and Welker, Frido and Pandolfi, *et al.* “Early Pleistocene enamel proteome from Dmanisi resolves Stephanorhinus phylogeny”. Στο: *Nature* 574.7776 (Οκτ. 2019), 103–107. ISSN: 1476-4687. DOI: {10.1038/s41586-019-1555-y}. URL: %7Bhttps://www.nature.com/articles/s41586-019-1555-y%7D.
- [10] Charlie Curtsinger and Daniel W. Barowy. “Riker: Always-Correct and Fast Incremental Builds from Simple Specifications”. Στο: *2022 USENIX Annual Technical Conference (USENIX ATC 22)*. Carlsbad, CA: USENIX Association, Ιούλ. 2022, 885–898. ISBN: 978-1-939133-29-70. URL: %7Bhttps://www.usenix.org/conference/atc22/presentation/curtsinger%7D.
- [11] Christian Bienia and Sanjeev Kumar and Jaswinder Pal Singh and Kai Li. “The PARSEC Benchmark Suite: Characterization and Architectural Implications”. Στο: *Proceedings of the 17th International Conference on Parallel Architectures and Compilation Techniques (PACT ’08)*. New York, NY, USA: Association for Computing Machinery, 2008, 72–81. DOI: {10.1145/1454115.1454128}. URL: %7Bhttps://dl.acm.org/doi/10.1145/1454115.1454128%7D.
- [12] Valve Corporation. *Steam for Linux: Critical file deletion bug due to unset environment variables*. GitHub Issue Tracker. GitHub Issue #3671. 2015.
- [13] Charlie Curtsinger και Daniel W. Barowy. “Riker: Always-Correct and Fast Incremental Builds from Simple Specifications”. Στο: *2022 USENIX Annual Technical Conference (USENIX ATC 22)*. Carlsbad, CA: USENIX Association, Ιούλ. 2022, σσ. 885–898. ISBN: 978-1-939133-29-70. URL: https://www.usenix.org/conference/atc22/presentation/curtsinger.

- [14] Dong, Yiwen and Li, Zheyang and Tian, Yongqiang and Sun, Chengnian and Godfrey, Michael W. and Nagappan, Meiyappan. “Bash in the Wild: Language Usage, Code Smells, and Bugs”. Στο: *ACM Transactions on Software Engineering and Methodology* 32.1 (31 Ιαν. 2023), 1–22. ISSN: 1049-331X, 1557-7392. DOI: [10.1145/3517193](https://doi.org/10.1145/3517193). URL: <https://dl.acm.org/doi/10.1145/3517193%7D> (επίσκεψη 19/06/2024).
- [15] Durumeric, Zakir and Wustrow, Eric and Halderman, J. Alex. “ZMap: fast internet-wide scanning and its security applications”. Στο: *Proceedings of the 22nd USENIX Conference on Security*. SEC’13. Washington, D.C.: USENIX Association, 2013, 605–620. ISBN: 9781931971034. DOI: [10.5555/2534766.2534818](https://doi.org/10.5555/2534766.2534818).
- [16] Edward Tufte. *New York City Weather Chart*. Accessed: 2025-06-02. 2004.
- [17] Elias Dabbas. *Web Server Access Logs*. Accessed June 3, 2025. 2020.
- [18] Evangelos Lamprou. “Foundation Models and Unix”. Στο: *Paged Out!* 6 (Μαρ. 2025), σ. 9.
- [19] Gabriel, Richard P. *Performance and Evaluation of LISP Systems*. The MIT Press, Αύγ. 1985. ISBN: 9780262256193. DOI: [10.7551/mitpress/5298.001.0001](https://doi.org/10.7551/mitpress/5298.001.0001). URL: <https://doi.org/10.7551/mitpress/5298.001.0001%7D>.
- [20] Gemma Team. “Gemma 3 Technical Report”. Στο: (2025). arXiv: (cs.CL). URL: <https://arxiv.org/abs/2503.19786%7D>.
- [21] Github Inc. *The top programming languages*. 2022.
- [22] Michael Greenberg. *The POSIX Shell Is an Interactive DSL for Concurrency*. URL: <https://cs.pomona.edu/~michael/papers/dsldi2018.pdf>.
- [23] Michael Greenberg και Austin J. Blatt. “Executable formal semantics for the POSIX shell”. Στο: *Proceedings of the ACM on Programming Languages* 4.POPL (Δεκ. 2019). DOI: [10.1145/3371111](https://doi.org/10.1145/3371111). URL: <https://doi.org/10.1145/3371111>.
- [24] Michael Greenberg, Konstantinos Kallas και Nikos Vasilakis. “The Future of the Shell: Unix and Beyond”. Στο: *Workshop on Hot Topics in Operating Systems*. HotOS ’21. Ann Arbor, Michigan: Association for Computing Machinery, 2021, σσ. 240–241. ISBN: 9781450384384. DOI: [10.1145/3458336.3465296](https://doi.org/10.1145/3458336.3465296). URL: <https://doi.org/10.1145/3458336.3465296>.
- [25] Michael Greenberg, Konstantinos Kallas και Nikos Vasilakis. “Unix Shell Programming: The Next 50 Years”. Στο: *Workshop on Hot Topics in Operating Systems*. HotOS ’21. Ann Arbor, Michigan: Association for Computing Machinery, 2021, σσ. 104–111. ISBN: 9781450384384. DOI: [10.1145/3458336.3465294](https://doi.org/10.1145/3458336.3465294). URL: <https://doi.org/10.1145/3458336.3465294>.
- [26] Shivam Handa κ.ά. “An order-aware dataflow model for parallel Unix pipelines”. Στο: *International Conference on Functional Programming*. Τόμ. 5. New York, NY, USA: Association for Computing Machinery, Αύγ. 2021. DOI: [10.1145/3473570](https://doi.org/10.1145/3473570). URL: <https://doi.org/10.1145/3473570>.
- [27] Zhicheng Huang κ.ά. “Fractal: Fault-Tolerant Shell-Script Distribution”. Στο: *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*. Renton, WA: USENIX Association, Μάι. 2026.
- [28] Ibrahim, Fadhl and Oppelt, Julian and Maragkakis, Manolis and Mourelatos, Zissimos. “TERA-Seq: true end-to-end sequencing of native RNA molecules for transcriptome characterization”. Στο: *Nucleic Acids Research* 49.20 (2021), e115. DOI: [10.1093/nar/gkab713](https://doi.org/10.1093/nar/gkab713).
- [29] Israel, Abebe. *VPS Audit*. Accessed: 2025-01-13.
- [30] Jon Puritz. *Bio594: Using genomic techniques to examine the evolution of populations*. 2019.
- [31] Justine Tunney. *Bash One-Liners for LLMs*. Accessed: 2025-06-01. 2023.
- [32] Konstantinos Kallas κ.ά. “Practically Correct, Just-in-Time Shell Script Parallelization”. Στο: *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. Carlsbad, CA: USENIX Association, Ιούλ. 2022, σσ. 769–785. ISBN: 978-1-939133-28-1. URL: <https://www.usenix.org/conference/osdi22/presentation/kallas>.

- [33] Kenneth Ward Church. *Unix for Poets*. 1994. URL: <https://web.stanford.edu/class/cs124/kwc-unix-for-poets.pdf>.
- [34] Kirillov, Alexander and Mintun, Eric and Ravi, Nikhila and Mao, Hanzi and Rolland, Chloe and Gustafson, Laura and Xiao, Tete and Whitehead, Spencer and Berg, Alexander C. and Lo, Wan-Yen and Dollár, Piotr and Girshick, Ross. “Segment Anything”. Στο: *2023 IEEE/CVF International Conference on Computer Vision (ICCV)*. 2023, 3992–4003. DOI: [10.1109/ICCV51070.2023.00371](https://doi.org/10.1109/ICCV51070.2023.00371).
- [35] Kirk D. Lucas and Contributors. *UnixBench: The BYTE UNIX Benchmark Suite*. Accessed: 2025-04-28. 2012.
- [36] Koichi Nakashima. *ShellBench: A POSIX Shell Benchmark Suite*. Accessed: 2025-04-28. 2021.
- [37] Evangelos Lamprou κ.ά. “The Koala Benchmarks for the Shell: Characterization and Implications”. Στο: *2025 USENIX Annual Technical Conference (USENIX ATC '25)*. Boston, MA: USENIX Association, Ιούλ. 2025, σσ. 449–64. ISBN: 978-1-939133-48-9. URL: <https://www.usenix.org/conference/atc25/presentation/lamprou>.
- [38] Lukas Lazarek κ.ά. “From Ahead-of- to Just-in-Time and Back Again: Static Analysis for Unix Shell Programs”. Στο: *2025 Workshop on Hot Topics in Operating Systems. HotOS '25*. Banff, AB, Canada: Association for Computing Machinery, 2025, σσ. 88–95. ISBN: 9798400714757. DOI: [10.1145/3713082.3730395](https://doi.org/10.1145/3713082.3730395). URL: <https://doi.org/10.1145/3713082.3730395>.
- [39] Lengauer, Philipp and Bitto, Verena and Mössenböck, Hanspeter and Weninger, Markus. “A Comprehensive Java Benchmark Study on Memory and Garbage Collection Behavior of DaCapo, DaCapo Scala, and SPECjvm2008”. Στο: *Proceedings of the 8th ACM/SPEC on International Conference on Performance Engineering. ICPE '17*. L'Aquila, Italy: Association for Computing Machinery, 2017, 3–14. ISBN: 9781450344043. DOI: [10.1145/3030207.3030211](https://doi.org/10.1145/3030207.3030211). URL: <https://doi.org/10.1145/3030207.3030211>.
- [40] Georgios Liargkovas κ.ά. “Executing Shell Scripts in the Wrong Order, Correctly”. Στο: *19th Workshop on Hot Topics in Operating Systems. HotOS '23*. Providence, RI, USA: Association for Computing Machinery, 2023, σσ. 103–109. ISBN: 9798400701955. DOI: [10.1145/3593856.3595891](https://doi.org/10.1145/3593856.3595891). URL: <https://doi.org/10.1145/3593856.3595891>.
- [41] libdash developers. *libdash*.
- [42] Aurèle Mahéo, Pierre Sutra και Tristan Tarrant. “The serverless shell”. Στο: *Proceedings of the 22nd International Middleware Conference: Industrial Track. Middleware '21*. Québec city, Canada: Association for Computing Machinery, 2021, σσ. 9–15. ISBN: 9781450391528. DOI: [10.1145/3491084.3491426](https://doi.org/10.1145/3491084.3491426). URL: <https://doi.org/10.1145/3491084.3491426>.
- [43] Mahéo, Aurèle and Sutra, Pierre and Tarrant, Tristan. “The serverless shell”. Στο: *Proceedings of the 22nd International Middleware Conference: Industrial Track. Middleware '21*. Québec city, Canada: Association for Computing Machinery, 2021, 9–15. ISBN: 9781450391528. DOI: [10.1145/3491084.3491426](https://doi.org/10.1145/3491084.3491426). URL: <https://doi.org/10.1145/3491084.3491426>.
- [44] Marek Majkowski. *When Bloom filters don't bloom*. Accessed: 2025-01-13. Map. 2020.
- [45] Merkel, Dirk. “Docker: lightweight linux containers for consistent development and deployment”. Στο: *Linux journal* 2014.239 (2014), σ. 2.
- [46] Michael S. Hart and Project Gutenberg. *Project Gutenberg*. 1971.
- [47] Tammam Mustafa κ.ά. “DiSh: Dynamic Shell-Script Distribution”. Στο: *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. Boston, MA: USENIX Association, Απρ. 2023, σσ. 341–356. ISBN: 978-1-939133-33-5. URL: <https://www.usenix.org/conference/nsdi23/presentation/mustafa>.
- [48] *National Oceanic and Atmospheric Administration (NOAA)*. Accessed: 2025-01-13.
- [49] Netresec. *Publicly available PCAP files*. Accessed: 2025-06-02. 2025.

- [50] *npm.js*. Accessed: 2025-01-13.
- [51] Ole Tange. “GNU Parallel - The Command-Line Power Tool”. Στο: *login: The USENIX Magazine* 36.1 (Φεβ. 2011), 42–47. URL: [%7Bhttps://doi.org/10.5281/zenodo.16303%7D](https://doi.org/10.5281/zenodo.16303%7D).
- [52] *Ollama: Run large language models locally*. Accessed: 2025-06-02. 2023.
- [53] Paszke, Adam and Gross, Sam and Massa, Francisco and Lerer, Adam and Bradbury, James and Chanan, Gregory and Killeen, Trevor and Lin, Zeming and Gimelshein, Natalia and Antiga, Luca and Desmaison, Alban and Köpf, Andreas and Yang, Edward and DeVito, Zach and Raison, Martin and Tejani, Alykhan and Chilamkurthy, Sasank and Steiner, Benoit and Fang, Lu and Bai, Junjie and Chintala, Soumith. “PyTorch: an imperative style, high-performance deep learning library”. Στο: *Proceedings of the 33rd International Conference on Neural Information Processing Systems*. Red Hook, NY, USA: Curran Associates Inc., 2019.
- [54] Pawan Bhandari. *Solutions to unixgame.io*. Accessed: 2020-04-14. 2020.
- [55] Pedregosa, Fabian and Varoquaux, Gaël and Gramfort, Alexandre and Michel, Vincent and Thirion, Bertrand and Grisel, Olivier and Blondel, Mathieu and Prettenhofer, Peter and Weiss, Ron and Dubourg, Vincent and Vanderplas, Jake and Passos, Alexandre and Cournapeau, David and Brucher, Matthieu and Perrot, Matthieu and Duchesnay, Édouard. “Scikit-learn: Machine Learning in Python”. Στο: *J. Mach. Learn. Res.* 12.null (Noέ. 2011), 2825–2830. ISSN: 1532-4435. DOI: [10.5555/1953048.2078195](https://doi.org/10.5555/1953048.2078195). URL: [%7Bhttps://dl.acm.org/doi/10.5555/1953048.2078195%7D](https://dl.acm.org/doi/10.5555/1953048.2078195%7D).
- [56] Peter, Stéphane. *makeself - Make self-extractable archives on Unix*. Accessed: 2025-01-13.
- [57] Poess, Meikel and Floyd, Chris. “New TPC benchmarks for decision support and web commerce”. Στο: *SIGMOD Rec.* 29.4 (Δεκ. 2000), 64–71. ISSN: 0163-5808. DOI: [10.1145/369275.369291](https://doi.org/10.1145/369275.369291). URL: [%7Bhttps://doi.org/10.1145/369275.369291%7D](https://doi.org/10.1145/369275.369291%7D).
- [58] Raghavan, Deepti and Fouladi, Sadjad and Levis, Philip and Zaharia, Matei. “POSH: a data-aware shell”. Στο: *Proceedings of the 2020 USENIX Conference on Usenix Annual Technical Conference*. USENIX ATC’20. USA: USENIX Association, 2020. ISBN: 978-1-939133-14-4. DOI: [10.5555/3489146.3489188](https://doi.org/10.5555/3489146.3489188). URL: [%7Bhttps://dl.acm.org/doi/10.5555/3489146.3489188%7D](https://dl.acm.org/doi/10.5555/3489146.3489188%7D).
- [59] Ritchie, Dennis M. and Thompson, Ken. “The UNIX time-sharing system”. Στο: *CACM* 17.7 (Ιούλ. 1974), 365–375. ISSN: 0001-0782. DOI: [10.1145/361011.361061](https://doi.org/10.1145/361011.361061). URL: [%7Bhttps://doi.org/10.1145/361011.361061%7D](https://doi.org/10.1145/361011.361061%7D).
- [60] S. Greenberg and I.H. Witten. “Directing the User Interface: How People Use Command-Based Computer Systems”. Στο: *IFAC Proceedings Volumes* 21.5 (1988). 3rd IFAC Conference on Analysis, Design and Evaluation of Man-Machine Systems 1988, Oulu, Finland, 14-16 June 1988, 349–355. ISSN: 1474-6670. DOI: [https://doi.org/10.1016/S1474-6670\(17\)53932-4](https://doi.org/10.1016/S1474-6670(17)53932-4). URL: [%7Bhttps://www.sciencedirect.com/science/article/pii/S1474667017539324%7D](https://www.sciencedirect.com/science/article/pii/S1474667017539324%7D).
- [61] Sadjad Fouladi and Francisco Romero and Dan Iter and Qian Li and Shuvo Chatterjee and Christos Kozyrakis and Matei Zaharia and Keith Winstein. “From Laptop to Lambda: Outsourcing Everyday Jobs to Thousands of Transient Functional Containers”. Στο: *2019 USENIX Annual Technical Conference (USENIX ATC 19)*. Renton, WA: USENIX Association, Ιούλ. 2019, 475–488. ISBN: 978-1-939133-03-8. URL: [%7Bhttp://www.usenix.org/conference/atc19/presentation/fouladi%7D](http://www.usenix.org/conference/atc19/presentation/fouladi%7D).
- [62] Schröder, Michael and Cito, Jürgen. “An empirical investigation of command-line customization”. Στο: *Empirical Software Engineering* 27.2 (14 Δεκ. 2021), σ. 30. DOI: [10.1007/s10664-021-10036-y](https://doi.org/10.1007/s10664-021-10036-y). URL: [%7Bhttps://doi.org/10.1007/s10664-021-10036-y%7D](https://doi.org/10.1007/s10664-021-10036-y%7D).

- [63] Jiasi Shen, Martin Rinard και Nikos Vasilakis. “Automatic synthesis of parallel unix commands and pipelines with KumQuat”. Στο: *Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*. PPOPP ’22. Seoul, Republic of Korea: Association for Computing Machinery, 2022, σσ. 431–432. ISBN: 9781450392044. DOI: [10 . 1145 / 3503221 . 3508400](https://doi.org/10.1145/3503221.3508400). URL: <https://doi.org/10.1145/3503221.3508400>.
- [64] Simon Willison. *LLM: A CLI tool and Python library for interacting with Large Language Models*. Accessed: 2025-06-02.
- [65] Spinellis, Diomidis and Fragkoulis, Marios. “Extending Unix Pipelines to DAGs”. Στο: *IEEE Transactions on Computers* 66.9 (2017), 1547–1561. DOI: [10.1109/TC.2017.2695447](https://doi.org/10.1109/TC.2017.2695447). URL: [7Bhttps://ieeexplore.ieee.org/document/79035797D](https://ieeexplore.ieee.org/document/7903579).
- [66] Spradling, Cloyce D. “SPEC CPU2006 benchmark tools”. Στο: *ACM SIGARCH Computer Architecture News* 35.1 (2007), 130–134.
- [67] Taylor, Dave and Perry, Brandon. *Wicked Cool Shell Scripts: 101 Scripts for Linux, OS X, and UNIX Systems*. No Starch Press, 2016. ISBN: 978-1-59327-602-7.
- [68] The Open Group. *VSCPCTS 2016 Test Suite*. Accessed: 2025-01-01.
- [69] The SAM/BAM Format Specification Working Group. *Sequence Alignment/Map Format Specification*. Version 1.6, last modified on 6 Nov 2024. Accessed: 2025-01-13.
- [70] Tom White. *Hadoop: The Definitive Guide*. O’Reilly Media, Inc, 2009. ISBN: 9780596521974.
- [71] Tsaliki, Eleftheria and Spinellis, Diomedes. *The Real Numbers for Athens Buses*. 2020. URL: [7Bhttps://insidestory.gr/article/noymera-leoforeia-athinas7D](https://insidestory.gr/article/noymera-leoforeia-athinas).
- [72] U.S. Securities and Exchange Commission. *EDGAR Log File Data Sets*. Accessed June 3, 2025. 2024.
- [73] Nikos Vasilakis κ.ά. “PaSh: Light-Touch Data-Parallel Shell Processing”. Στο: *16th European Conference on Computer Systems*. EuroSys ’21. Online Event, United Kingdom: Association for Computing Machinery, 2021, σσ. 49–66. ISBN: 9781450383349. DOI: [10.1145/3447786.3456228](https://doi.org/10.1145/3447786.3456228). URL: <https://doi.org/10.1145/3447786.3456228>.
- [74] Vasilakis, Nikos and Kallas, Konstantinos and Mamouras, Konstantinos and Benetopoulos, Achilles and Cvetković, Lazar. “PaSh: light-touch data-parallel shell processing”. Στο: *Proceedings of the Sixteenth European Conference on Computer Systems*. EuroSys ’21. Online Event, United Kingdom: Association for Computing Machinery, 2021, 49–66. ISBN: 9781450383349. DOI: [10 . 1145 / 3447786 . 3456228](https://doi.org/10.1145/3447786.3456228). URL: [7Bhttps://doi.org/10.1145/3447786.34562287D](https://doi.org/10.1145/3447786.3456228).
- [75] Vasilakis, Nikos and Staicu, Cristian-Alexandru and Ntousakis, Grigoris and Kallas, Konstantinos and Karel, Ben and DeHon, André and Pradel, Michael. “Preventing Dynamic Library Compromise on node via RWX-Based Privilege Reduction”. Στο: *ACM Conference on Computer and Communications Security (CCS)*. 2021, 1821–1838.
- [76] Zhao, Tianqi and Kong, Ming and Liang, Tian and Zhu, Qiang and Kuang, Kun and Wu, Fei. “CLAP: Contrastive Language-Audio Pre-training Model for Multi-modal Sentiment Analysis”. Στο: *Proceedings of the 2023 ACM International Conference on Multimedia Retrieval*. ICMR ’23. Thessaloniki, Greece: Association for Computing Machinery, 2023, 622–626. ISBN: 9798400701788. DOI: [10.1145/3591106.3592296](https://doi.org/10.1145/3591106.3592296). URL: [7Bhttps://doi.org/10.1145/3591106.35922967D](https://doi.org/10.1145/3591106.3592296).

Παράρτημα Α

Παραδείγματα Σεναρίων Υποδομής για το Σύνολο Μετροπρογραμμάτων weather

```
1  #!/bin/bash
2
3  sudo apt-get update
4  sudo apt-get install -y --no-install-recommends curl \
5      wget \
6      unzip \
7      coreutils \
8      gzip \
9      gawk \
10     sed \
11     findutils \
12     git \
13     python3 \
14     python3-pip \
15     python3-venv
16
17  pip install --break-system-packages --upgrade pip
18  pip install --break-system-packages \
19     numpy \
20     matplotlib
```

Κώδικας Α.1: Παράδειγμα σεναρίου `install.sh` για το σύνολο μετροπρογραμμάτων weather.

```
1  #!/bin/bash
2
3  TOP=$(git rev-parse --show-toplevel)
4
5  eval_dir="${TOP}/weather"
6  input_dir="${eval_dir}/inputs"
7
8  URL='https://atlas.cs.brown.edu/data'
9  URL=$URL/max-temp
10 FROM=2000
11 TO=2015
12
13 n_samples=99999
14 suffix="full"
15
16 mkdir -p "${input_dir}"
17 size=full
18 for arg in "$@"; do
19     case "$arg" in
20         --small) size=small ;;
21         --min) size=min ;;
22         esac
23 done
```

```

23 done
24 if [[ "$size" == "min" ]]; then
25     if [[ -f "$input_dir/temperatures.min.txt" && -f "$input_dir/tuft_weather.$size.txt"
26         ↪ ]]; then
27         echo "Data already downloaded and extracted."
28         exit 0
29     fi
30     min_inputs="$eval_dir/min_inputs/"
31     mkdir -p "$input_dir"
32     cp -r "$min_inputs"/* "$input_dir/"
33     python3 $eval_dir/scripts/generate_input.py $input_dir/tuft_weather.$size.txt --size
34     ↪ $size
35     exit 0
36 fi
37 if [[ "$size" == "small" ]]; then
38     if [[ -f "$input_dir/temperatures.small.txt" && -f "$input_dir/tuft_weather.$size.txt"
39         ↪ ]]; then
40         echo "Data already downloaded and extracted."
41         exit 0
42     fi
43     data_url="${URL}/temperatures.small.tar.gz"
44     wget --no-check-certificate "$data_url" -O "$input_dir/temperatures.small.tar.gz" || {
45         echo "Failed to download $data_url"
46         exit 1
47     }
48     tar -xzf "$input_dir/temperatures.small.tar.gz" -C "$input_dir" --no-same-owner || {
49         echo "Failed to extract $input_dir/temperatures.small.tar.gz"
50         exit 1
51     }
52     rm "$input_dir/temperatures.small.tar.gz"
53     mv "$input_dir/inputs/temperatures.small.txt" "$input_dir/temperatures.small.txt"
54     rm -rf "$input_dir/inputs"
55     python3 $eval_dir/scripts/generate_input.py $input_dir/tuft_weather.$size.txt --size
56     ↪ $size
57     exit 0
58 fi
59 if [[ -f "$input_dir/temperatures.full.txt" && -f "$input_dir/tuft_weather.$size.txt" ]];
60 ↪ then
61     echo "Data already downloaded and extracted."
62     exit 0
63 fi
64 data_url="${URL}/temperatures.full.tar.gz"
65 wget --no-check-certificate "$data_url" -O "$input_dir/temperatures.full.tar.gz" || {
66     echo "Failed to download $data_url"
67     exit 1
68 }
69 tar -xzf "$input_dir/temperatures.full.tar.gz" -C "$input_dir" --no-same-owner || {
70     echo "Failed to extract $input_dir/temperatures.full.tar.gz"
71     exit 1
72 }
73 rm "$input_dir/temperatures.full.tar.gz"
74 mv "$input_dir/inputs/temperatures.full.txt" "$input_dir/temperatures.full.txt"
75 rm -rf "$input_dir/inputs"
76 python3 $eval_dir/scripts/generate_input.py $input_dir/tuft_weather.$size.txt --size $size

```

Κώδικας A.2: Παράδειγμα σεναρίου `fetch.sh` για το σύνολο μετροπρογραμμάτων `weather`.

```

1 #!/bin/bash
2
3 TOP=$(git rev-parse --show-toplevel)
4 eval_dir="${TOP}/weather"

```

```

5 outputs_dir="${eval_dir}/outputs"
6 scripts_dir="${eval_dir}/scripts"
7 input_dir="${eval_dir}/inputs"
8 export LC_ALL=C
9 size=full
10 selected_scripts=""
11 while (($#)); do
12     case $1 in
13         --small)    size=small ;;
14         --min)     size=min ;;
15         -s|--scripts)
16             shift
17             while (($#) && [[ $1 != -* ]]; do
18                 selected_scripts+=" $1"
19                 shift
20             done
21             continue
22         ;;
23     esac
24     shift
25 done
26
27 KOALA_SHELL=${KOALA_SHELL:-bash}
28 export BENCHMARK_CATEGORY="weather"
29
30 should_run() {
31     script_name=$1
32     if [ -z "$selected_scripts" ]; then
33         return 0
34     fi
35     for selected in $selected_scripts; do
36         if [ "$selected" = "$script_name" ]; then
37             return 0
38         fi
39     done
40     return 1
41 }
42 if should_run "max-temp"; then
43     echo "max-temp"
44     export input_file="${input_dir}/temperatures.$size.txt"
45     export statistics_dir="$outputs_dir/statistics.$size"
46     mkdir -p "$statistics_dir"
47     BENCHMARK_INPUT_FILE="$(realpath "$input_file")"
48     export BENCHMARK_INPUT_FILE
49     BENCHMARK_SCRIPT="$(realpath "${scripts_dir}/temp-analytics.sh")"
50     export BENCHMARK_SCRIPT
51     $KOALA_SHELL "$scripts_dir/temp-analytics.sh"
52     echo "$?"
53 fi
54 if should_run "tuft-weather"; then
55     echo "tuft-weather"
56     export BENCHMARK_SCRIPT="$scripts_dir/tuft-weather.sh"
57     export BENCHMARK_INPUT_FILE="$input_dir/tuft_weather.${size}.txt"
58     mkdir -p "$outputs_dir/$size"
59     $KOALA_SHELL "$BENCHMARK_SCRIPT" "$BENCHMARK_INPUT_FILE" "$size" >
60     ↪ "$outputs_dir/$size/turf_weather.log"
61     echo "$?"
62     rm -rf "$outputs_dir/$size/plots" || true
63     mkdir -p "$outputs_dir/$size/plots"
64     if [ -d "$eval_dir/plots" ]; then
65         mv "$eval_dir/plots"/* "$outputs_dir/$size/plots/"
66     fi
67 fi

```

Κώδικας A.3: Παράδειγμα σεναρίου `execute.sh` για το σύνολο μετροπρογραμμάτων `weather`.

```
1  #!/bin/bash
2
3  set -e
4
5  TOP=$(git rev-parse --show-toplevel)
6
7  eval_dir="${TOP}/weather"
8
9  size="full"
10 generate=false
11 selected_scripts=""
12
13 while (($#)); do
14     case $1 in
15         --generate) generate=true ;;
16         --small)    size=small ;;
17         --min)      size=min ;;
18         -s|--scripts)
19             shift
20             while (($#) && [[ $1 != -* ]]; do
21                 selected_scripts+=" $1"
22                 shift
23             done
24             continue
25         ;;
26     esac
27     shift
28 done
29
30 statistics_dir="${eval_dir}/outputs/statistics.$size"
31 correct_dir="${eval_dir}/correct-results/statistics.$size"
32
33 should_run() {
34     script_name=$1
35     if [ -z "$selected_scripts" ]; then
36         return 0
37     fi
38     for selected in $selected_scripts; do
39         if [ "$selected" = "$script_name" ]; then
40             return 0
41         fi
42     done
43     return 1
44 }
45 if $generate; then
46     if should_run "max-temp"; then
47         mkdir -p "$correct_dir"
48         cp -r "$statistics_dir"/* "$correct_dir"
49     fi
50     if should_run "tuft-weather"; then
51         hash_dir="$eval_dir/hashes/$size"
52         hash_file="$hash_dir/tuft-weather.hash"
53         plot_root="$eval_dir/outputs/$size/plots"
54         mkdir -p "$hash_dir"
55         find "$plot_root" -type f -name '*.png' ! -path '*/tmp/*' -print0 |
56             sort -z | tr '\0' '\n' > "$hash_file"
57     fi
58     exit 0
59 fi
60
```

```

61 if should_run "max-temp"; then
62     diff -q "$statistics_dir/average.txt" "$correct_dir/average.txt"
63     echo average.$size $?
64     diff -q "$statistics_dir/min.txt" "$correct_dir/min.txt"
65     echo min.$size $?
66     diff -q "$statistics_dir/max.txt" "$correct_dir/max.txt"
67     echo max.$size $?
68 fi
69
70 if should_run "tuft-weather"; then
71     hash_dir="$eval_dir/hashes/$size"
72     hash_file="$hash_dir/tuft-weather.hash"
73     plot_root="$eval_dir/outputs/$size/plots"
74     all_exist=true
75     while IFS= read -r filepath; do
76         if [ ! -f "$filepath" ]; then
77             echo "Missing: $filepath"
78             all_exist=false
79         fi
80     done < "$hash_file"
81     if [ "$all_exist" = true ]; then
82         echo "tuft-weather 0"
83     else
84         echo "tuft-weather 1"
85     fi
86 fi

```

Κώδικας A.4: Παράδειγμα σεναρίου `validate.sh` για το σύνολο μετροπρογραμμάτων `weather`.

```

1  #!/bin/bash
2
3  for arg in "$@"; do
4      case "$arg" in
5          "-f") force=true ;;
6      esac
7  done
8  TOP=$(git rev-parse --show-toplevel)
9  input_dir="$TOP/weather/inputs"
10 outputs_dir="$TOP/weather/outputs"
11 rm -rf "$outputs_dir"
12 if [ "$force" = true ]; then
13     rm -rf "$input_dir"
14 fi

```

Κώδικας A.5: Παράδειγμα σεναρίου `clean.sh` για το σύνολο μετροπρογραμμάτων `weather`.

Παράρτημα Β

Πλήρες Πρόγραμμα Κελύφους που Παράγεται από το PASH για το παράδειγμα weather.

```
1  #!/bin/bash
2  cd "$(dirname "${0}")"
3  [ -z "${PASH_TOP}" ]
4  rm_pash_fifos() {
5      rm -f /tmp/xUz5/fifo9
6      rm -f /tmp/xUz5/fifo10
7      # ...similarly for the rest of the fifos...
8  }
9  mkfifo_pash_/tmp/xUz5/fifos() {
10     mk/tmp/xUz5/fifo /tmp/xUz5/fifo9
11     mk/tmp/xUz5/fifo /tmp/xUz5/fifo10
12     # ...similarly for the rest of the fifos...
13 }
14 rm_pash_/tmp/xUz5/fifos; mk/tmp/xUz5/fifo_pash_/tmp/xUz5/fifos; pids_to_kill=""
15 d="./data/temperatures"
16
17 { r_split "./data/temperatures/2000" 1000000 \ /tmp/xUz5/fifo9 /tmp/xUz5/fifo10 & }
18 pids_to_kill="${!} ${pids_to_kill}"
19 { r_wrap bash -c ' cut -c 89-92 ' 0< /tmp/xUz5/fifo9 > /tmp/xUz5/fifo11 & }
20 pids_to_kill="${!} ${pids_to_kill}"
21 { r_wrap bash -c ' cut -c 89-92 ' 0< /tmp/xUz5/fifo10 > /tmp/xUz5/fifo12 & }
22 pids_to_kill="${!} ${pids_to_kill}"
23 { r_wrap bash -c ' grep -v 999 ' 0< /tmp/xUz5/fifo11 > /tmp/xUz5/fifo13 & }
24 pids_to_kill="${!} ${pids_to_kill}"
25 { r_wrap bash -c ' grep -v 999 ' 0< /tmp/xUz5/fifo12 > /tmp/xUz5/fifo14 & }
26 pids_to_kill="${!} ${pids_to_kill}"
27 { r_unwrap 0< /tmp/xUz5/fifo13 > /tmp/xUz5/fifo15 & }
28 pids_to_kill="${!} ${pids_to_kill}"
29 { r_unwrap 0< /tmp/xUz5/fifo14 > /tmp/xUz5/fifo16 & }
30 pids_to_kill="${!} ${pids_to_kill}"
31 { dgsh-tee -i /tmp/xUz5/fifo15 -o /tmp/xUz5/fifo20 -I -f -b 5M & }
32 pids_to_kill="${!} ${pids_to_kill}"
33 { dgsh-tee -i /tmp/xUz5/fifo16 -o /tmp/xUz5/fifo21 -I -f -b 5M & }
34 pids_to_kill="${!} ${pids_to_kill}"
35 { sort -r -n 0< /tmp/xUz5/fifo20 > /tmp/xUz5/fifo17 & }
36 pids_to_kill="${!} ${pids_to_kill}"
37 { sort -r -n 0< /tmp/xUz5/fifo21 > /tmp/xUz5/fifo18 & }
38 pids_to_kill="${!} ${pids_to_kill}"
```

```

39 { dgsh-tee -i /tmp/xUz5/fifo17 -o /tmp/xUz5/fifo22 -I -f -b 5M & }
40 pids_to_kill="${!} ${pids_to_kill}"
41 { dgsh-tee -i /tmp/xUz5/fifo18 -o /tmp/xUz5/fifo23 -I -f -b 5M & }
42 pids_to_kill="${!} ${pids_to_kill}"
43 { sort -r -n -m /tmp/xUz5/fifo22 /tmp/xUz5/fifo23 > /tmp/xUz5/fifo6 & }
44 pids_to_kill="${!} ${pids_to_kill}"
45 { head -n 1 0< /tmp/xUz5/fifo6 > "max.2000" & }
46 pids_to_kill="${!} ${pids_to_kill}"
47 #...
48 # For average calculation, replace the last two steps with:
49 r_merge intermediate_stream1 intermediate_stream2 > merged_stream
50 # Process stateful commands sequentially (Reduce)
51 awk '{t+=$1; i++;} END {if (i>0) print t/i}' < merged_stream > "avg.2000"
52 # Wait for completion, cleanup FIFOs, and exit
53 source wait_for_output_and_sigpipe_rest.sh ${!}; rm_pash_fifos;
54 (exit "${internal_exec_status}")

```

Κώδικας Β.1: Πλήρες πρόγραμμα κελύφους που παράγεται από το PASH για το παράδειγμα weather (-width=2).

Παράρτημα C

Κώδικας Συγκριτικής Αξιολόγησης

C.1 Μετροπρόγραμμα count_trigrams

```
1  #!/bin/bash
2
3  IN=${IN:-$SUITE_DIR/inputs/pg}
4  OUT=${1:-$SUITE_DIR/outputs/4_3b/}
5  ENTRIES=${ENTRIES:-1000}
6  mkdir -p "$OUT"
7  pure_func() {
8      input=$1
9      TEMPDIR=$(mktemp -d)
10     cat > ${TEMPDIR}/${input}.words
11     tail +2 ${TEMPDIR}/${input}.words > ${TEMPDIR}/${input}.nextwords
12     tail +3 ${TEMPDIR}/${input}.words > ${TEMPDIR}/${input}.nextwords2
13     paste ${TEMPDIR}/${input}.words \${TEMPDIR}/${input}.nextwords
14     ↪ ${TEMPDIR}/${input}.nextwords2 |
15     sort | uniq -c
16     rm -rf ${TEMPDIR}
17 }
18 export -f pure_func
19 for input in $(ls ${IN} | head -n ${ENTRIES} | xargs -I arg1 basename arg1)
20 do
21     cat ${IN}/${input} | tr -c 'A-Za-z' '\n' | grep -v "^\\s*$" | pure_func $input >
22     ↪ ${OUT}/${input}.trigrams
23 done
```

Κώδικας C.1: Το αρχικό σενάριο count_trigrams. Χρησιμοποιεί προσωρινά αρχεία και σειριακή εκτέλεση.

```
1  #!/bin/sh
2
3  IN="$IN:-$SUITE_DIR/inputs/pg"
4  OUT="$1:-$SUITE_DIR/outputs/4_3b/"
5  ENTRIES="$ENTRIES:-1000"
6  mkdir -p "$OUT"
7  pure_func() {
8      input_file="$1"
9      output_file="$2"
10     job_id=$(basename "$input_file")
11     tmp_dir="$OUT/.tmp_${job_id}"; mkdir -p "$tmp_dir"
12     f_raw1="$tmp_dir/raw1"; f_raw2="$tmp_dir/raw2"; f_raw3="$tmp_dir/raw3"
13     f_tail2="$tmp_dir/tail2"; f_tail3="$tmp_dir/tail3"
```

```

14     mkfifo "$f_raw1" "$f_raw2" "$f_raw3" "$f_tail2" "$f_tail3"
15     tail -n +2 < "$f_raw2" > "$f_tail2" &
16     tail -n +3 < "$f_raw3" > "$f_tail3" &
17     paste "$f_raw1" "$f_tail2" "$f_tail3" | sort | uniq -c > "$output_file" &
18     agg_pid=$!
19     tr -c 'A-Za-z' '\n' < "$input_file" | \
20     grep -v '^[[:space:]]*$' | \
21     tee "$f_raw2" "$f_raw3" > "$f_raw1"
22     wait $agg_pid
23     rm -rf "$tmp_dir"
24 }
25 MAX_PROCS=$(nproc 2>/dev/null || echo 4)
26 job_count=0; processed_count=0
27 for file in "$IN"/*; do
28     [ -f "$file" ] || continue
29     if [ "$processed_count" -ge "$ENTRIES" ]; then
30         break
31     fi
32     input_name=${file##*/}
33     pure_func "$file" "$OUT/${input_name}.trigrams" &
34     job_count=$((job_count + 1))
35     if [ "$job_count" -ge "$MAX_PROCS" ]; then
36         wait; job_count=0
37     fi
38     processed_count=$((processed_count + 1))
39 done
40 wait

```

Κώδικας C.2: Το σενάριο `count_trigrams` βελτιστοποιημένο κατά Shark. Χρήση FIFOs και παραλληλισμού παρασκηνίου.

C.2 Μετροπρόγραμμα covid

```

1  #!/bin/bash
2
3  cat "$1" |
4      sed 's/T.....//' |
5      cut -d ',' -f 1,3 |
6      sort -u |
7      cut -d ',' -f 1 |
8      sort |
9      uniq -c |
10     awk "{print \$2, \$1}"

```

Κώδικας C.3: Το πρώτο σενάριο του συνόλου `covid`. Τυπική σωλήνωση επεξεργασίας.

```

1  #!/bin/bash
2
3  INPUT="$1"
4  MAX_PROCS=${MAX_PROCS:-$(nproc)}

```

```

5 chunk_size=${chunk_size:-100M}
6 process_chunk() {
7     sed 's/T.....//'|
8     cut -d ',' -f 1,3
9 }
10 export -f process_chunk
11
12 tmp_dir=$(mktemp -d)
13 trap "rm -rf $tmp_dir" EXIT
14
15 cat "$INPUT" | parallel --pipe --block "$chunk_size" -j "$MAX_PROCS" process_chunk >
16 ↪ "$tmp_dir/combined.tmp"
17
18 sort -u "$tmp_dir/combined.tmp" |
19     cut -d ',' -f 1 |
20     sort |
21     uniq -c |
22     awk '{print $2,$1}'

```

Κώδικας C.4: Το πρώτο σενάριο του συνόλου covid μετασχηματισμένο μέσω GNU parallel. Χειροκίνητη τμηματοποίηση και ορισμός συναρτήσεων.

C.3 Μετροπρόγραμμα spell

```

1 #!/bin/bash
2 # Calculate misspelled words in an input
3
4 dict=$SUITE_DIR/inputs/dict.txt
5
6 cat $1 |
7     sed 's/[^[:print:]]//g' |      # remove non-printing characters
8     col -bx |                     # remove backspaces / linefeeds
9     tr -cs A-Za-z '\n' |
10    tr A-Z a-z |                  # map upper to lower case
11    tr -d '[:punct:]' |           # remove punctuation
12    sort |                        # put words in alphabetical order
13    uniq |                        # remove duplicate words
14    comm -23 - $dict              # report words not in dictionary

```

Κώδικας C.5: Το αρχικό σενάριο spell. Έλεγχος ορθογραφίας.

```

1 #!/bin/bash
2 # Calculate misspelled words in an input
3
4 dict=$SUITE_DIR/inputs/dict.txt
5
6 TEMP_C1="/tmp/{/}.out1"
7 TEMP1=$(seq -w 0 $(( $JOBS - 1 )) | sed 's^+ /tmp/in+' | sed 's^/$.out1/' | tr '\n' ' ')
8 TEMP1=$(echo $TEMP1)
9 mkfifo $TEMP1

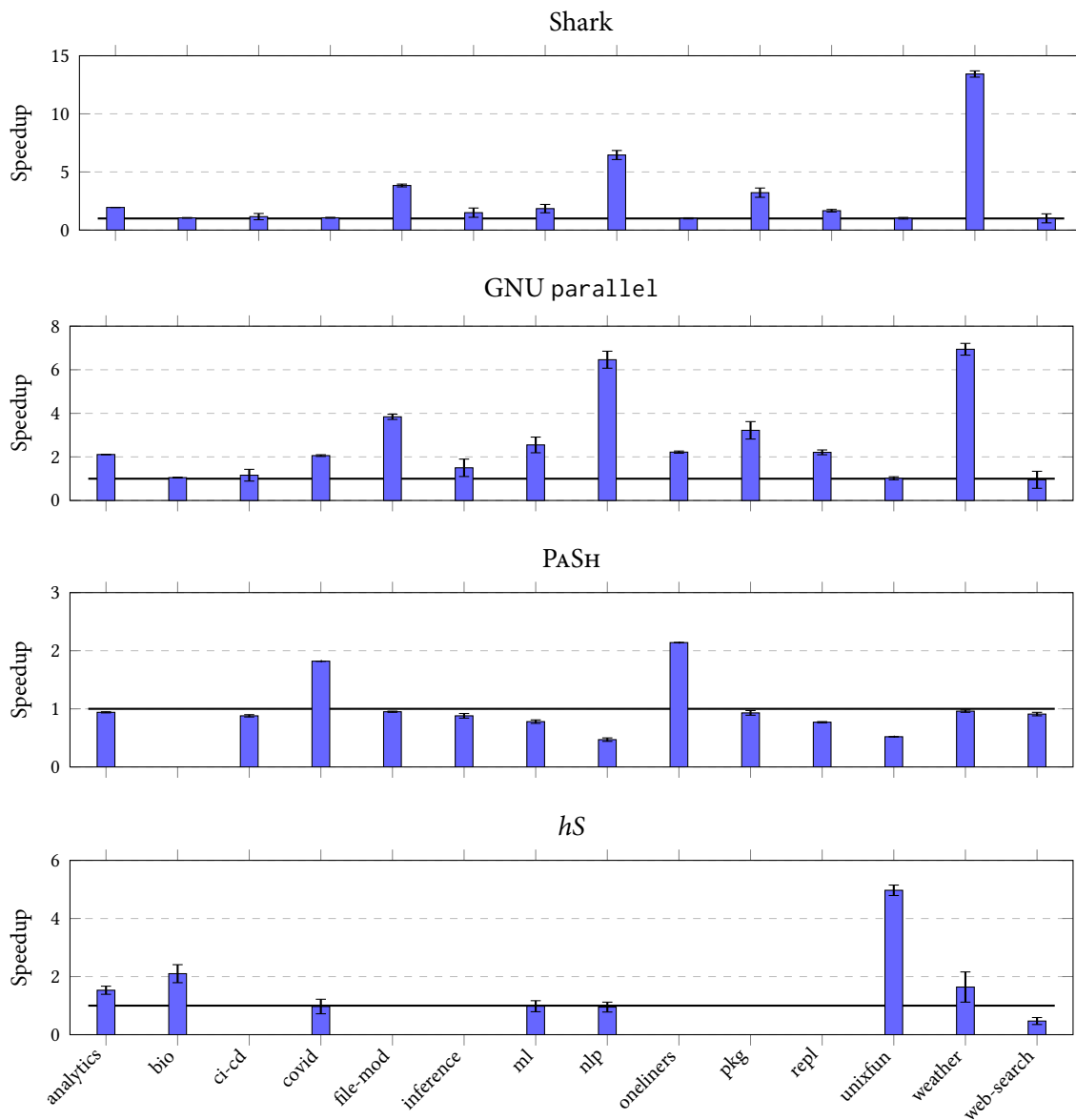
```

```
10 parallel "cat {} | col -bx | tr -cs A-Za-z '\n' | tr A-Z a-z | \  
11 tr -d '[:punct:]' | sort > $TEMP_C1" ::: $IN &  
12 sort -m $TEMP1 | parallel -k --jobs ${JOBS} --pipe --block "$BLOCK_SIZE" "uniq" |  
13 uniq | parallel -k --jobs ${JOBS} --pipe --block "$BLOCK_SIZE" "grep -vx -f $dict -"  
14 rm $TEMP1
```

Κώδικας C.6: Το σενάριο `spell` μετασχηματισμένο μέσω GNU `parallel`. Πολύπλοκη διαχείριση ροών και χαρακτήρων διαφυγής.

Παράρτημα D

Συγκριτικά Διαγράμματα Επιτάχυνσης



Σχήμα D.1: Συγκριτική επισκόπηση επιταχύνσεων. Τα διαγράμματα παρουσιάζουν τη σχετική επιτάχυνση (speedup) για τα 14 βασικά σύνολα μετροπρογραμμάτων. Στις περιπτώσεις που οι μπάρες απουσιάζουν, η εκτέλεση δεν υποστηρίζεται ή απέτυχε.